

Asymptotically Optimal Adaptive Asynchronous Common Coin and DKG with Silent Setup

Hanwen Feng and Qiang Tang

* School of Computer Science,
The University of Sydney, Australia
hanw.feng94@gmail.com, qiang.tang@sydney.edu.au

Abstract. We present the first optimal-resilient, adaptively secure asynchronous common coin protocol with $O(\lambda n^2)$ communication complexity and $O(1)$ rounds, requiring only a public silent setup. Our protocol immediately implies a sequence of quadratic-communication, constant-round asynchronous Byzantine agreement protocols, and also asynchronous distributed key generation with a silent setup. Along the way, we formulate a new primitive called *asynchronous subset alignment*, and introduce a simple framework to reason about specific composition security suitable for asynchronous common coin, enhancing security and functionality of silent-setup threshold encryption, which may be of independent interests.

1 Introduction

Common coin [17, 55], is a classical problem with significant modern applications [26, 36, 44, 54, 60, 67, 74]. At a high level, a group of n parties jointly execute a protocol, where all parties agree on one value that is “pseudorandom”. A closely related problem is distributed key generation (DKG) [9, 24, 43, 53, 61, 65, 78], in which each of the honest participants obtains a share of a secret key. It is well known that a DKG protocol immediately yields a common coin protocol.

In this work, we investigate the common coin (and DKG) problem in an *asynchronous setting*, where an adversary has the power to arbitrarily delay and reschedule messages between honest parties. We focus on improving the communication and round complexities of the protocol relative to the group size n . Beyond its theoretical interest, asynchronous common coin (and DKG) are critical components of asynchronous consensus protocols [5, 59, 60, 74, 77], which could involve a large number of participants, making scalability a key concern.

There has been a recent surge of interest in asynchronous common coins [12, 31, 82] and DKG protocols [3, 4, 33, 49, 68]. Unfortunately, *all* of them suffer from high communication complexity of at least $O(n^3)$. On the other hand, there is one striking exception: the seminal work of Cachin, Kursawe, and Shoup [22] achieved $O(\lambda n^2)$ via a unique threshold signature, but this inherently requires a *private* setup.¹ In a private setup, a trusted party disperses correlated secrets to each participant, which is significantly stronger than a common public setup, such as a public bulletin board, CRS, or PKI. Besides the drawbacks of private setups in distributed systems, the basic motivation of DKG is to eliminate such private setups. The following major question remains open,

Can we construct an asynchronous common coin (and DKG) protocol, that is with only $O(\lambda n^2)$ communication, secure against strongly adaptive adversary corrupting up to $\lceil \frac{n}{3} \rceil - 1$ parties, but, without relying on private setups?

¹ We summarize all existing asynchronous common coin, including DKG protocols, in Table 1, which clearly demonstrates the current status. More detailed explanations of broader developments are provided in Appendix A.

1.1 Our Contributions

In this work, we answer this question affirmatively.

Primary result: asymptotically optimal common coin without private setup. We present an asynchronous common coin protocol, ensuring that, except with negligible probability, all honest nodes can output a same value. This value is both unpredictable and unbiased, even in the presence of a computationally bounded adversary capable of adaptively corrupting up to $f = \lceil \frac{n}{3} \rceil - 1$ nodes. The communication complexity and round complexity of our protocol are $O(\lambda n^2)$ and $O(1)$ in expectation, respectively². Our protocol asymptotically matches the performance of Cachin et al.’s common coin [22], but requires only a silent setup.³ Also, we propose a more fine-grained design methodology with new, delicately abstracted key components with efficient and provable constructions.

Table 1. Comparison with optimal-resilient *asynchronous* common coin protocols⁴

Protocol	Type	Comm.Cost	Round	Adaptive?	Assumption	Setup
CKS [22]	Strong	$O(\lambda n^2)$	1	✓ [†]	RO, DDH, co-CDH	[†] Private Setup
DDL+ [31]	Weak	$O(\lambda n^3)$	$O(1)$	✗	RO	SecChan
FKT [82]	Weak ⁺	$O(\lambda n^3 \log n)$	$O(1)$	✗	DLog	SecChan
BBB+ [12]	Weak ⁺	$O(\lambda n^3 \log n)$ [‡]	$O(1)$	✓	RO	SecChan
KMS [68]	Strong	$O(\lambda n^4)$	$O(n)$	✗	RO, DDH	PKI
DYX+ [35]	Strong	$O(\lambda n^3)$	$O(\log n)$	✗	RO, DDH	PKI
AJM+ [4]	Strong	$O(\lambda n^3)$	$O(1)$	✓ [*]	RO, AGM, COMDL	PKI
GLL+ [49]	Strong	$O(\lambda n^3)$	$O(1)$	✓ [*]	RO, AGM, COMDL	PKI
Bingo [3]	Strong	$O(\lambda n^3)$	$O(1)$	✓	RO, AGM, q -SDH	CRS, SecChan
Ours	Strong	$O(\lambda n^2)$	$O(1)$	✓	RO, GGM	CRS, PKI ^{**}

– **Type:** **Strong** coins ensure *agreement* with $1 - \text{negl}(\lambda)$ probability. **Weak** coins ensure agreement with a constant probability $\phi \in (0, 1)$. **Weak**⁺ coins ensure agreement with $(1 - \text{negl}(\lambda) - \delta)$ probability for some adjustable small δ .

– **Comm.Cost** is the *expected* number of bits sent by all honest nodes, λ is the security parameter. **Round** measures the *expected* number of rounds within which all honest nodes can terminate. The costs of CKS and **Weak** or **Weak**⁺ coins are exact.

– [‡] BBB+ may reduce amortized cost to $O(\lambda n^2 \log n)$, we measure a *single-shot* coin cost.

– **Adaptive?** asks if the protocol is strongly adaptive secure [2].

[†]CKS is strongly adaptive secure if using an adaptively secure unique threshold signature scheme, for example, the scheme from [32] under DDH and co-CDH assumptions.

^{*}AJM+ and GLL+ proved the static security of their protocol, but their results can be adaptively secure by using a component from [11].

– **Assumption:** RO: Random Oracle; AGM: Algebraic Group Model [46]; GGM: Generic Group Model [73, 81]; q -SDH: q -Strong Diffie-Hellman Assumption [18, 64]; COMDL: Co-One-More Discrete Logarithm Assumption [11].

– **Setup:** SecChan: Secure Channels; CRS: Common Reference String. [3, 4, 35, 49, 68] may also need a uniformly distributed CRS, which can be implied by RO.

^{**} An alternative setup for us is registered PKI with a key curator. See Remark 2 for details.

² Our protocol achieves asymptotically better per-node computation cost of $\tilde{O}(n)$, compared to $\tilde{O}(n^2)$ in prior works without private setup, though the concrete cost may be higher at small scale due to heavier cryptographic tools.

³ In a silent setup, each party only posts *once* their public key (e.g., writes to the public bulletin board or sends it to a trusted party). This generalizes the conventional PKI. At the same time, a silent setup remains a public setup that disallows the distribution of secrets to individual parties, as discussed in Remark 1.

⁴ This table only includes computational asynchronous coin protocols, there are also asynchronous coins against information theoretic adversary, which, naturally have even higher complexity. Also, there are many synchronous

Direct implications: optimal Asynchronous Byzantine Agreements without private setup and near-optimal ADKG protocol. All existing asymptotically optimal asynchronous consensus protocols *assume* a quadratic communication asynchronous coin (or use [22] directly via private setup). This includes the binary version (ABA) [76], the multi-valued version (MBA) [77], the multi-valued validated version (MVBA) [72], and the asynchronous common subset (ACS) [22, 72]. Our common coin protocol can be directly plugged in, providing the first set of these asynchronous consensus protocols without a private setup.

Furthermore, very recently, [42] introduced an ADKG protocol in the *coin-aided* model,⁵ which generates field-elements secret keys, enjoys strong adaptive security (assuming adaptively secure coins), $O(\kappa\lambda n^2)$ communication complexity, and $O(1)$ round complexity. Here, κ is a parameter ensuring that the protocol’s statistical failure probability is at most $2^{-\kappa}$, which could be just a few tens in practice. Instantiating the coin oracle in [42] with our common coin protocol directly yields a full-fledged ADKG protocol with the same asymptotic complexity under a silent setup. In contrast, all existing ADKG protocols [3, 4, 35, 49, 68], as outlined in Table 1, require at least $O(\lambda n^3)$ communication cost⁶.

A simple framework for analyzing specific composition security. Since our common coin protocols involve several sub-protocols as components, we need to ensure that all components remain secure when composed. Most previous works on computational asynchronous common coins appear to have overlooked this issue. While the security properties of each component are defined and analyzed in a stand-alone setting, their security in composition is often taken for granted, and the final proofs are built upon this implicit assumption.⁷

We introduce a simple framework for analyzing the composability in specific forms (instead of UC, which might be unnecessarily heavy for now), which may be of independent interest. At a high level, we abstract the information an adversary could possibly learn from a protocol instance Π_A as an oracle $\mathcal{O}$. If a protocol instance Π_B remains secure even when the adversary has access to $\mathcal{O}$, then Π_B is secure in composition with Π_A . Our analysis method is essentially a generalization of the work by Lindell, Lysyanskaya, and Rabin [71]. Details are presented in Sect. 4.2.

Strengthening functionality and security for silent-setup threshold encryption. Along the way, we introduce *Silent Threshold Encryption with Tag-Homomorphism* (STHE), a variant of the Silent Threshold Encryption (STE) scheme recently proposed by Garg et al. [52]. STHE enhances STE by offering additional useful functionalities including **Tag-homomorphism** (ciphertexts under the same tag can be homomorphically evaluated) and **Tag-decryption** (ciphertexts under the same tag can be decrypted using one key). Moreover, STHE is defined to provide security against *stronger* adversaries who may further (1) obtain secret keys for arbitrary tags, and (2) adaptively corrupt participants based on such information. While STHE can be constructed

common coin and DKG protocols, most of which are also with cubic communication; only until very recently, we have the first synchronous coin and DKG protocols with subcubic communication [9, 43], still, both of them are not yet matching $O(\lambda n^2)$, and with $O(n)$ rounds.

⁵ We also sketch this construction in Appendix F.

⁶ An independent concurrent work [1] also achieves ADKG with subcubic communication, but with different approaches, including incomparable setups. Our constructions have better round and communication complexity. See Appendix A.1.

⁷ We show one asynchronous verifiable secret sharing [25, 34] (an important component) example, which is secure in the stand-alone setting but becomes *insecure* when two instances are run together (see Appendix C.1). While the example does not imply that existing constructions are insecure, when their components indeed need security beyond the stand-alone setting, security analysis does require more care.

through slight modifications to STE, its refined definition and strengthened security analysis may be of independent interest.

2 Technical Overview

Our goal is to construct a communication-optimal strong common coin in the asynchronous setting, which, except with negligible probability, ensures the following properties: (1) *termination*, that all honest nodes eventually terminate with a value; (2) *agreement*, that all honest nodes terminate with the same value; and (3) *unpredictability* and *bias-resistance*, that the agreed value is unpredictable and pseudorandom to the adversary.

Unique challenge in asynchronous common coin. In multi-party computation (MPC) [15], one critical step is an *input agreement* phase that all parties agree on a subset of “inputs” to continue the protocol. Otherwise, if parties decide to evaluate function on inputs from all parties (and wait for all relevant messages), one corrupted party could simply be silent, and the protocol may never *terminate*. Some variant of such *agreement* is similarly needed for common coin protocols. In fact, *assuming* an agreement module exists that can output a common subset of all inputs (large enough to include one honest input), the classical “commit-and-reveal” paradigm may already work, as in Fig. 1.

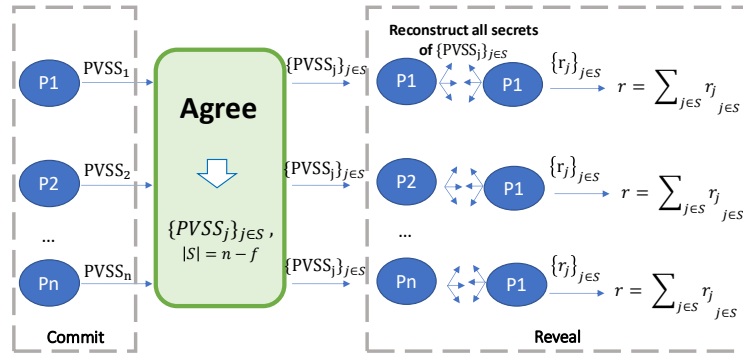


Fig. 1. A straw-man coin protocol (PVSS means a variant of verifiable secret sharing)

In general MPC, such *input agreement* is usually abstracted out, and could be realized by some variant of Byzantine agreement protocols. For example, in the synchronous setting, this can be implemented by applying *deterministic* Byzantine consensus [28, 38, 43]. However, due to the famous FLP impossibility [45], *asynchronous* consensus has to be *randomized*, relying on common coin.

This immediately creates a circularity issue for asynchronous common coin.

Existing methods. To avoid the circularity, asynchronous common coin protocols have to abandon the *agreement*. A sequence of seminal work, thus turn attentions to a weaker variant, asynchronous *weak* common coin. In a *weak* coin protocol, both *agreement* and *unpredictability* & *bias-resistance* may hold only with a constant probability, thus bringing the possibility of avoiding a full-fledged *agreement*. We overview a recent construction from [4] as an example (Fig. 2).

Gather, instead of agree. Indeed, following the pioneering work by Feldman [40] and Canetti, Rabin [25], almost all existing weak coin constructions [3, 4, 31, 33, 68] employ a same form of *weak* input

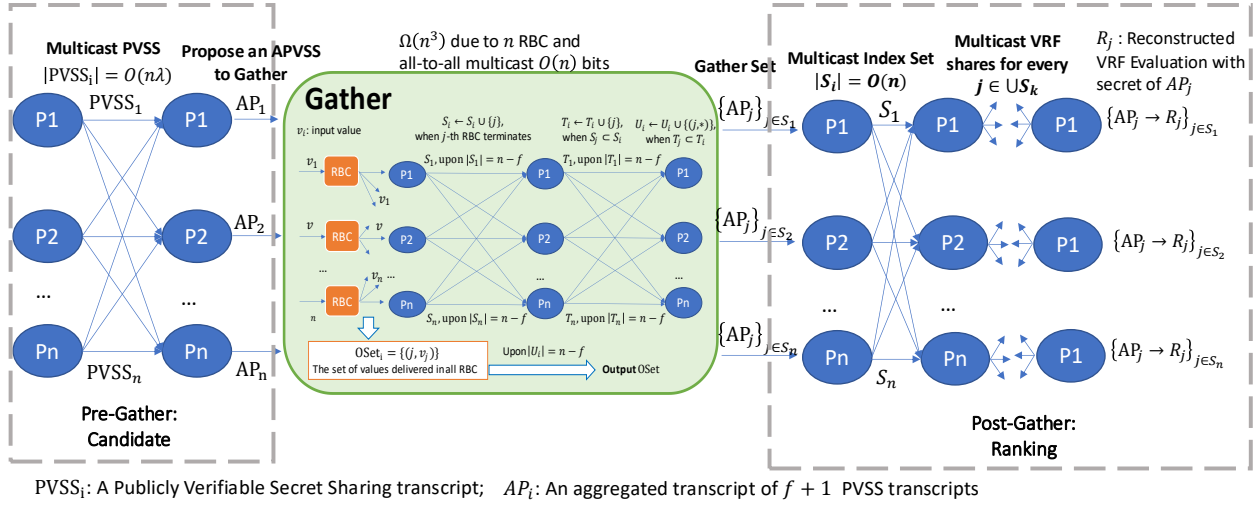


Fig. 2. The Weak Common Coin Protocol from Abraham et al. [4]

agreement, which was termed *information gathering* (Gather) since [4]. Informally, instead of letting all honest parties to *agree* on a common subset, *Gather* only requires the existence of a *binding core set* S^* with $|S^*| = n - f$, which is included in every honest party's output set (and determined at the moment the first honest node outputs).

With this weakening, *Gather* can be implemented with deterministic protocols [4, 31, 40]. But to leverage the core set provided by *Gather*, both the “commit” and “reveal” phases need to be strengthened, as now each party may output different set of values. At a high level, the following *candidate-ranking* mechanism [4, 25, 31, 40, 49] is employed for this purpose.

Pre-Gather: the candidate phase. Each node P_i needs to prepare a “committed candidate coin” c^* , which is still a commitment to a random value, but should be unpredictable even to P_i . Note that a committed candidate coin could be a collection of $f + 1$ commitments provided by different nodes, or the aggregation.

Post-Gather: the ranking phase. Now, every committed candidate coin in the *Gather* output set of P_i should be ranked in a deterministic yet unpredictable manner, and P_i will take the opening of the highest-ranked commitment as its output coin. Intuitively, as ensured by *Gather*, there are at most n committed candidate coins and a core set of $n - f$ candidates. Therefore, with a constant probability of roughly $\frac{n-f}{n}$ that the highest-ranked commitment is *in the core set*. If this occurs, every honest node will have the same common coin. Note that a commitment can be ranked by mapping its opening to a total-order set.

Cubic communication cost in every step. Unfortunately, both *Gather* and the ranking mechanism incur $\Omega(n^3)$ communication complexity.

Cubic cost for Gather. The $\Omega(n^3)$ cost in *Gather* arises for *multiple* reasons, which persists even when the candidate size (*Gather* input) is as small as $O(1)$. First, as illustrated in Fig. 2, the *Gather* protocol involves n instances of reliable broadcast (RBC) [20]. In addition, after the RBC phase, it follows an instance of a so-called index-*Gather* protocol [31] to determine which RBC instances should be included in the output set, in which every P_i needs to multicast the $O(n)$ -sized set of indexes it has received. Both the n RBC instances and the index-*Gather* incur $\Omega(n^3)$ communication cost.

On the other hand, a weaker version of Gather, known as the Weak Core Set, has been studied in [49], which only requires that the output sets of $f + 1$ honest nodes contain the core set. However, similar communication barriers persist.

Cubic cost for the candidate-ranking mechanism. Cubic communication cost also exists for both the candidate phase and the ranking phase.

- For the candidate phase, in existing protocols, the “commitment” scheme supporting forced opening is realized using (variants of) asynchronous verifiable secret sharing (AVSS) [3, 34]. An adaptively secure AVSS incurs $O(\lambda n^2)$ communication cost, leading to $\Omega(\lambda n^3)$ cost for preparing committed candidate coins, as $\Omega(n)$ instances of AVSS are utilized.

In addition, even with the aggregation techniques such as [11, 61], the candidate size is still as large as $O(n\lambda)$. As the Gather phase is to let all nodes receive $O(n)$ candidates, the $O(n\lambda)$ -sized candidate trivially leads to an inherent $\Omega(n^3\lambda)$ -bit communication cost during the Gather phase.

- For the ranking phase, it requires opening *all* candidates. Even if each node multicasts only $O(\lambda)$ bits for a candidate, the total cost amounts to $O(\lambda n^3)$.

Cubic cost compiling weak coin to strong coin. With a weak coin, one might construct an asynchronous consensus, thus plugging the simple framework in Fig 1 to get a strong coin. A cheapest way is to build a multi-valued validated asynchronous Byzantine agreement (MVBA) [21, 72]. However, this step currently also incurs a cubic cost. Most existing MVBA protocols fall into two categories: (1) Protocols such as [4, 31] that use a *weak* leader election subroutine, which, with constant probability, selects the same leader ι for all nodes (thus can be from weak coin). However, those MVBA schemes still incur $\Omega(\lambda n^3)$ communication complexity, even when ignoring the cost of the weak coin. (2) Protocols such as [5, 21, 72] that use an *ideal* leader election subroutine. While some of these protocols [5, 72] achieve quadratic communication (excluding the cost of leader election), implementing leader election using a weak coin requires additional effort. The state-of-the-art compiler [49] involves n instances of reliable broadcast (RBC), which inherently incurs $\Omega(n^3)$ communication complexity.

2.1 Our solutions to achieve quadratic communication

We reduce the communication complexity of all steps using a set of techniques that enable an asynchronous strong common coin with quadratic communication complexity. The execution flow of our weak coin solution is illustrated in Fig. 3, while our strong common coin is depicted in Fig. 5.

Align : a further weakening of Gather. Our first task is to eliminate the cubic cost of Gather. However, achieving a quadratic-communication Gather protocol appears to be challenging, if not impossible. Specifically, when a node outputs a set in Gather, it must be assured that a core set of $n - f$ values will be included in the output sets of *all* honest nodes. To achieve this, it seems necessary for each node to share its intended output with *all* other nodes, which inherently requires $\Omega(n^3)$ communication. This similarly applies to the Weak Core Set [49] (core set to be contained in $f + 1$ honest output set).

Fortunately, we observe that the guarantees provided by Gather or the Weak Core Set are still unnecessarily strong. Recall that the purpose of Gather is to enable a constant probability for all honest nodes to decide on the globally highest-ranked candidate. First, as also observed in [49], once $f + 1$ honest nodes have seen the globally highest-ranked candidate, we can ensure that all honest

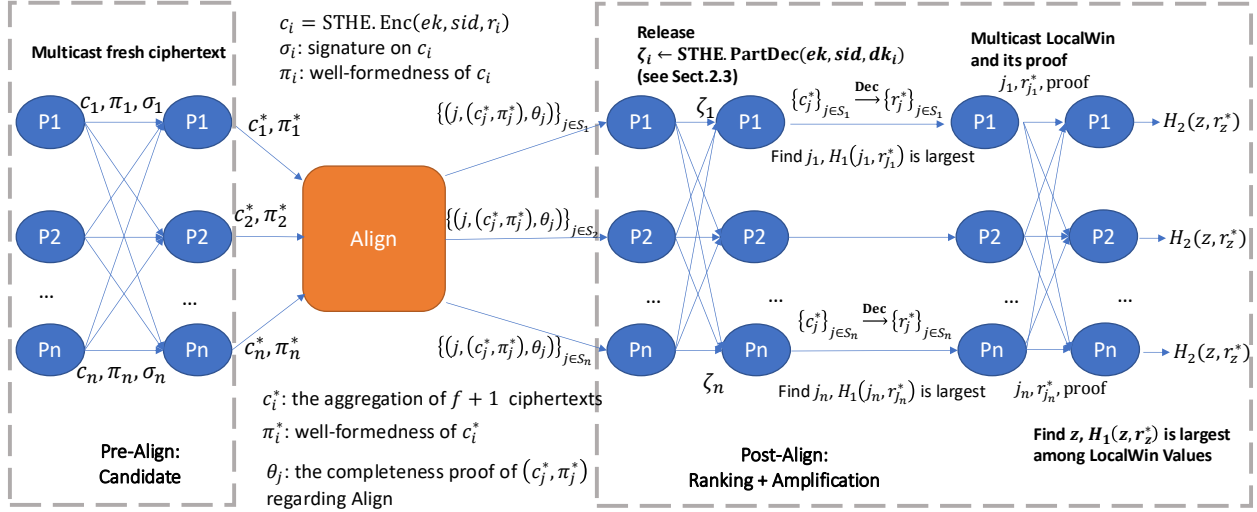


Fig. 3. Our Weak Common Coin Protocol

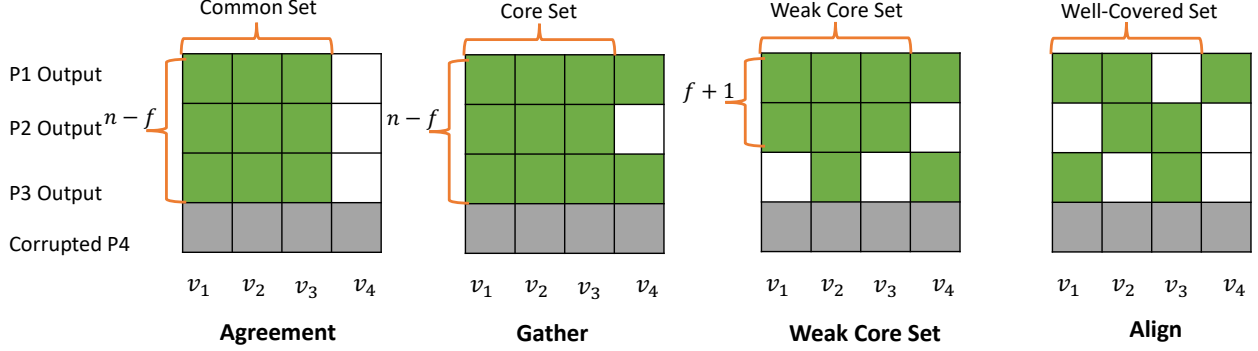


Fig. 4. The output guarantees in Agree, Gather, Weak Core Set, and Align

nodes can decide on this candidate, by performing an additional round of multicast for *amplification* after ranking. Second, we further observe that to ensure $f + 1$ nodes see the globally highest-ranked candidate with a certain probability, it is not necessary for the output sets of $f + 1$ nodes to contain a core set. Instead, this can be guaranteed by the existence of a large “well-covered” subset *WS*, where every *element* is “well-covered,” i.e., included in the output sets of at least $f + 1$ honest nodes. Unlike Gather or Weak Core Set, elements in *WS* can be covered by *different* subsets of $f + 1$ nodes.

Observing those, we introduce a new primitive termed Asynchronous Subset Alignment (Align). Fig.4 provides an example with four nodes, illustrating how the guarantees of Agree, Gather, the Weak Core Set, and Align differ. In addition to the existence of a “well-covered” set, Align also requires the existence of a *binding cover* as in [31], ensuring that all elements verifiable as valid Align outputs are known at the time the first node generates an output. This property is crucial to prevent an adversary from injecting new candidates once the ranking phase has started. A formal definition of Align is presented in Sect.5.1.

More importantly, we demonstrate that our weakening enables a solution with $O(1)$ rounds and $O(\ell n^2)$ communication (ℓ is input size). Our protocol carefully applies n parallel chains of three

consecutive provable broadcast (PB) [5, 22, 59] instances (fewer instances would result in a security issue) to realize Align. The details of our Align protocol can be found in Sect. 5.2.

Enabling quadratic candidate-ranking. Next, it becomes clear that the candidate-ranking mechanism needs to meet the following requirements: (1) The communication cost for generating *each* committed candidate coin must be $O(n\lambda)$; (2) The size of a committed candidate coin must be $\ell = O(\lambda)$, otherwise the cost of Align becomes cubic again; (3) Opening $O(n)$ committed candidate coins incurs at most $O(n^2\lambda)$ communication cost.

As we discussed, existing asynchronous common coin protocols exclusively use n concurrent (variants of) AVSS instances as the “commitment”, which leads to $\Omega(n^3)$ total communication cost. Instead, we shift to another tool, threshold decryption, to realize this purpose. Conventionally, threshold decryption requires a private setup. Recently, this perspective has changed due to Garg et al.’s silent-setup threshold encryption (STE) scheme [52].

However, a “plain” STE scheme only suffices for the requirement (1). For fully enjoying the benefits of an STE scheme, we consider a variant of it called STE with *tag-homomorphism* (STHE), also with strengthened security, which has the following useful features:

- *Tag-homomorphism* for (2): The encryption algorithm takes an additional input, *tag*, such that ciphertexts under the same *tag* can be homomorphically aggregated into a single compact ciphertext. In our protocol, *tag* corresponds to the instance identifier *sid*. With this property, these ciphertexts can be aggregated into a single compact ciphertext representing the candidate.
- *Tag-decryption* for (3): During the decryption phase, each node only needs to release an $O(\lambda)$ -sized partial decryption key for the corresponding *tag* (instead of $O(n\lambda)$ -sized shares). Once enough partial decryption keys are released, they can be aggregated into a complete decryption key for the *tag*. This key allows the decryption of all ciphertexts under that *tag*. This property reduces the ranking phase to $O(\lambda n^2)$ -bit communication cost.

Besides the more powerful functionalities, we define *stronger security* for STHE. In particular, under the same setup, the STHE might be used across multiple weak coin instances, such that the adversary can obtain different partial decryption keys from an honest node, which is not captured by the definition in [52]. A formal definition for STHE is provided in Sect. 4.1, and a construction (adapted from [52]) and analysis are presented in Appendix D.

Now, a committed candidate coin is represented as an aggregation of $f + 1$ ciphertexts (each encrypting a random number) contributed by distinct nodes. We need to ensure both that this aggregated ciphertext is well formed (thus can be jointly decrypted at a later stage), and the encrypted value of it is unknown to all participants including the adversary. Our overall strategy is to employ a SNARK [56] to prove that the ciphertext is an aggregation of $f + 1$ ciphertexts c_j , each signed by a distinct node. However, one more subtle issue arises: the aggregation of $f + 1$ ciphertexts may not encrypt a value unknown to the adversary. In particular, assume the only honest node among the $f + 1$ ciphertext creators may encrypt a random value v_c to c_z , and $\mathcal{A}$ might create a ciphertext c' on behalf of corrupted nodes such that the decryption of c' is related to r_z (e.g., $r_z + 1$), thereby revealing the final r^* to $\mathcal{A}$. To prevent such malleability attacks, we ensure that every fresh ciphertext c_j is accompanied by a validity proof π_j with respect to a simulation-extractable NIZK system [69]. Additionally, the SNARK proof must demonstrate that each fresh ciphertext being aggregated has a corresponding validity proof.

With these measures, we prove that the value encrypted by any ciphertext c^* with the specified SNARK proof remains unpredictable to the adversary, even under adaptive corruption. In partic-

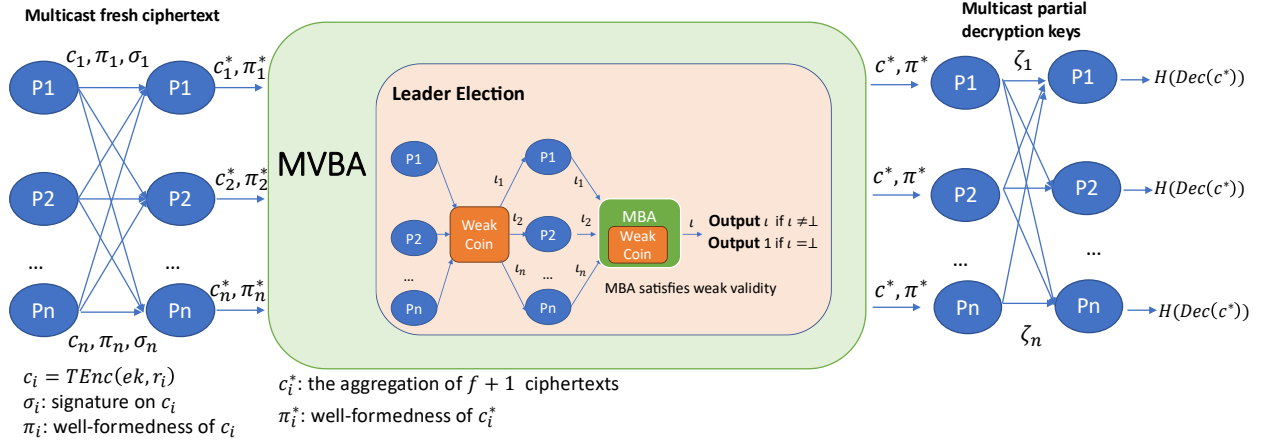


Fig. 5. From Asynchronous Weak Coin to Asynchronous Strong Common Coin

ular, we carefully apply the *single inconsistency party* proof technique [24, 32] to handle adaptive corruption without relying on memory erasures. More details are available in Sect. 6.1.

Cheaper conversion from weak coin to strong coin. After constructing a quadratic-communication weak common coin, we build a strong common coin through the following two steps: (1) Construct an MVBA protocol with quadratic communication complexity using our weak common coin; (2) Use the MVBA to enable the network to agree on a specific candidate committed coin, such that all honest nodes can obtain the coin value by opening this candidate, which, in our case, involves decrypting the ciphertext.

While there are quadratic-communication MVBA protocols, they all rely on a quadratic-communi-

cation leader election subroutine, which relies on a strong coin. [49] proposed a compiler from a weak common coin to leader election, which however, incurs an $O(\lambda n^3)$ -bit communication.

To address this gap, we propose a simpler compiler built upon asynchronous multivalued Byzantine agreement (MBA) [77] (which does not have external validity as in MVBA that the output value satisfies a predefined predicate). After receiving a value $\iota_i \in [n]$ from a weak common coin instance with output space $[n]$, each honest node P_i inputs ι_i into the MBA protocol. This ensures that, except with negligible probability, the network agrees on the same $\iota \in [n]$. If the MBA protocol returns $\perp$, the process simply returns $\iota = 1$. A prudent reader may notice that the MBA protocol [77] itself requires randomization, which can be readily supplied by concurrent instances of weak coins. A detailed construction and analysis are provided in Sect. 7.1.⁸

3 Models, Goals and Preliminary

Notations. Throughout the paper, let λ represent the bit length of the cryptographic security parameter. The notation $[i, n]$ denotes the set $\{i, i + 1, \dots, n\}$, where i and n are integers with $i < n$. We may abbreviate $[1, n]$ as $[n]$. For a set $\{x_1, x_2, \dots, x_n\}$ and a sequence $(x_1, x_2, \dots, x_n)$, we denote them as $\{x_i\}_{i \in [n]}$ and $(x_i)_{i \in [n]}$, respectively, for brevity. A function $f(n)$ is considered

⁸ We note that our compiler, apart from the weak common coin instance, does not require any setup or cryptographic operations. We expect it to be also useful in hash-only settings or information-theoretical settings.

negligible in n , denoted by $f(n) \leq \text{negl}(n)$, if for every positive integer c , there exists an n_0 such that for all $n > n_0$, $f(n) < n^{-c}$. Conversely, a non-negligible function satisfies $f(n) > \text{negl}(n)$. For a set $\mathbb{X}$, the notation $x \leftarrow \$ \mathbb{X}$ signifies sampling x uniformly from $\mathbb{X}$. Given a distribution X , $x \leftarrow X$ denotes sampling x from X . For a probabilistic algorithm A , $A(x_1, x_2, \dots; r)$ represents the result of running A with inputs $x_1, x_2, \dots$ and random coins r . We use $y \leftarrow A(x_1, x_2, \dots)$ to denote randomly choosing r and computing $A(x_1, x_2, \dots; r)$ to obtain the output y . We assume that every protocol instance is assigned a unique identifier **sid**, and the identifier of an instance serves as the prefix for the identifiers of its subroutines. The concatenation of **sid** and **sid**₀ is denoted as $\langle \text{sid}, \text{sid}_0 \rangle$, where **sid** is a prefix of this concatenated identifier.

Network and Adversary Models. We consider an *asynchronous* network where nodes are pairwise connected via *authenticated* channels, and we assume the identities of all participating nodes are publicly known. The adversary cannot omit, change, or inject messages sent by honest nodes. However, the adversary can reorder the messages and arbitrarily delay them, though it cannot delay them indefinitely. We focus on optimal resilience, meaning that the total number of corrupted nodes is at most $f = \lceil \frac{n}{3} \rceil - 1$.

We consider the *adaptive* adversaries which can adaptively corrupt nodes at any time during the protocol execution. Once a node is corrupted, the adversary gains access to its local state and controls its subsequent behavior. During execution, the nodes that have remained honest up to a given point are referred to as *so-far-honest* nodes, while those that remain honest until the end of execution are called *forever-honest* nodes. For simplicity, when we refer to honest nodes in this paper, we mean *forever-honest* nodes. In particular, an adaptive adversary can perform “after-the-fact-removal” attacks [2, 75], *i.e.*, corrupt a node and *remove* all messages the node just sent before they are delivered.

Silent Setup. Following [51, 52], we consider a setup phase to be *silent* if the setup can be performed with the help of a public bulletin board, to which each participant has only one-time writing access (and unbounded reading access). It is easy to observe that the conventional PKI setup and common reference string (CRS) setup fall into this category. In this work, we consider a slightly more general setup phase that can be described by the following three algorithms:

- $\text{CRS}(1^\lambda, n, t) \rightarrow \text{crs}$. On input the security parameter λ , the number of participants n , and the threshold number t , it generates a common reference string **crs** to be available to everyone.
- $\text{KeyGen}(\text{crs}, i) \rightarrow ((pk_i, \text{hint}_i), sk_i)$. A user P_i locally runs the algorithm to generate its own key pair (pk_i, sk_i) , and additionally a hint information hint_i . (pk_i, hint_i) can be viewed as an extended public key of P_i and will be posted on bulletin board.
- $\text{GroupPKGen}(\text{crs}, (pk_1, \text{hint}_1), \dots, (pk_n, \text{hint}_n)) \rightarrow pk$. Everyone can run the deterministic algorithm to compute the group public key pk .

For notational simplicity, we denote the above process as

$$\text{SilSetup}(1^\lambda, n, t) \rightarrow (\text{crs}, pk, (pk_i)_{i \in [n]}, (sk_i)_{i \in [n]}).$$

Furthermore, an adversary $\mathcal{A}$ is allowed to corrupt some participants during the setup phase (after seeing the CRS) and generate keys on behalf of the corrupted parties (after seeing the public keys of uncorrupted parties). We denote an execution of **SilSetup** involving $\mathcal{A}$ as

$$\text{SilSetup}^{\mathcal{A}}(1^\lambda, n, t) \rightarrow (\text{crs}, pk, (pk_i)_{i \in [n]}; \text{state}_{\mathcal{A}}; (sk_i)_{i \in [n] \setminus \mathcal{C}_{\text{init}}}),$$

where $\mathcal{C}_{\text{init}} \subset [n]$ is the set of initially corrupted parties' identities, and $\text{state}_{\mathcal{A}}$ is the state of $\mathcal{A}$ after the setup.

Remark 1. As discussed in [51], a silent setup does not enable the participants to run a DKG during the setup phase. In particular, even for the non-interactive DKG protocols [57, 65], the parties need to write to the bulletin board *twice*. First, they must post their public keys to the bulletin board. Then, based on the public keys of all other nodes, they generate transcripts, which also need to be posted to the bulletin board. In contrast, a silent setup allows only *one-time* writing access to the bulletin board, in which the nodes can join “asynchronously”: when each node arrives, they read the CRS and post their public key; the participants do not need to coordinate with each other to complete the setup.

Remark 2 (Registered PKI Setup with a Key Curator). Looking ahead, our coin construction requires each node to generate $O(\lambda n)$ -sized hint information. If all honest nodes read all hints and run **GroupPKGen** locally, the setup incurs $O(\lambda n^3)$ communication cost during the setup. This cost can be reduced to $O(\lambda n^2)$ under a registered PKI setup with a key curator [50], which collects (pk_i, hint_i) from participants, runs **GroupPKGen** locally, and returns $(pk, (pk_i)_{i \in [n]})$ to all.

We note that key curators have been extensively studied in registration-based encryption [50], representing a strictly weaker setup assumption than a trusted key generator which is needed by identity-based encryption [19] and the unique threshold signature schemes. A key curator is fully transparent, holds no secret information, and performs only deterministic tasks.

Design Goal: Common Coin. We aim at designing a silent-setup common coin protocol with optimal resilience and optimal communication. Specifically, a common coin protocol provides a random source observable by all participants but unpredictable by an adversary [22], which can be formally defined as follows.

Definition 1 (Common Coin). *Let Π be a protocol involving n participants in which each P_i does not have an input and will be instructed to output a value $v_i \in V$. We say Π is an (n, f, φ) common coin protocol for $\varphi \in (0, 1]$, if, in any instance of Π , and for any PPT adversary $\mathcal{A}$ which can corrupt up to f parties, the following properties are ensured:*

- **Termination:** *Except with negligible probability, every honest party P_i eventually outputs a value $r_i \in V$.*
- **φ -Fairness:** *With a probability of at least $\varphi - \text{negl}(\lambda)$, the protocol has a fair outcome, satisfying:*
 - *Agreement:* *There is a value $v \in V$ such that $v_i = v$ for any $i \in \mathcal{H}$, where $\{P_i\}_{i \in \mathcal{H}}$ is the set of honest parties.*
 - *Unpredictability and bias-resistance:* *Let $\text{State}_{\mathcal{A}}^{\text{before}}$ denote the internal state of $\mathcal{A}$ before $n - f - |\mathcal{C}|$ so-far-honest nodes participate in the protocol, where $|\mathcal{C}|$ is the number of corrupted parties. For any PPT distinguisher $\mathcal{D}$ and $u \leftarrow \$ V$,*

$$|\Pr[\mathcal{D}(\text{State}_{\mathcal{A}}^{\text{before}}, v) = 1] - \Pr[\mathcal{D}(\text{State}_{\mathcal{A}}^{\text{before}}, u) = 1]| \leq \text{negl}(\lambda).$$

Remark 3. When $\varphi = 1$, the definition captures the multivalued version of the cryptographic *strong* common coin [22], which ensures termination, agreement, unpredictability, and bias-resistance except with negligible probability. When $\varphi < 1$, it can be seen as a multivalued and cryptographically secure version of the *weak* common coin by Feldman and Micali [41], and it can trivially imply a protocol satisfying their definition, as we show in Lemma 13 in Appendix B.1.

4 Our Tools

In this section, we introduce a few new tools for our common coin protocols. In particular, in Sect. 4.1, we present our core cryptographic tool, which is a variant of the silent threshold encryption (STE) from [52] but with stronger security guarantees that are necessary for our coin protocols. In Sect. 4.2, we introduce a simple framework for analyzing specific compositions.

4.1 Silent Threshold Encryption with Tag-homomorphism.

A threshold encryption scheme allows a group of nodes to jointly decrypt a ciphertext under their group public key, while an adversary controlling up to a threshold number of nodes cannot break the secrecy guarantee. It is called a silent threshold encryption scheme if its setup phase is silent (as in Remark 1). In addition, we consider *tag-homomorphism*, which means the encryption algorithm takes an extra input *tag*, and ciphertexts under the same public key and the same tag can be homomorphically evaluated.

Syntax. Formally, an (n, t) silent threshold tag-homomorphic encryption (STHE) scheme consists of the following algorithms:

- **SilSetup** $(1^\lambda, n, t) \rightarrow (\text{crs}, ek, (ek_i)_{i \in [n]}, (dk_i)_{i \in [n]})$. This is the silent setup phase of STHE. *crs* is the implicit input to all the following algorithms.
- **Enc** (ek, tag, m) Encrypts the plaintext m under ek and tag , producing a ciphertext c . In particular, we assume the plaintext space $\mathcal{M}$ is a commutative group with an operation $\oplus$.
- **Eval** $(ek, tag, \{c_i\}_{i \in S})$ On input a set of ciphertexts under the same encryption key ek and tag , outputs a ciphertext c^* .
- **PartDec** (ek, tag, dk_i) Generates a partial decryption key ζ_i for ciphertexts under ek and tag .
- **PartVrfy** (ek, tag, ek_i, ζ_i) Verifies whether ζ_i is a partial decryption key generated by node i .
- **DkAgg** $(ek, tag, \{\zeta_i\}_{i \in S})$ Aggregates a set of valid partial decryption keys into a decryption key dk_{tag} for tag .
- **Dec** (ek, tag, c, dk_{tag}) Decrypts a ciphertext c using the decryption key dk_{tag} .

Remark 4. Compared with the STE syntax in [52], our STHE introduced above includes the notion of a tag, which serves as an additional input to the encryption algorithm and enables two essential features: (1) *Tag-homomorphism*, allowing ciphertexts encrypted under the same tag to be homomorphically aggregated; (2) *Tag-based decryption*, enabling all participants to release a decryption key for a specific tag to decrypt all ciphertexts under that tag.

Correctness. Let $\mathcal{A}$ be any PPT adversary corrupting up to t participants. Let $\{\text{crs}, ek, (ek_i)_{i \in [n]}\} \leftarrow \text{SilSetup}^{\mathcal{A}}(1^\lambda, n, t)$. The following properties hold:

- *Partial Verification Correctness:* For any tag and $i \notin \mathcal{C}_{\text{init}}$, it holds that

$$\Pr[\text{PartVrfy}(ek, tag, ek_i, \text{PartDec}(ek, tag, dk_i))] = 1.$$

- *Decryption Correctness:* For any tag , m , and γ , such that $c = \text{Enc}(ek, tag, m; \gamma)$, and for $t + 1$ partial decryption keys $\{\zeta_j\}_{j \in S}$ where $\text{PartVrfy}(ek, tag, ek_j, \zeta_j) = 1$, it holds that

$$\Pr[\text{Dec}(ek, tag, c, \text{DkAgg}(ek, tag, \{\zeta_j\}_{j \in S}))] = m.$$

- *Evaluation Correctness:* For any $\{c_i = \text{Enc}(ek, \text{tag}, m_i; \gamma_i)\}_{i \in S}$ and $c^* \leftarrow \text{Eval}(ek, \text{tag}, \{c_i\}_{i \in S})$, there exists γ^* such that $c^* = \text{Enc}(ek, \text{tag}, \bigoplus_{i \in S} m_i; \gamma^*)$.

Semantic Security. At a high level, the semantic security captures that any PPT adversary cannot learn any non-trivial information from a ciphertext as long as the adversary does not know enough partial decryptions for the tag of the ciphertext. We further allow the adversary to see the partial decryptions produced by honest participants for any specific tags, which captures the fact that the decryption process for other tags will not undermine the secrecy of ciphertexts under the current tag. We use $\mathcal{O}_{\text{PartD}}$ to formalize such an ability. In addition, for an adaptive adversary, we consider a corruption oracle $\mathcal{O}_{\text{Corr}}$ through which the adversary can obtain the secret key, and the randomness previously used to generate the partial decryption keys; The adversary can decide which participant to corrupt based on the query results of $\mathcal{O}_{\text{PartD}}$ and the challenge oracle $\mathcal{O}_b$.

$\text{aIND}_b^{\mathcal{A}, \text{tag}^*}(1^\lambda, n, f)$	$\mathcal{O}_{\text{PartD}}(\text{tag}, i)$
$\mathcal{C}, \text{Tags}, \text{State}_1, \dots, \text{State}_n \leftarrow \emptyset$ $(\text{crs}, ek, (ek_i)_{i \in [n]}; \text{state}_\mathcal{A}; (dk_i)_{i \in [n] \setminus \mathcal{C}_{\text{init}}})$ $\leftarrow \text{SilSetup}^\mathcal{A}(1^\lambda, n, t), \mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}_{\text{init}}$ $1 \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Corr}}(\cdot), \mathcal{O}_{\text{PartD}}(\cdot), \mathcal{O}_b(\cdot)}(\text{state}_\mathcal{A})$	if $\text{tag} \neq \text{tag}^* \wedge i \notin \mathcal{C}$ $\text{Tags} \leftarrow \text{Tags} \cup \{\text{tag}\}$ $\gamma \leftarrow \text{Rand}_{\text{PartD}}$ $\zeta_i \leftarrow \text{PartDec}(ek, \text{tag}, dk_i; \gamma)$ $\text{State}_i \leftarrow \text{State}_i \cup \{(\text{tag}, i, \gamma, \zeta_i)\}$ return ζ_i
$\mathcal{O}_{\text{Corr}}(i)$	$\mathcal{O}_b(m_0, m_1)$
if $i \in [n] \setminus \mathcal{C} \wedge \mathcal{C} < t$ $\mathcal{C} \leftarrow \mathcal{C} \cup \{i\}$ return (dk_i, State_i)	return $c_b \leftarrow \text{Enc}(ek, \text{tag}^*, m_b)$

Formally, we say an (n, t) -STHE scheme satisfies the adaptively semantic security if for any PPT adversary $\mathcal{A}$, it holds that, $\forall \lambda \in \mathbb{N}$ and $\forall \text{tag}^* \in \{0, 1\}^*$,

$$\text{Adv}_{\mathcal{A}, \text{tag}^*}^{\text{aind}}(\lambda) = |\Pr[\text{aIND}_0^{\mathcal{A}, \text{tag}^*}(1^\lambda, n, t)] - \Pr[\text{aIND}_1^{\mathcal{A}, \text{tag}^*}(1^\lambda, n, t)]| \leq \text{negl}(\lambda). \quad (1)$$

Instantiation. We show an instantiation of STHE in Appendix D, which is a variant of the silent threshold encryption scheme in [52]. In summary, the ciphertext size and the partial decryption size are both $O(\lambda)$, independent of the number of participants n .

4.2 A Simple Framework for Analyzing Specific Composition

In this section, we introduce a framework to analyze the security of a protocol instance when it is composed of certain other instances in a particular manner.

A formal composition proof is necessary. In most works on common coins or Byzantine consensus, protocol security is defined through a set of desired properties. A protocol typically consists of multiple components, each with its own security properties, which are proven in the stand-alone setting. When analyzing the security of the final protocol, existing works often reduce certain properties of the overall protocol to properties of specific components. However, they usually do not address whether these properties still hold when the components are composed together in the final protocol.

In Appendix C.1, we present a counter-example illustrating that: a component secure in the stand-alone setting fails when composed with another instance in the same protocol. Specifically, we modify [34], in which a digital signature scheme is employed, and each node signs exactly one message per instance. If the underlying signature scheme is replaced with a one-time signature scheme, the modified protocol remains secure in the stand-alone setting. However, when two instances are composed under the same PKI setup, security breaks down.

Why not UC? One approach is to prove each component’s security within the Universal Composability (UC) framework [23]. However, our construction relies on certain “non-UC” components, such as succinct non-interactive arguments of knowledge (SNARKs) [56], making a direct UC proof challenging without extensive modeling. Furthermore, in asynchronous protocols, messages from different components cannot be scheduled in a strict sequential order, standard composition theorems—such as those for sequential composition—do not readily apply.

Our Treatment. We prove that a subroutine S_1 remains secure when composed with other subroutines $\{S_i\}$ by demonstrating that any information an adversary learns from is $\{S_i\}$ “useless” for compromising the security of S_1 .

Overview. Consider two protocol instances, $\Pi[\text{sid}]$ and $\Pi'[\text{sid}']$, running concurrently. An adversary $\mathcal{A}$ can access information through two primary channels: observing communication transcripts—whether exchanged between honest nodes or sent to corrupted parties—and corrupting honest nodes, thereby gaining access to their internal states. If $\Pi[\text{sid}]$ and $\Pi'[\text{sid}']$ share common participants, corrupting a participant in one instance may trigger their corruption in the other.

To formalize the idea that any information $\mathcal{A}$ gains from $\Pi'[\text{sid}']$ is “useless” for attacking $\Pi[\text{sid}]$, we may consider the existence of a polynomial-time *simulator* algorithm that reconstructs this information solely from what $\mathcal{A}$ learns in $\Pi[\text{sid}]$. However, this formulation can be overly restrictive. For example, if $\Pi[\text{sid}]$ and $\Pi'[\text{sid}']$ are signature-based protocols sharing the same PKI setup, $\mathcal{A}$ may learn new signatures from $\Pi'[\text{sid}']$ that cannot be simulated due to the unforgeability of the underlying signature scheme. Nonetheless, since the two instances have distinct identifiers, signatures on messages with different identifiers could be considered “useless” for attacking $\Pi[\text{sid}]$. This observation motivates a natural extension: allowing the simulator access to a signing oracle under *specific restrictions*. We further generalize the available oracle beyond signature-related queries to accommodate a broader class of cryptographic primitives.

Composition lemma with oracle-aided simulator. We consider a set of oracles $\mathbb{O} = \{\mathcal{O}_z\}$, where each $\mathcal{O}_z$ accepts inputs in the form of $(\text{sid}, i, \cdot)$, where sid denotes the protocol instance identifier and i represents the index of an uncorrupted node. A key restriction is that an adversary attacking an instance with identifier sid can only query $\mathcal{O}_z$ with $(\text{sid}', i, \cdot)$ such that $\text{sid}' \neq \text{sid}$.

An adaptive adversary may corrupt a party P_i during execution, gaining access to all internal states used to generate prior messages. To model this, we let $\mathbb{O}$ include a special corruption oracle $\mathbb{O}_{\text{Corr}}$. When an adversary $\mathcal{A}$ corrupts P_i , the query i is automatically triggered at $\mathbb{O}_{\text{Corr}}$, which returns all internal secrets and randomness used by any oracle $\mathcal{O}_z$ that previously responded to queries in the form of $(\cdot, i, \cdot)$.

Furthermore, some information from an instance $\Pi'[\text{sid}']$ may not be captured by oracles, such as protocol inputs. To address this, we introduce an auxiliary string Aux , which is available to both the adversary against $\Pi[\text{sid}]$ and the simulator algorithm $\mathcal{S}$. Formally, we introduce the following lemma.

Lemma 1. *Let $\Pi[\text{sid}]$ and $\Pi'[\text{sid}']$ be two protocol instances running concurrently. Let Aux be an auxiliary string. The protocol $\Pi[\text{sid}]$ remains secure in composition with $\Pi'[\text{sid}']$ if it remains secure in the stand-alone setting, even when the adversary has access to oracles in $\mathbb{O}$ and Aux , and if for any $\mathcal{A}$ attacking the composition, there exists an algorithm $\mathcal{S}$ whose complexity is polynomial in $\mathcal{A}$'s complexity and which has access to $\mathbb{O}$ and Aux , such that at any point in the execution, the distribution of $\text{VIEW}_{\mathcal{A}} = (\text{VIEW}_{\mathcal{A},\Pi[\text{sid}]}, \text{VIEW}_{\mathcal{A},\Pi'[\text{sid}']})$ is identical to $(\text{VIEW}_{\mathcal{A},\Pi[\text{sid}]}, \mathcal{S}^{\text{Aux},\mathbb{O}}(\text{VIEW}_{\mathcal{A},\Pi[\text{sid}]})$.*

Proof. Suppose an adversary $\mathcal{A}$ can violate a security property of $\Pi[\text{sid}]$ in composition with $\Pi'[\text{sid}']$. We construct a stand-alone adversary $\mathcal{B}^{\mathbb{O}}$ that violates the security of $\Pi[\text{sid}]$ in isolation. The adversary $\mathcal{B}$ runs a copy of $\mathcal{A}$ as follows: it relays messages between $\mathcal{A}$ and $\Pi[\text{sid}]$ (so that $\text{VIEW}_{\mathcal{A},\Pi[\text{sid}]}$ is available to both $\mathcal{A}$, $\mathcal{B}$) and provides $\text{VIEW}_{\mathcal{A},\Pi'[\text{sid}']} = \mathcal{S}^{\mathbb{O}}(\text{Aux}, \text{VIEW}_{\mathcal{A},\Pi[\text{sid}]})$ to $\mathcal{A}$. Since

$$(\text{VIEW}_{\mathcal{A},\Pi[\text{sid}]}, \text{VIEW}_{\mathcal{A},\Pi'[\text{sid}']}) = (\text{VIEW}_{\mathcal{A},\Pi[\text{sid}]}, \mathcal{S}^{\mathbb{O}}(\text{Aux}, \text{VIEW}_{\mathcal{A},\Pi[\text{sid}]})$$

$\mathcal{A}$ can violate the security of $\Pi[\text{sid}]$, and therefore, so can $\mathcal{B}^{\mathbb{O}}$. $\square$

Oracle-Emulatable Protocols. In Lemma 1, we show that a protocol instance $\Pi[\text{sid}]$ remains secure when composed with another instance $\Pi'[\text{sid}']$, provided that $\Pi[\text{sid}]$ remains secure in the presence of oracles $\mathbb{O}$, and that $\Pi'[\text{sid}']$ can be “emulated” using $\mathbb{O}$ and the adversary’s view in $\Pi[\text{sid}]$. However, determining whether $\Pi'[\text{sid}']$ is emulatable in this way requires checking it with respect to $\Pi[\text{sid}]$, making the analysis case-dependent. To establish a more general result on the “emulatability” of $\Pi'[\text{sid}']$ independently of its composition with any specific $\Pi[\text{sid}]$, we introduce the notion of $\mathbb{O}$ -emulatable instances.

In particular, we aim to cover scenarios where multiple instances of the same protocol execute under the same setup.

Definition 2 ($\mathbb{O}$ -Emulatable Protocol). *Let $\Pi.\text{Setup}[\text{sid}]$ be an instance of the setup phase of protocol Π , and let $\Pi[\langle \text{sid}, \text{sid}_0 \rangle]$ be an instance executed under the setup established by $\Pi.\text{Setup}[\text{sid}]$. We say that $\Pi[\langle \text{sid}, \text{sid}_0 \rangle]$ is $\mathbb{O}$ -emulatable under $\Pi.\text{Setup}[\text{sid}]$, if for any arbitrary protocol instance $\Pi'[\text{sid}']$ running in an arbitrary composition with $\Pi.\text{Setup}[\text{sid}]$ and Π , and for any PPT adversary $\mathcal{A}$ attacking the composition, there exists a PPT simulator $\mathcal{S}$ that can query the oracles in $\mathbb{O}$ with messages prefixed by $\langle \text{sid}, \text{sid}_0 \rangle$, such that the adversary’s view in the composition:*

$$(\text{VIEW}_{\mathcal{A},\Pi'[\text{sid}']}, \text{VIEW}_{\mathcal{A},\Pi.\text{Setup}[\text{sid}]}, \text{VIEW}_{\mathcal{A},\Pi[\langle \text{sid}, \text{sid}_0 \rangle]})$$

is identically distributed to:

$$(\text{VIEW}_{\mathcal{A},\Pi'[\text{sid}']}, \text{VIEW}_{\mathcal{A},\Pi.\text{Setup}}, \mathcal{S}^{\mathbb{O}}(\text{VIEW}_{\mathcal{A},\Pi'[\text{sid}']}, \text{VIEW}_{\mathcal{A},\Pi.\text{Setup}})).$$

If we omit $\Pi.\text{Setup}[\text{sid}]$, then $\Pi[\langle \text{sid}, \text{sid}_0 \rangle]$ is said to be $\mathbb{O}$ -emulatable.

How we analyze the composition security for components in this paper. ⁹ We use the framework established above to ensure the composition security of components in this paper.

⁹ To also understand our composition lemma in an intuitive way, when a larger protocol, e.g., our coin protocol, assembles the components in a specific way, and components are proven secure under such compositions, then the analysis of the larger protocol can indeed be done as in the stand-alone cases. This fills the gap in existing works which analyze components themselves only in the standard-alone setting.

Specifically, we additionally prove two facts about every component. First, this component remains secure even given a set of relevant oracles to the adversary. Second, all information an adversary can learn from an instance of the component can be emulated using a set of oracles. Therefore, when arguing whether a component A is secure in a composition, we only need to check whether the rest of the composition can be emulated using the set of oracles with which the component A remains secure.

As most of the composition analysis for components in this paper is easy to follow and more like a sanity check, we defer all composition analysis to a dedicated section in the Appendix E.

5 Asynchronous Subset Alignment

One of the main tools in our design of communication-efficient asynchronous coin protocol is a new primitive termed *Asynchronous Subset Alignment* (Align). It can be seen as a (strictly) weaker form of the well-studied Gather problem [4, 31] or the Weak Core Set problem [49], while it is actually sufficient for weak coin constructions as we will soon demonstrate in the next section.

5.1 Definition

Following the intuition discussed in §2, we formally define the primitive as follows. A detailed discussion on the necessity of each property is provided in the remarks following the definition.

Syntax. Let Π be a protocol involving n participants in which each P_i has an input v_i and will be instructed to output a set OSet_i which either is empty or consists of tuples in the form of (j, v_j, θ_j) . We say Π is an (n, f) -secure Align with a predicate function $\text{Predicate} : \{0, 1\}^* \rightarrow \{0, 1\}$, if it is equipped with an algorithm $\text{Align.Vrfy} : (\text{sid}, j, v_j, \theta_j) \rightarrow \{0, 1\}$, such that in any instance $\Pi[\text{sid}]$, and for any PPT adversary $\mathcal{A}$ which can corrupt up to f nodes, the *termination*, *external validity*, and $f + 1$ -*alignment* properties are ensured.

For facilitating a formal definition, we use the following variables: (1) $\mathcal{H}_{\text{first}} \subset [n]$, so that $\{P_i\}_{i \in \mathcal{H}_{\text{first}}}$ is the set of all so-far-honest nodes when the first honest P_i outputted; (2) $\mathcal{H}_{\text{end}} \subset [n]$, so that $\{P_i\}_{i \in \mathcal{H}_{\text{end}}}$ is the set of all forever-honest nodes; (3) $\text{State}_{\mathcal{A}}^{\text{end}}$, which is the state of $\mathcal{A}$ when all honest nodes have outputted in the instance; (4) $\text{State}_i^{\text{first}}$, which is the state of P_i when the first honest node generated an output. With these notions, we define the properties as follows.

- *Termination.* When every honest participant P_i is activated on sid with a valid input v_i such that $\text{Predicate}(v_i) = 1$, except with a negligible probability, every honest participant P_i eventually outputs a set OSet_i which is either $\emptyset$ or in the form of $\{(j, v_j, \theta_j)\}_{j \in S_i}$ for some $S_i \subset [n]$.
- *External Validity.* When an honest node P_i outputs a non-empty $\text{OSet}_i = \{(j, v_j, \theta_j)\}_{j \in S_i}$, it satisfies that for all $j \in S_i$, $\text{Align.Vrfy}(\text{sid}, j, v_j, \theta_j) = 1$ and $\text{Predicate}(v_j) = 1$.
- $(f+1)$ -*Alignment.* There is a polynomial-time algorithm CSGen , which on the input of $\{\text{State}_i^{\text{first}}\}_{i \in \mathcal{H}_{\text{first}}}$, outputs a set $\text{CoverSet} = \{(j, v_j)\}_{j \in \mathbb{I}_c}$ so that $n - f \leq |\text{CoverSet}| = |\mathbb{I}_c| \leq n$, and a subset

$WS = \{(j, v_j)\}_{j \in \mathbb{I}_s} \subset \text{CoverSet}$ s.t. $|\mathbb{I}_s| = n - f$. For any PPT algorithm $\mathcal{A}'$, it holds that:

$$\Pr \left[\begin{array}{l} \text{CSGen}(\{\text{State}_i^{\text{first}}\}_{i \in \mathcal{H}_{\text{first}}}) \rightarrow (\text{CoverSet}, WS) \\ \mathcal{A}'(\text{State}_{\mathcal{A}}^{\text{end}}, \{\text{OSet}_i\}_{i \in \mathcal{H}_{\text{end}}}) \rightarrow (j', v', \theta') : \\ \left(\exists (j, v_j) \in WS, \text{s.t. } \sum_{i \in \mathcal{H}_{\text{end}}} \text{isCovered}(\text{OSet}_i, (j, v_j)) < f + 1 \right) \\ \vee (\text{Align.Vrfy}(\text{sid}, j', v', \theta') = 1 \wedge (j', v') \notin \text{CoverSet}) \end{array} \right] \leq \text{negl}(\lambda),$$

where $\text{isCovered}(\text{OSet}_i, (j, v_j)) = 1$, if and only if there exists some θ_j s.t. $(j, v_j, \theta_j) \in \text{OSet}_i$.

Remark 5. By properly defining **Predicate**, the external validity ensures the outputs in **Align** are “meaningful” to its application scenarios. Looking ahead, in our weak common coin protocol, we expect **Align** to return sets of valid aggregated ciphertexts. Without this property, each node may return a set of invalid ciphertexts that cannot be decrypted at all, which is useless for our application even if the other properties are all ensured.

Remark 6. The $(f + 1)$ -alignment captures that although the output sets OSet_i ’s of different nodes can be different, all of them are “well-aligned” according to certain constraints which are determined when the first honest generated its output set. In particular:

- There is a $(n - f)$ -sized well-covered subset **WS** of **CoverSet**, such that for every (j, v_j) in the subset, it must be contained in the output set of at least $f + 1$ forever honest nodes.
- There is a binding cover **CoverSet** determined when the first honest node generated its output, such that all valid values w.r.t. **Align.Vrfy** are contained in it. It implies the output set OSet_j of any honest P_j is a subset of **CoverSet**.

Remark 7. The existence of both the well-covered subset **WS** and the binding cover set **CoverSet** is fundamental to our weak common coin protocol.

As we sketched in Sect. 2, in our weak common coin protocol, after **Align**, there will be a ranking phase to rank all values in the output set of **Align**, and it is important to ensure that there is a constant probability that all nodes can decide on the highest-ranked value. So, intuitively, as **WS** has $n - f$ values, and **CoverSet** has at most n values, there is a good probability that **WS** has the value with the highest rank. So, $f + 1$ honest nodes will immediately decide on this value. As ensured by **CoverSet**, the adversary cannot inject new values with a valid proof w.r.t. **Align.Vrfy**, so all honest nodes can decide on the highest-ranked value with the help of the $f + 1$ honest nodes.

It is necessary to ensure that both **CoverSet** and **WS** can be determined at the time that the first honest node generates an output. This is because when an honest node finishes **Align**, it may start to assist in ranking the values (in our weak common coin, it helps decrypt all ciphertexts). If **CoverSet** and **WS** have not been decided at that point, $\mathcal{A}$ may, according to the ranking information, maliciously delay messages so that the highest-ranked value will never be contained in **WS**.

5.2 Our Construction

Overview. Our construction leverages a careful combination of provable broadcast (PB) instances [21]. (We recall the definition and a threshold signature-based construction of PB in Appendix B.2.)

PB serves as a more efficient alternative to conventional reliable broadcast, achieving a communication complexity of $O(n(\lambda + \ell))$ when broadcasting ℓ bits to n nodes. In each PB instance, at most one value possesses a broadcast proof, and the existence of such proof implies that at least $f + 1$ forever-honest nodes have delivered this value.

In what follows, we start with a simple PB-based protocol, analyze its shortcomings, and incrementally refine it to satisfy all properties of Align.

A Naive Attempt. A straightforward approach is to let all nodes broadcast their input values via n parallel PB instances. Once the sender obtains the PB proof, it multicasts the proof along with the input value to the network. Each node then outputs the set of $n - f$ values with valid PB proofs.

While this ensures that each node contributes the same value to different output sets, it does not guarantee that both **CoverSet** and the well-covered subset are determined when the first honest node generates its output.

Two Consecutive PB Instances. The primary issue with the above approach is that the multicasting step fails to ensure that a certain subset of $n - f$ proof-value pairs consistently appears in enough honest nodes' output sets. To address this, we introduce an additional layer of PB instances. Specifically, all nodes first broadcast their inputs using n parallel PB instances (denoted as PB-I). Once having the PB proof, each node broadcasts the proof-value pair using another PB instance (PB-II). Finally, each node multicasts the value along with the PB-II proof and outputs the set of values delivered in PB-II (that have valid proofs from PB-I) once it has received $n - f$ proofs for PB-II.

With this approach, when an honest node outputs, there exist $n - f$ proof-value pairs from PB-II such that at least $f + 1$ forever-honest nodes have delivered them, ensuring the existence of the well-covered subset. The PB-I proofs further ensure the uniqueness of these values.

Adding a Tail Phase. The above approach still requires honest nodes to participate in the protocol as receivers even after they have outputted; otherwise, other honest nodes may fail to receive enough PB proofs and terminate. This, however, allows an adaptive adversary to introduce new values into the output set of other honest nodes after the first honest node has terminated, violating the existence of a binding cover **CoverSet**.

To counter this, we introduce a tail phase: once an honest node decides to output, it abandons all PB instances, preventing any new values from being added while ensuring all honest nodes can terminate even if they have not yet received enough PB-II proofs.

Adding a Pre-lock PB. One subtle issue remains: the size of the binding cover is not necessarily bounded by n . A corrupted node can send $f + 1$ different values in PB-I to $f + 1$ honest nodes before they abandon the protocol. When the first honest node outputs, the corrupted node can still choose any of these $f + 1$ values to finalize the first PB, causing different values to appear in various honest output sets. Since up to f corrupted nodes can act this way, the number of values appearing in output sets may reach $f(f + 1)$, exceeding n .

To mitigate this, we introduce an additional PB instance that locks input values before PB-I. The final protocol consists of n parallel chains of three consecutive PB instances. Hereafter, the “pre-lock” PB, PB-I, and PB-II are referred to as the first PB, the second PB, and the third PB, respectively, due to their natural execution order. The execution flow of the protocol is illustrated in Fig. 6.

Construction. With a silent setup phase for enabling PB, we present the pseudocode of our Align protocol in Algorithm 1. Our protocol works as follows:

Algorithm 1 The Align protocol with identifier sid and Predicate for P_i

```
1:  $\text{Locked}_i, \text{OSet}_i, \text{Covered}_i \leftarrow \emptyset$ 
2: for  $j = 1$  to  $n$  do
3:   Initialize  $\text{PB}[\langle \text{sid}, j, \text{NUM} \rangle]$  with the predicate  $\text{Pred}_{\text{NUM}}$ , for  $\text{NUM} = 1, 2, 3$ 
4: upon activated with input  $v_i$  s.t.  $\text{Predicate}(v_i) = 1$  do
5:   Invoke  $\text{PB}[\langle \text{sid}, i, 1 \rangle]$  as the sender with input  $v_i$ .
6:   wait until  $\text{PB}[\langle \text{sid}, i, 1 \rangle]$  returns  $\rho_{i,1}$ 
7:   Invoke  $\text{PB}[\langle \text{sid}, i, 2 \rangle]$  as the sender with input  $(v_i, \rho_{i,1})$ 
8:   wait until  $\text{PB}[\langle \text{sid}, i, 2 \rangle]$  returns  $\rho_{i,2}$ 
9:   Invoke  $\text{PB}[\langle \text{sid}, i, 3 \rangle]$  as the sender with input  $(v_i, \rho_{i,1}, \rho_{i,2})$ 
10:  wait until  $\text{PB}[\langle \text{sid}, i, 3 \rangle]$  returns  $\rho_{i,3}$ 
11:  Multicast  $(\text{sid}, \text{COVERED}, v_i, \rho_{i,1}, \rho_{i,2}, \rho_{i,3})$ 
12: upon  $\text{PB}[\langle \text{sid}, j, 2 \rangle]$  delivers  $(v_j, \rho_{j,1})$  do
13:   $\text{Locked}_i \leftarrow \text{Locked}_i \cup \{(j, v_j)\}$ 
14: upon  $\text{PB}[\langle \text{sid}, j, 3 \rangle]$  delivers  $(v_j, \rho_{j,1}, \rho_{j,2})$  do
15:   $\theta_j \leftarrow (\rho_{j,1}, \rho_{j,2}), \text{OSet}_i \leftarrow \text{OSet}_i \cup \{(j, v_j, \theta_j)\}$ 
16: upon receiving  $(\text{sid}, \text{COVERED}, v_j, \rho_{j,1}, \rho_{j,2}, \rho_{j,3})$  from  $P_j$  for the first time do
17:  if  $\text{PB}[\langle \text{sid}, j, 3 \rangle].\text{Vrfy}((v_j, \rho_{j,1}, \rho_{j,2}), \rho_{j,3}) = 1$  then
18:     $\text{Covered}_i \leftarrow \text{Covered}_i \cup \{(j, v_j)\}$ 
19:  if  $|\text{Covered}_i| = n - f$  and the ENOUGH-COVERED has not been sent then
20:    Multicast  $(\text{sid}, \text{ENOUGH-COVERED})$ 
21: upon receiving  $(\text{sid}, \text{ENOUGH-COVERED})$  from at least  $f+1$  distinct nodes and  $P_i$  has not sent ENOUGH-COVERED do
22:  Multicast  $(\text{sid}, \text{ENOUGH-COVERED})$ 
23: upon receiving  $(\text{sid}, \text{ENOUGH-COVERED})$  from at least  $n - f$  distinct nodes do
24:  for  $j = 1$  to  $n$  do
25:    Call  $\text{PB.Abandon}(\langle \text{sid}, j, \text{NUM} \rangle)$ , for  $\text{NUM} = 1, 2, 3$ .
26:  Multicast  $(\text{sid}, \text{FINISHED})$ 
27: upon receiving  $(\text{sid}, \text{FINISHED})$  from at least  $n - f$  distinct nodes do
28:  return  $\text{OSet}_i$ 
29: Algorithm  $\text{Align.Vrfy}(\text{sid}, j, v_j, \theta_j)$ 
30:  Parse  $\theta_j \rightarrow (\rho_{j,1}, \rho_{j,2})$ , and return  $\text{PB}[\langle \text{sid}, j, 2 \rangle].\text{Vrfy}((v_j, \rho_{j,1}), \rho_{j,2})$ 
31: Predicate  $\text{Pred}_{\text{NUM}}$ 
32:   $\text{Pred}_1(v) = 1$  iff  $\text{Predicate}(v) = 1$ 
33:   $\text{Pred}_2(j, v_j, \rho_{j,1}) = 1$  iff  $\text{PB}[\langle \text{sid}, j, 1 \rangle].\text{Vrfy}(v_j, \rho_{j,1}) = 1$ 
34:   $\text{Pred}_3(j, v_j, \rho_{j,1}, \rho_{j,2}) = 1$  iff  $\text{PB}[\langle \text{sid}, j, 2 \rangle].\text{Vrfy}((v_j, \rho_{j,1}), \rho_{j,2}) = 1$ 
```

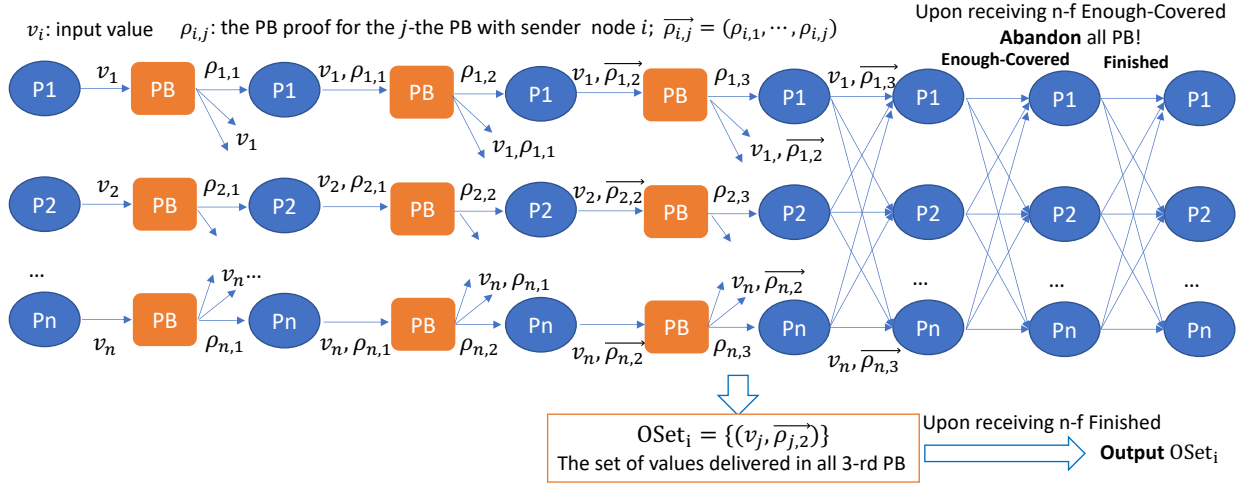


Fig. 6. Asynchronous Subset Alignment Align

First PB: At Line 4-5, every honest node P_i broadcasts its input v_i via the PB instance $PB[\langle \text{sid}, i, 1 \rangle]$, whose predicate is the external predicate of Align.

Second PB: At Line 6-7, upon obtaining a proof $\rho_{i,1}$ w.r.t $PB[\langle \text{sid}, i, 1 \rangle]$, P_i broadcasts $(v_i, \rho_{i,1})$ via $PB[\langle \text{sid}, i, 2 \rangle]$, whose predicate is that $\rho_{i,1}$ should be a valid delivery proof for v_i w.r.t. $PB[\langle \text{sid}, i, 1 \rangle]$. In addition, at Line 12-13, when P_i delivers $(v_j, \rho_{j,1})$ in $PB[\langle \text{sid}, j, 2 \rangle]$, it adds (j, v_j) to Lock_i .

Thrid PB: At Line 8-9, upon obtaining a proof $\rho_{i,2}$ w.r.t $PB[\langle \text{sid}, i, 2 \rangle]$, P_i broadcasts $(v_i, \rho_{i,1}, \rho_{i,2})$ via $PB[\langle \text{sid}, i, 3 \rangle]$, whose predicate is that $\rho_{i,2}$ should be a valid delivery proof for $(v_i, \rho_{i,1})$ w.r.t. $PB[\langle \text{sid}, i, 2 \rangle]$. In addition, at Line 14-15, when P_i delivers $(v_j, \rho_{j,1}, \rho_{j,2})$ in $PB[\langle \text{sid}, j, 3 \rangle]$, it adds $(j, v_j, \rho_{j,1}, \rho_{j,2})$ to $OSet_i$.

Tail Phase: At line 10-11, upon obtaining a proof $\rho_{i,3}$ w.r.t $PB[\langle \text{sid}, i, 3 \rangle]$, P_i multicasts $(\text{sid}, \text{COVERED } v_i, \rho_{i,1}, \rho_{i,2}, \rho_{i,3})$. At line 16-20, after receiving valid COVERED messages from at least $n - f$ distinct nodes, P_i multicasts $(\text{sid}, \text{ENOUGH } -\text{Covered})$. Otherwise, it will multicast this message when it receives $(\text{sid}, \text{ENOUGH } -\text{Covered})$ from at least $f + 1$ distinct nodes.

Abandon: An honest P_i abandons all PB instances after receiving $(\text{sid}, \text{ENOUGH } -\text{Covered})$ from at least $n - f$ distinct nodes. Meanwhile, $(\text{sid}, \text{FINISHED})$ is multicasted to the network.

Output: Finally, upon receiving the FINISHED messages from $n - f$ distinct nodes, P_i outputs $OSet_i$.

Efficiency analysis. Note that a PB instance incurs $O(n(\ell + \lambda))$ -bit communication cost and 2 rounds when broadcasting ℓ bits. And its proof size is $O(\lambda)$. Since our protocol involves three consecutive PB instances and three additional rounds after PB, it requires 9 rounds to terminate. It is easy to verify the communication complexity is $O(n^2(\ell + \lambda))$.

Instantiation and Assumptions. The Align protocol uses a PB protocol, for which we outline an instantiation in Appendix B.2, leveraging a silent threshold signature scheme [30]. The security of this scheme is based on the q -SDH assumption [18, 64] and the co-CDH assumption within the algebraic group model (AGM) and the random oracle model (ROM). Consequently, the Align protocol inherits the assumptions underlying the silent threshold signature scheme [30].

Security analysis. We establish the security of our Align protocol.

Theorem 1 (Align). *Assuming the security of the underlying PB, the Align protocol in Algorithm 1 ensures the properties of the termination, external validity, and $f + 1$ -alignment.*

Lemma 2 (Align termination). *Our Align protocol ensures termination, i.e., every honest node P_i can eventually terminate with a set OSet_i , which can be empty.*

Proof. We show that every honest node P_i can proceed with the instruction of returning OSet_i (line 28). Note that if an honest node P_j called **PB.Abandon** (line 25), then it must have received the **ENOUGH-COVERED** message from at least $n - f$ nodes, while at least $f + 1$ among them are forever-honest. Therefore, every honest node can receive at least $f + 1$ **ENOUGH-COVERED** messages. Following the instruction at line 21, all honest nodes will eventually multicast the **ENOUGH-COVERED** message. Hence, all honest nodes will multicast the **FINISHED** message (line 26), which grants the execution of the instruction at line 28. It leaves us to discuss whether there exists an honest node that can receive $n - f$ **ENOUGH-COVERED** messages, and we prove this by showing contradictions. In particular, if no honest node saw $n - f$ **ENOUGH-COVERED** messages, then no honest node called **PB.Abandon**. Following the termination property of PB, all honest senders will eventually obtain the broadcast proofs. It follows that all honest nodes can receive $n - f$ “valid” **COVERED** messages and then broadcast the **ENOUGH-COVERED** messages, which grants that an honest node to observe $n - f$ **ENOUGH-COVERED** messages. $\square$

Lemma 3 (Align External Validity). *The protocol Align satisfies the external validity.*

Proof. According to the protocol description, an honest node P_i will add a tuple (j, v_j, θ_j) to the set OSet_i only when $\text{PB}[\langle \text{sid}, j, 3 \rangle]$ delivers $(v_j, \rho_{j,1}, \rho_{j,2})$. Ensured by the external validity of PB, it follows that $\text{PB}[\langle \text{sid}, j, 2 \rangle].\text{Vrfy}(v_j, \rho_{j,1}, \rho_{j,2}) = 1$, which implies that at least $f + 1$ honest nodes delivered $(v_j, \rho_{j,1})$ in $\text{PB}[\langle \text{sid}, j, 2 \rangle]$. Following a similar argument, at least $f + 1$ honest nodes delivered v_j in $\text{PB}[\langle \text{sid}, j, 1 \rangle]$, which implies $\text{Predicate}(v_j) = 1$. As *Align.Vrfy* just checks if $\rho_{j,2}$ is valid proof for $(v_j, \rho_{j,1})$ w.r.t. $\text{PB}[\langle \text{sid}, j, 2 \rangle]$, the external validity of our *Align* is ensured. $\square$

Lemma 4 (Align $f + 1$ -Alignment). *Our Align protocol satisfies the $f + 1$ -alignment.*

Proof. First, we describe the algorithm *CSGen*, which on input of $\{\text{State}_i^{\text{first}}\}_{i \in \mathcal{H}_{\text{first}}}$ outputs a cover $\text{CoverSet} = \{(j, v_j)\}_{j \in \mathbb{I}_c}$ and a well covered subset $\mathbb{I}_s$. In particular, let $\{\text{Locked}_j\}_{j \in \mathcal{H}_{\text{first}}}$ be the local Locked sets of so-far-honest nodes at the time that the first honest nodes generated an output, which are contained in $\{\text{State}_i^{\text{first}}\}_{i \in \mathcal{H}_{\text{first}}}$. *CSGen* will output

$$\text{CoverSet} = \bigcup_{j \in \mathcal{H}} \text{Locked}_j = \{(j, v_j)\}_{j \in \mathbb{I}_c}$$

Regarding the well-covered subset $\mathbb{I}_w$, *CSGen* checks the $\{\text{Covered}_i\}_{i \in \mathcal{H}_{\text{first}}}$ (also available in the states), and finds any i' such that $|\text{Covered}_{i'}| = n - f$. It outputs $\text{WS} = \text{Covered}_{i'}$.

Now we argue that CoverSet are well-formed. Namely, $|\text{CoverSet}| = |\mathbb{I}_c| \leq n$, which implies that there are no $(j, v_j), (j, v'_j) \in \text{CoverSet}$ such that $v_j \neq v'_j$. Since each $(z, v_z) \in \text{Locked}_j$, P_j delivers $(v_z, \rho_{z,1})$ in $\text{PB}[\langle \text{sid}, z, 2 \rangle]$. According the predicate Pred_2 of $\text{PB}[\langle \text{sid}, z, 2 \rangle]$, it holds that $\text{PB}[\langle \text{sid}, z, 1 \rangle].\text{Vrfy}(v_z, \rho_{z,1}) = 1$. Ensured by the provability of PB, if two honest parties deliver $(v_z, \rho_{z,1})$ and $(v'_z, \rho'_{z,1})$ in $\text{PB}[\langle \text{sid}, z, 2 \rangle]$, respectively, it must follow $v_z = v'_z$. Therefore, $|\text{CoverSet}| \leq n$.

Next, we show that if the adversary $\mathcal{A}'$ outputs some (j, v_j, θ_j) such that $\text{Align.Vrfy}(\text{sid}, j, v_j, \theta_j) = 1$, (j, v_j) must belong to CoverSet . Note that when P_i outputs, at least $f + 1$ forever-honest nodes have multicasted the `FINISHED` message. According to line 23-25, these $f + 1$ nodes have called `PB.Abandon` for all PB instances. On the other hand, if (j, v_j) is not included in CoverSet , then no honest party has delivered v_j in $\text{PB}[\langle \text{sid}, j, 2 \rangle]$; In addition, there are $f + 1$ forever-honest nodes who have abandoned all PB instances, then, according to the provability of PB, the sender cannot obtain a proof $\rho_{j,2}$ for the value v_j . Therefore, due to the absence of the proof, no tuple in the form of $(j, v_j, \rho_{j,1}, \cdot)$ can be delivered in $\text{PB}[\langle \text{sid}, j, 3 \rangle]$, so (j, v_j) will not be included in OSet_i of any honest P_i .

Then, we argue that CSGen can always find some $i' \in \mathcal{H}_{\text{first}}$ such that $|\text{Covered}_{i'}| = n - f$, and $\text{Covered}_{i'}$ is a subset of CoverSet . Note that an honest node P_i outputs OSet_i when it has seen $n - f$ `FINISHED` messages, while at least $f + 1$ of them are sent by forever-honest nodes, which implies at least $f + 1$ forever honest nodes have seen $n - f$ `ENOUGH-COVERED` messages. Then, at least one honest $P_{i'}$ sent the `ENOUGH-COVERED` message because of $|\text{Covered}_{i'}| = n - f$. For every $(j, v_j) \in \text{Covered}_{i'}$, according to Line 17, there exists a proof showing it has been delivered by at least $f + 1$ forever honest nodes in $\text{PB}[\langle \text{sid}, j, 3 \rangle]$, and it follows that it was delivered by at least $f + 1$ forever honest nodes in $\text{PB}[\langle \text{sid}, j, 2 \rangle]$. Therefore, according to Line 12-13, (j, v_j) must be included in at the Locked_i of at least $f + 1$ forever honest P_i 's, which makes $(j, v_j) \in \text{CoverSet}$.

Finally, we prove that for every $j \in \mathbb{I}_s$, (j, v_j) must appear in the output set of at least $f + 1$ forever honest nodes. For every $(z, v_z) \in \text{Covered}_{i'}$, there exists a tuple $(v_z, \rho_{z,1}, \rho_{z,2})$ that was delivered by $\text{PB}[\langle \text{sid}, z, 3 \rangle]$, such that $\text{PB}[\langle \text{sid}, z, 2 \rangle].\text{Vrfy}((v_z, \rho_{z,1}), \rho_{z,2}) = 1$, Ensured by the provability of PB, at least $f + 1$ forever-honest nodes have delivered $(v_z, \rho_{z,1})$ in $\text{PB}[\langle \text{sid}, z, 2 \rangle]$. Therefore, (z, v_z) is included in at least $f + 1$ forever-honest nodes' OSet . $\square$

6 Communication-Optimal Asynchronous Weak Coin

We now present our asynchronous weak common coin protocol with $O(\lambda n^2)$ communication complexity and $O(1)$ round complexity.

6.1 The Construction

Building blocks: Our scheme is built upon the following ingredients.

STHE. An (n, f) -secure STHE scheme $\text{STHE} = \{\text{STHE.SilSetup}, \text{STHE.Enc}, \text{STHE.Eval}, \text{STHE.PartDec}, \text{STHE.PartVrfy}, \text{STHE.DkAgg}, \text{STHE.Dec}\}$, introduced in Sect. 4.1, is assumed in our weak coin protocol. For notational convenience, we denote its message space and randomness space by Msg_{te} , Rand_{te} , respectively.

Digital signature. A standard-model digital signature scheme $\text{DS} = \{\text{DS.KeyG}, \text{DS.Sign}, \text{DS.Vrfy}\}$ is assumed. DS shall satisfy the conventional *existential unforgeability under chosen message attacks*, which could have various instantiations, for example, the Boneh-Boyen signature [18].

Argument systems. We need a $\text{NIZK} = \{\text{NIZK.Setup}, \text{NIZK.Prove}, \text{NIZK.Vrfy}\}$ for the following relation:

$$\mathcal{R}_{\text{enc}} := \{((ek, tag, c, aux), (m, \gamma)) : c = \text{STHE.Enc}(ek, tag, m; \gamma)\}, \quad (2)$$

which demonstrates that a ciphertext c is well-formed under ek and tag .

We further need a $\text{SNARK} = \{\text{SNARK.Setup}, \text{SNARK.Prove}, \text{SNARK.Vrfy}\}$ for the following relation:

$$\begin{aligned} \mathcal{R}_{\text{agg}} := & \{(n, f, (vk_i)_{i \in [n]}, ek, tag, \text{crs}_{\text{enc}}, c^*), \{(c_{i_j}, \pi_{\text{enc}, i_j}, \sigma_{i_j})\}_{j \in [t+1]} : \\ & c^* = \text{STHE.Eval}(ek, tag, \{c_{i_j}\}_{j \in [f+1]}), \text{ and } \forall j \in [f+1], \text{DS.Vrfy}(vk_{i_j}, \\ & (tag, c_{i_j}, \sigma_{i_j}) = 1 \wedge \text{NIZK.Vrfy}(\text{crs}_{\text{enc}}, (ek, tag, c_{i_j}, i_j), \pi_{\text{enc}, i_j}) = 1\}. \end{aligned} \quad (3)$$

SNARK is used to show a ciphertext c^* is obtain by honestly aggregating $f+1$ ciphertexts that are signed by distinct users. As we introduced in Sect. B.2, NIZK shall satisfy the completeness, simulation extractability, and zero-knowledge property, while SNARK ensures the completeness and knowledge soundness and provides $O(1)$ -sized proofs. In addition, as the verification algorithm of NIZK is a part of the relation $\mathcal{R}_{\text{agg}}$, we require NIZK to be secure in the standard model.

Align. Our weak coin protocol uses our (n, f) -secure Align protocol as a subroutine. In particular, the external predicate Predicate , is parameterized by $(\text{crs}_{\text{agg}}, n, f, (vk_i)_{i \in [n]}, ek, \text{sid}, \text{crs}_{\text{enc}})$, and $\text{Predicate}(c^*, \pi^*) = 1$, iff

$$\text{SNARK.Vrfy}(\text{crs}_{\text{agg}}, (n, f, (vk_i)_{i \in [n]}, ek, \text{sid}, \text{crs}_{\text{enc}}, c^*), \pi^*) = 1. \quad (4)$$

Hash functions. We explicitly use two hash functions $H_1 : \{0, 1\}^* \rightarrow \text{TO}$ and $H_2 : \{0, 1\}^* \rightarrow \mathbf{V}$, which will be modeled as random oracles in the security analysis. Here, TO is a totally ordered set with space greater than $2^{2\lambda}$, so that the probability of two independent sampling in TO yielding the same value is negligibly small. $\mathbf{V}$ is the space of coin values.

The protocol description. With the ingredients introduced above, we present our weak common coin protocol in Algorithm 2. It works under a silent setup, where (1) STHE.SilSetup has been finished, so every P_i has the key pair (ek_i, dk_i) , while $(ek, (ek_i)_{i \in [n]})$ and the CRS crs_{te} are available to everyone; (2) the CRS setups for both NIZK and SNARK have been done, so crs_{enc} and crs_{agg} are publicly available; (3) the PKI setup for DS is finished, so that everyone has its own key pair (vk_i, sk_i) and knows the public keys of others; (4) the silent setup needed by Align is also finalized. At a high level, our protocol works as follows.

- *Contribute a random ciphertext.* At line 2-5, each user P_i encrypts a fresh chosen randomness under ek and the session ID sid to a ciphertext c_i , proves the well-formedness of c_i , and signs it. The ciphertext, c_i , the proof π_i , and the signature σ_i , are multicasted to the network.
- *Aggregate $f+1$ signed ciphertext.* At line 6-11, each user P_i collects $f+1$ ciphertexts which each has a well-formedness proof and a valid signature. Then, it aggregates those ciphertexts to c_i^* , and provides a proof π_i^* showing aggregation correctness. Note that any ciphertext c^* with a valid proof is an aggregation of $f+1$ ciphertexts, which contains at least one ciphertext contributed by the forever-honest node, so the encrypted value is unknown to the adversary before the decryption starts.
- *Reach a set of well-covered aggregated ciphertexts.* At line 12, each honest P_i provides (c_i^*, π_i^*) as input to Align , and it obtains a set $\text{Candidates} = \{(j, c_j^*, \pi_j^*, \theta_j)\}$ at line 13-14, where θ_j is a proof w.r.t. Align , showing (c_j^*, π_j^*) is in the outcome of the Align .
- *Decide the “winner” among local candidates.* At lines 17-18, each honest P_i provides its partial decryption key w.r.t. the tag sid , and then at lines 19-23, it obtains the full decryption key for the tag sid . Then, at lines 24-25, P_i decrypts all ciphertexts in Candidates and then hashes

Algorithm 2 The asynchronous weak coin protocol WeakCoin with identifier sid for P_i

```

1: Initialize:  $\text{Candidates}, \text{LocWins} \leftarrow \emptyset$ , and  $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$  instance.
2: upon activated with  $\text{sid}$  do
3:    $r_i \leftarrow \text{Msg}_{\text{te}}, \gamma_i \leftarrow \text{Rand}_{\text{te}}, c_i \leftarrow \text{STHE.Enc}(ek, \text{sid}, r_i; \gamma_i)$ 
4:    $\sigma_i \leftarrow \text{DS.Sign}(sk_i, (\text{sid}, c_i))$   $\pi_i \leftarrow \text{NIZK.Prove}(\text{crs}_{\text{enc}}, (ek, \text{sid}, c_i, i), (r_i, \gamma_i))$ 
5:   Multicast  $(\text{sid}, \text{FRESH}, c_i, \sigma_i, \pi_i)$ 

6: upon receiving  $(\text{sid}, \text{FRESH}, c_j, \sigma_j, \pi_j)$  from node  $P_j$  for the first time do
7:   if  $\text{DS.Vrfy}(vk_j, (\text{sid}, c_j), \sigma_j) = 1 \wedge \text{NIZK.Vrfy}(\text{crs}_{\text{enc}}, (ek, \text{sid}, c_j, j), \pi_j) = 1$  then
8:      $\text{Fresh} \leftarrow \text{Fresh} \cup \{(j, c_j, \sigma_j, \pi_j)\}$ 
9:     if  $|\text{Fresh}| = f + 1$  and has not invoked  $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$  then
10:       $c_i^* \leftarrow \text{STHE.Eval}(ek, \text{sid}, \{c_j\}_{(j, \dots) \in \text{Fresh}})$  ▷ Aggregate then prove
11:       $\pi_i^* \leftarrow \text{SNARK.Prove}(\text{crs}_{\text{agg}}, (n, f, (vk_i)_{i \in [n]}, ek, \text{sid}, \text{crs}_{\text{enc}}, c^*), \text{Fresh})$ 
12:      Invoke  $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$  with input  $(c_i^*, \pi_i^*)$ 

13: upon  $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$  returns  $\text{OSet}_i = \{(z, c_z^*, \pi_z^*, \theta_z^*)\}$  do
14:    $\text{Candidates} \leftarrow \text{OSet}_i$ 
15:   if  $\text{Candidates} = \emptyset$  then
16:     Multicast  $(\text{sid}, \text{LOCALWIN}, \perp)$ 
17:      $\zeta_i \leftarrow \text{STHE.PartDec}(ek, \text{sid}, dk_i)$ 
18:     Multicast  $(\text{sid}, \text{PartDec}, \zeta_i)$ 

19: upon receiving  $(\text{sid}, \text{PARTDEC}, \zeta_j)$  from  $P_j$  for the first time do
20:   if  $\text{STHE.PartVrfy}(ek, \text{sid}, pk_j, \zeta_j) = 1$  then
21:      $\text{ParK} \leftarrow \text{ParK} \cup \{(j, \zeta_j)\}$ 
22:     if  $|\text{ParK}| = n - f$  and has not sent  $\text{LOCALWIN}$  message then
23:        $dk_{\text{sid}} \leftarrow \text{STHE.DkAgg}(ek, \text{sid}, \{\zeta_k\}_{(k, \cdot) \in \text{ParK}})$  ▷ decryption key
24:       for  $(z, (c_z^*, \pi_z^*), \theta_z) \in \text{Candidates}$  do
25:          $r_z^* \leftarrow \text{STHE.Dec}(ek, \text{sid}, dk_{\text{sid}}, c_z^*)$ , and  $\text{to}_z \leftarrow H_1(\text{sid}, z, r_z^*)$ 
26:       Pick  $a$  such that  $\text{to}_a = \text{MAX}(\{\text{to}_a\}_{(a, \cdot) \in \text{Candidates}})$ .
27:       Multicast  $(\text{sid}, \text{LOCALWIN}, (a, c_a^*, \pi_a^*, \theta_a))$ 

28: upon receiving  $(\text{sid}, \text{LOCALWIN}, (a, c_a^*, \pi_a^*, \theta_a))$  from  $P_j$  for the first time do
29:   if  $(a, c_a^*, \pi_a^*, \theta_a) = \perp$  then  $\text{LocWins} \leftarrow \text{LocWins} \cup \{(a, \perp)\}$ 
30:   if  $\text{Align.Vrfy}(\text{sid}, a, (c_a^*, \pi_a^*), \theta_a) = 1$  then  $r_a^* \leftarrow \text{STHE.Dec}(ek, \text{sid}, dk_{\text{sid}}, c_a^*)$ 
31:    $\text{LocWins} \leftarrow \text{LocWins} \cup \{(a, r_a^*, \text{to}_a = H_1(\text{sid}, a, r_a^*))\}$ 
32:   if  $|\text{LocWins}| = n - f$  then
33:     Decide  $z$  such that  $\text{to}_z = \text{Max}(\{\text{to}_a\})$ 
34:     Return  $H_2(\text{sid}, z, r_z^*)$ , where  $H_2$  is a random oracle whose range is  $V$ .

```

the plaintexts into a total order set. At lines 26-27, P_i picks the ciphertext c_a^* whose decryption result yields the largest hash value as the “local winner”, and then multicasts c_a^* with its validity proof in a LOCALWIN message. Note if **Candidates** is empty, P_i multicasts $\perp$ as a placeholder at lines 15-16.

- *Decide the output.* At lines 28-33, P_i collects $n - f$ LOCALWIN messages and then applies the same rule to decide the winner among all the ciphertexts carried by this textsc LocalWin messages. Applying another random oracle to the decryption of the winner ciphertext, P_i obtains the output value.

Efficiency Analysis. According to Algorithm 2, and as clearly shown in Fig.3, the protocol WeakCoin has 3 more rounds besides Align which in turn has 9 rounds. Hence, WeakCoin requires 12 rounds. In addition, since the input size to Align is merely $O(\lambda)$, the communication complexity due to Align is $O(\lambda n^2)$. Then, it is easy to verify the overall communication complexity is $O(\lambda n^2)$.

Instantiation and Assumptions. The STHE scheme is developed in Appendix D, which is secure in the generic group model (GGM) and the random oracle model (ROM). The GGM and ROM indeed have subsumed the assumptions needed for other components. Specifically, the Align protocol requires the q -SDH assumption and the co-CDH assumption in the AGM, which can be implied by GGM, according to [63]. We can instantiate the underlying digital signature scheme with the Boneh-Boyen signature [18] and the SNARK with Groth16 [56], whose assumptions are all subsumed by GGM. Regarding the NIZK, it is easy to see that R_{enc} falls in the family of relations supported by the Groth-Sahai proof system [58]. So, a NIZK with simulation extractability for R_{enc} can be realized by applying the generic transformation [69] to the Groth-Sahai [58], and the assumptions needed for the NIZK can be subsumed by GGM.

6.2 Security Analysis

We establish the security results of our WeakCoin in Theorem 2. Before proceeding to it, in the following, we consider a security game, which we call Random Aggregation Unpredictability (raUNP). Basically, this security notion captures all security guarantees that WeakCoin needs from the underlying STHE, NIZK, DS, and SNARK. It says that any PPT adversary $\mathcal{A}$ cannot produce an aggregated ciphertext c^* along with a valid SNARK proof π^* while knowing the decryption of c^* . Here π^* shows c^* is an honest aggregation of $f + 1$ ciphertext from $f + 1$ distinct nodes, i.e., the relation R_{agg} in Eq.3. The rest of the proofs for Theorem 2 can be built upon the raUNP security without giving a security reduction to the security of each cryptographic component.

Random aggregation unpredictability. For facilitating the security analysis, we consider the *Random Aggregation Unpredictability* (raUNP) security game $\text{Exp}_{\mathcal{A}}^{\text{raUNP}}$ in Fig.7. In Lemma.5, we prove that the probability of winning the security is negligible for any PPT adversary.

Lemma 5. *Assume STHE, NIZK, SNARK, and DS satisfy the security properties outlined in Sect.6.1. For any PPT adversary $\mathcal{A}$, any $\lambda \in \mathbb{N}$, any positive integers f and $n = 3f + 1$ that are polynomial in λ , any $\text{tag}^* \in \{0, 1\}^*$, it holds that*

$$\text{Adv}_{\mathcal{A}, \text{tag}^*}^{\text{raunp}}(\lambda) := \Pr[\text{Exp}_{\text{raUNP}}^{\mathcal{A}, \text{tag}^*}(1^\lambda, n, f) = 1] \leq \text{negl}(\lambda),$$

where the experiment is defined in Fig.7.

$\text{Exp}_{\text{raUNP}}^{\mathcal{A}, \text{tag}^*}(1^\lambda, n, f)$	
$\mathcal{C}, \text{Tags}, \text{State}_1, \dots, \text{State}_n \leftarrow \emptyset.$ Generate CRS := (crs _{enc} , crs _{agg} , crs _{sthe}) $(\mathcal{C}_{\text{init}}, \text{state}_{\mathcal{A}}) \leftarrow \mathcal{A}(\text{CRS}), \mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}_{\text{init}}$ for $i \notin \mathcal{C}_{\text{init}}$, Generate $(ek_i, dk_i), (vk_i, sk_i)$, provide public keys to $\mathcal{A}$, and wait $\mathcal{A}$ to provide public keys of corrupted parties. Let ek be the group encryption key for $i \in [n] \setminus \mathcal{C}_{\text{init}}$ $\gamma_i \leftarrow \text{Rand}_{\text{te}}, r_i \leftarrow \text{Msg}_{\text{te}}, \hat{\gamma}_i \leftarrow \text{Rand}_{\text{zk}}, \tilde{\gamma}_i \leftarrow \text{Rand}_{\text{sig}}, c_i \leftarrow \text{STHE.Enc}(ek, \text{tag}^*, r_i; \gamma_i)$ $\pi_i \leftarrow \text{NIZK.Prove}(\text{crs}_{\text{enc}}, (ek, \text{tag}^*, c_i, i), (r_i, \gamma_i); \hat{\gamma}_i); \sigma_i \leftarrow \text{DS.Sign}(sk_i, (\text{tag}^*, c_i); \tilde{\gamma}_i)$ $(c^*, \pi^*, r^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Corr}}(\cdot), \mathcal{O}_{\text{PartD}}(\cdot), \mathcal{O}_{\text{Sign}}(\cdot)}(\text{state}_{\mathcal{A}}, (c_i, \pi_i, \sigma_i)_{i \in [n] \setminus \mathcal{C}_{\text{init}}})$ if $\text{SNARK.Vrfy}(\text{crs}_{\text{agg}}, (n, f, (vk_i)_{i \in [n]}, ek, \text{tag}^*, \text{crs}_{\text{enc}}, c^*), \pi^*) = 1$ $\wedge r^* = \text{STHE.Dec}(ek, dk_{\text{tag}^*}, c^*)$ then return 1 else return 0	
$\mathcal{O}_{\text{Sign}}(i, (\text{tag}, \text{msg}))$	$\mathcal{O}_{\text{PartD}}(\text{tag}, i)$
if $\text{tag} \neq \text{tag}^* \wedge i \notin \mathcal{C}$ Tags $\leftarrow \text{Tags} \cup \{\text{tag}\}, \gamma \leftarrow \text{Rand}_{\text{Sign}}$ $\sigma_i \leftarrow \text{DS.Sign}(sk_i, (\text{tag}, \text{msg}); \gamma)$ State _{i} $\leftarrow \text{State}_i \cup \{(\text{Sign}, i, \text{tag}, \text{msg}, \gamma, \sigma_i)\}$ return σ_i	if $\text{tag} \neq \text{tag}^* \wedge i \notin \mathcal{C}$ Tags $\leftarrow \text{Tags} \cup \{\text{tag}\}, \gamma \leftarrow \text{Rand}_{\text{PartD}}$ $\zeta_i \leftarrow \text{PartDec}(ek, \text{tag}, dk_i; \gamma)$ State _{i} $\leftarrow \text{State}_i \cup \{(\text{PartD}, i, \text{tag}, \gamma, \zeta_i)\}$ return ζ_i
$\mathcal{O}_{\text{Corr}}(i)$	
if $i \in [n] \setminus \mathcal{C} \wedge \mathcal{C} \leq f$ $\mathcal{C} \leftarrow \mathcal{C} \cup \{i\};$ return $dk_i, sk_i, r_i, \gamma_i, \hat{\gamma}_i, \tilde{\gamma}_i, \text{State}_i$	

Fig. 7. ra-UNP Security Game.

Proof Intuitions. In raUNP, an adversary can see n ciphertexts and selectively open up to f of them, with the goal of producing a ciphertext along with its decryption, such that the ciphertext is proved to be an aggregation of $f + 1$ ciphertexts.

Usually, as the adversary can selectively corrupt some ciphertexts, the non-committing or equivocation property of the underlying STHE would be expected. However, we manage to prove the raUNP without assuming equivocation of STHE by leveraging two facts: (1) These ciphertexts encrypt independent randomness; (2) We only need the aggregated ciphertext to be unpredictable instead of being indistinguishable. Specifically, we applied the single inconsistency party proof technique. In the proof, the challenger of raUNP obtains a challenge ciphertext from the STHE game, and uniformly sampled $i^* \leftarrow [n]$, so that the i^* -th ciphertext is the challenge ciphertext, while all other ciphertexts are generated honestly. Then, when the adversary chooses to open the j -th ciphertext for $j \neq i^*$, the challenger can return the encryption randomness; if choosing to open i^* -th ciphertext, the challenger aborts. Finally, there is a non-negligible probability that i^* -th ciphertext is not opened and has been aggregated to the final ciphertext. In that case, the challenger can use the output from the adversary to win the STHE game.

Proof. We prove this lemma by using the following hybrid experiments. We use Win_k to denote that **Hybrid** k returns 1.

Hybrid 0. It is the experiment defined in Fig. 7.

Hybrid 1. It is almost identical to **Hybrid 0**, except that it has an additional rule:

- In the beginning, uniformly sample $\bar{i} \leftarrow \$ [n]$. If, during the protocol execution, $\mathcal{A}$ chooses to corrupt $P_{\bar{i}}$, then abort the experiment.

At the point of $\mathcal{A}$'s view, as long as the experiment does not abort, then it is identical to **Hybrid 0**. Since the choice of $\bar{i}$ is independent of $\mathcal{A}$'s view, the probability that $\mathcal{A}$ chooses to corrupt $\bar{i}$ is bounded by f/n . Thus, $\Pr[\text{Win}_1] \geq \Pr[\text{Win}_0] \cdot (\frac{n-f}{n})$.

Hybrid 2. It is almost identical to Hybrid 1, except that it uses the simulated setup of NIZK to generate the CRS crs_{enc} along with the trapdoor tk , and $\pi_{\bar{i}}$ is generated with the simulated prover algorithm using tk .

Ensured by the zero-knowledge of NIZK, we have $|\Pr[\text{Win}_2] - \Pr[\text{Win}_1]| \leq \text{negl}(\lambda)$.

Hybrid 3. It is almost identical to Hybrid 2, except that under the condition that Hybrid 2 does not abort, after $\mathcal{A}$ provides the tuple (c^*, π^*, r^*) , it additionally applies the extractor algorithm $E_{\mathcal{A}}$, whose existence is granted by the knowledge soundness of SNARK, to extract the knowledge $\{j, c_j, \sigma_j, \pi_j\}_{j \in \mathbb{I}'}$ for some $\mathbb{I}' \subset [n]$ and $|\mathbb{I}'| = f + 1$. For every $j \in \mathbb{I}'$, check whether $\text{DS.Vrfy}(vk_j, \sigma_j, c_j) = 1$ and $\text{NIZK.Vrfy}(\text{crs}_{\text{enc}}, (ek, \text{tag}^*, c_j, j), \pi_j) = 1$. It aborts if any checking does not pass.

If Hybrid 3 does not abort due to the new checking procedure, then at the point of $\mathcal{A}$'s view, it is identical to Hybrid 2. Ensured by the knowledge soundness of SNARK, the probability that the extraction fails is negligible. Therefore, $|\Pr[\text{Win}_3] - \Pr[\text{Win}_2]| \leq \text{negl}(\lambda)$.

Hybrid 4. It is almost identical to Hybrid 3, except that under the condition that Hybrid 3 does not abort, after extracting the knowledge $\{j, c_j, \sigma_j, \pi_j\}_{j \in \mathbb{I}'}$, it additionally checks whether $\bar{i} \in \mathbb{I}'$, and if not, it aborts.

At the point of $\mathcal{A}$'s view, as long as the experiment does not abort due to the new checking procedure, then it is identical to Hybrid 3. Notice that $\mathcal{A}$ has to include at least one corrupted index into $\mathbb{I}'$, and the choice of $\bar{i}$ is independent of $\mathcal{A}$'s view. Hence, the probability that $\mathcal{A}$ does not include $\bar{i}$ into $\mathbb{I}'$ is bounded by $\frac{1}{n-f}$. It follows that $\Pr[\text{Win}_4] \geq \Pr[\text{Win}_3] \cdot (\frac{1}{n-f})$.

Hybrid 5. It is almost identical to Hybrid 4, except that under the condition that Hybrid 4 does not abort, for any $j \in \mathbb{I}' \cap \mathcal{C}$, it additionally applies the simulation extractor SimExt , whose existence is granted by the simulation extractability of NIZK, to extract the knowledge of (r_j, γ_j) w.r.t. c_j for all $j \in \mathbb{I}' \setminus \{\bar{i}\}$, such that $\text{STHE.Enc}(ek, \text{tag}^*, r_j; \gamma_j) = c_j$. If the extraction fails, it aborts.

Notice that since the statement $(ek, \text{tag}^*, c_j, j)$ w.r.t. NIZK contains the identity j . Therefore, for all $j \neq \bar{i}$, the statement is not equal to the statement of the simulated proof. Hence, ensured by simulation extractability of NIZK, the probability that the extraction fails is negligible. It follows that $|\Pr[\text{Win}_5] - \Pr[\text{Win}_4]| \leq \text{negl}(\lambda)$.

Hybrid 6. It is almost identical to Hybrid 5, except that under the condition that Hybrid 5 does not abort, it additionally checks whether the tuple $(\bar{i}, c_{\bar{i}}, \sigma_{\bar{i}}, \pi_{\bar{i}})$, extracted by $E_{\mathcal{A}}$, contains the same ciphertext $c_{\bar{i}}$ that was honestly generated and provided to $\mathcal{A}$ in the experiment. If not, it aborts.

Note that the party $P_{\bar{i}}$ is not corrupted throughout the execution, and $(\text{sid}, c_{\bar{i}})$ has a valid signature under $vk_{\bar{i}}$. Due to the unforgeability of DS, the probability that $c_{\bar{i}}$ is different from the honestly generated ciphertext is negligible. Thus, $|\Pr[\text{Win}_6] - \Pr[\text{Win}_5]| \leq \text{negl}(\lambda)$.

Following the above arguments, we can see that $\Pr[\text{Win}_6] \geq \frac{\Pr[\text{Win}_0]}{n} - \text{negl}(\lambda)$. Therefore, if $\Pr[\text{Win}_0]$ is non-negligible, then $\Pr[\text{Win}_6]$ must be non-negligible.

On the other hand, assuming there is a PPT adversary $\mathcal{A}$ such that $\Pr[\text{Win}_6] > \text{negl}(\lambda)$, we can build a PPT adversary $\mathcal{B}$ against the adaptive semantic security of STHE, such that $\text{Adv}_{\mathcal{B}}^{\text{aIND}}(\lambda) > \text{negl}(\lambda)$. $\mathcal{B}$ works as follows.

- **Setup:** $\mathcal{B}$ uniformly samples $\bar{i} \leftarrow_{\$} [n]$, runs a copy of $\mathcal{A}$ and finishes the setup phase in the following way: for the setup w.r.t. STHE, $\mathcal{B}$ just forwards the messages it receives in the aIND experiment; for the setup w.r.t. the digital signature DS, and SNARK, $\mathcal{B}$ follows the specifications in Fig.7. For NIZK, it runs the simulation setup NIZK.SimSetup to generate the CRS and its trapdoor tk . If $\mathcal{A}$ requests to corrupt the party P_i , it returns $b' \leftarrow_{\$} \{0, 1\}$ and aborts.
- **Prepare ciphertexts, proofs, and signatures:** For every $j \in [n] \setminus (\mathcal{C}_{\text{Init}} \cup \{\bar{i}\})$, it generates the ciphertext c_j , the proof π_j , and the signature σ_j by following the specifications in Fig.7. Regarding the ciphertext $c_{\bar{i}}$, it uniformly samples $m_0, m_1 \leftarrow_{\$} \text{Msg}_{\text{te}}$, queries the oracle $\mathcal{O}_b$ with (m_0, m_1) in the experiment aIND, and assigns $c_{\bar{i}} \leftarrow c_b$ which is returned by $\mathcal{O}_b$. Then, it generates a simulated proof $\pi_{\bar{i}}$ using tk , and signs it honestly. All $(c_i, \pi_i, \sigma_i)_{i \in [n] \setminus \mathcal{C}_{\text{Init}}}$ are then provided to $\mathcal{A}$.
- **Respond queries to $\mathcal{O}_{\text{PartD}}$:** $\mathcal{B}$ just forwards these requeries to $\mathcal{O}_{\text{PartD}}$ in the experiment aIND.
- **Respond queries to $\mathcal{O}_{\text{Sign}}$:** For a query $(i, (\text{tag}, \text{msg}))$, $\mathcal{B}$ responds it honestly as specified in Fig.7, using the signing key of P_i .
- **Respond queries to $\mathcal{O}_{\text{Corr}}$:** When $\mathcal{A}$ queries $\mathcal{O}_{\text{Corr}}$ with i , if $i = \bar{i}$, it returns $b' \leftarrow_{\$} \{0, 1\}$ and aborts. Otherwise, it requies $\mathcal{O}_{\text{Corr}}$ in the aIND experiment with i and obtains the decryption key dk_i along $\text{State}_{\text{PD}, i}$. Then, it returns $(dk_i, sk_i, \gamma_i, \hat{\gamma}_i, \bar{\gamma}_i, \text{State}_i)$ to $\mathcal{A}$, where sk_i is the signing key of P_i , $\gamma_i, \hat{\gamma}_i, \bar{\gamma}_i$ are the randomness for generating the ciphertext, the proof, and the signature; State_i contains the randomness used to respond each query to $\mathcal{O}_{\text{Sign}}$ and $\mathcal{O}_{\text{PartD}}$.
- **Output:** After $\mathcal{A}$ returns (c^*, π^*, r^*) , it applies the extractor $E_{\mathcal{A}}$ to obtain the list $\{j, c_j, \sigma_j, \pi_j\}_{j \in \mathbb{I}'}$ as specified in Hybrid 3, applies the extractor SimExt to obtain the plaintext and randomness $(r_j, \gamma_j)_{j \in \mathbb{I}' \setminus \bar{i}}$ as specified in Hybrid 5. It will return $b' \leftarrow_{\$} \{0, 1\}$ and aborts under the same conditions of Hybrid 6. If not abort, then it calculates $r'_i \leftarrow r^* \ominus \bigoplus_{j \in \mathbb{I}' \setminus \{\bar{i}\}} r_j$. Then, if $r'_i = m_0$, $\mathcal{B}$ returns 0; If $r'_i = m_1$, $\mathcal{B}$ returns 1. In any other cases, it returns $b' \leftarrow_{\$} \{0, 1\}$.

It is easy to verify that $\mathcal{B}$ perfectly simulates the experiment Hybrid 6 in $\mathcal{A}$'s view. For notational simplicity, we still use Win_6 to denote the following event: $\mathcal{B}$ does not abort, and $\mathcal{A}$ returns (c^*, π^*, r^*) such that π^* is a valid proof for c^* w.r.t. SNARK and r^* is the decryption of c^* .

Now, we analyze $\text{Adv}_{\mathcal{B}}^{\text{aIND}}(\lambda) = |\Pr[\text{aIND}_1^{\mathcal{B}}] - \Pr[\text{aIND}_0^{\mathcal{B}}]|$. Note that due to the *evaluation correctness* of STHE, when Win_6 happens, there exists γ^* such that $c^* = \text{STHE.Enc}(ek, \text{tag}^*, \bigoplus_{j \in \mathbb{I}'} r_j \oplus m_b)$. Then, due to the decryption correctness, we have $r^* = \bigoplus_{j \in \mathbb{I}'} r_j \oplus m_b$. It follows that $\Pr[\text{aIND}_1^{\mathcal{B}} | \text{Win}_6] = 1$, while $\Pr[\text{aIND}_0^{\mathcal{B}} | \text{Win}_6] = 0$. On the other hand, there is a special event SE that $\mathcal{A}$ returns (c^*, π^*, r^*) but $r^* = \bigoplus_{j \in \mathbb{I}'} r_j \oplus m_{1-b}$. It is easy to verify that $\Pr[\text{aIND}_1^{\mathcal{B}} | \text{SE}] = 0$, while $\Pr[\text{aIND}_0^{\mathcal{B}} | \text{SE}] = 1$. However, since m_{1-b} is uniformly sampled and independent of $\mathcal{A}$'s view, so $\Pr[\text{SE}] = \frac{1}{|\text{Msg}_{\text{te}}|}$. In addition, under the condition that $\neg(\text{SE} \cup \text{Win}_6)$, $\mathcal{B}$ simply returns a uniformly sampled bit, so $\Pr[\text{aIND}_1^{\mathcal{B}} | \neg(\text{SE} \cup \text{Win}_6)] = \Pr[\text{aIND}_0^{\mathcal{B}} | \neg(\text{SE} \cup \text{Win}_6)] = \frac{1}{2}$. Putting the above facts together, we have:

$$\begin{aligned}
|\Pr[\text{aIND}_1^{\mathcal{B}}] - \Pr[\text{aIND}_0^{\mathcal{B}}]| &= |\Pr[\text{aIND}_1^{\mathcal{B}} | \text{Win}_6] \Pr[\text{Win}_6] + \Pr[\text{aIND}_1^{\mathcal{B}} | \text{SE}] \Pr[\text{SE}] \\
&\quad - \Pr[\text{aIND}_0^{\mathcal{B}} | \text{Win}_6] \Pr[\text{Win}_6] - \Pr[\text{aIND}_0^{\mathcal{B}} | \text{SE}] \Pr[\text{SE}]| \\
&= |\Pr[\text{Win}_6] - \Pr[\text{SE}]|
\end{aligned}$$

If $\Pr[\text{Win}_6]$ is non-negligible, then $\text{Adv}_{\mathcal{B}}^{\text{aind}}(\lambda)$ is also non-negligible. Under the assumption that STHE satisfies the adaptive semantic security for (n, f) , $\text{Adv}_{\mathcal{B}}^{\text{aind}}(\lambda)$ is negligible for any PPT $\mathcal{B}$, so $\Pr[\text{Win}_6]$ (and thus $\Pr[\text{Win}_0]$) must also be negligible. $\square$

Security of our weak coin protocol. We establish the following theorem for our weak coin protocol in Algorithm 2. In particular, following the composition security lemmas in Sect. 4.2, we prove an “enhanced” version of the security, where the adversary against the instance $\text{WeakCoin}[\text{sid}]$ is given additional access to the following oracles:

- $\mathcal{O}_{\widehat{\text{PartD}}}$: On input (sid', i) , if $\text{sid}' \neq \text{sid}$, returns $\text{STHE.PartD}(ek, \text{sid}', dk_i)$
- $\mathcal{O}_{\widehat{\text{Sign}}}$: On input $(\text{sid}', i, \text{msg})$, if $\text{sid}' \neq \text{sid}$, returns $\text{DS.Sign}(sk_i, (\text{sid}', \text{msg}))$.
- Oracles in $\mathbb{O}_{\text{Align}}$, defined in Lemma 15 for the underlying Align.

We denote the set of the above oracles as well as the corruption oracle by $\mathbb{O}_{\text{WeakCoin}}$.

Theorem 2. *The WeakCoin protocol in Algorithm 2 satisfies the termination and φ -fairness for $\varphi = \frac{n-f}{n}$, even when the adversary has additional access to the oracles in $\mathbb{O}_{\text{WeakCoin}}$.*

We prove the termination in Lemma 6 and the φ -fairness in Lemma 7, respectively. We argue in Proposition 6 in Appendix E that Align remains secure in the composition.

Lemma 6 (Termination of WeakCoin). *WeakCoin satisfies the termination property, even when the adversary has additional access to oracles in $\mathbb{O}_{\text{WeakCoin}}$.*

Proof. We prove the termination by showing every honest node can execute the code of Line 32-34 in Algorithm 2. For Lines 1-5, every honest node can perform the task without issues. For 6-12, due to the completeness of the underlying NIZK and the correctness of the underlying DS, every honestly generated proof π_j and signature σ_i can be verified, so that $(j, c_j, \sigma_j, \pi_j)$ carried by the FRESH message will be included in the set Fresh. Since all honest nodes have multicasted the FRESH message, the condition in Line 9 can be satisfied, and every honest node can execute Line 9-12.

Every honest node P_i will invoke the $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$ with (c_i^*, π_i^*) . Due to the completeness of SNARK, π_i^* is a valid proof w.r.t. c_i^* and the public parameters, so (c_i^*, π_i^*) satisfies the predicate of $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$. Since all honest nodes provide valid inputs to $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$, every honest node P_i can terminate in $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$ with a set OSet_i (which can be empty). Therefore, Lines 13-14 and Lines 17-18 can be executed by all honest nodes. Following the correctness of STHE, all honest nodes can receive enough valid partial decryption keys, so Lines 19-21 can be executed by all honest nodes. In addition, every honest node will execute either Line 15-16 or Line 22-27, and due to the $f + 1$ -alignment property of Align, at least $f + 1$ honest nodes execute Line 22-27. Ensured by the correctness of STHE and termination of Align, all honestly generated LOCALWIN messages will be included in the set LocWins. When $|\text{LocWins}| = n - f$, at least one of element in LocWins carries $(a, r_a^*, to_a) \neq \perp$. Thus, every honest node can execute Line 34 and return a value.

Note that giving $\mathcal{A}$ access to oracles in $\mathbb{O}_{\text{WeakCoin}}$ does not violate any security properties of the underlying primitives that we need in the above analysis. $\square$

Lemma 7 ($\frac{n-f}{n}$ -Fairness). *WeakCoin satisfies the φ -Fairness property with $\varphi = \frac{n-f}{n}$, even when the adversary has additional access to oracles in $\mathbb{O}_{\text{WeakCoin}}$.*

Proof. We first describe an event that happens with the probability φ , conditioned on which the agreement, the unpredictability, and the bias-resistance are granted. Due to the $(f + 1)$ -alignment property (binding cover property and well-covered set property), at the first time that an honest node P_i receives an output from $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$, there exists the well-covered set $\text{WS} = \{(j, (c_j^*, \pi_j^*), \cdot)\}_{j \in \mathbb{I}_s}$ whose size is $n - f$, as well as the binding cover $\text{CoverSet} = \{(j, (c_j^*, \pi_j^*), \cdot)\}_{j \in \mathbb{I}_c}$ whose size is at most n . Note that we have $\text{WS} \subset \text{CoverSet}$ (while well-formedness proofs are ignored). With this notion, we can define the following event:

- **Fair:** Let dk_{sid} be the decryption key obtained by an honest node at Line 23. Let $r_j^* = \text{STHE.Dec}(ek, \text{sid}, dk_{\text{sid}}, c_j^*)$, and let $\text{to}_j = H_1(\text{sid}, r_j^*)$, for $j \in \mathbb{I}_c$. In this event, there exists an index $j_{\max} \in \mathbb{I}_s$, such that $\text{to}_{j_{\max}}$ is the largest value among all $\{\text{to}_j\}_{j \in \mathbb{I}_c}$.

In Proposition 2, we show **Fair** happens with a probability of at least $\varphi - \text{negl}(\lambda)$, despite any attacking strategy. Then, we prove that agreement and unpredictability in Proposition 3 and Proposition 4, respectively. These proofs rely on the fact that the random oracles H_1 and H_2 had not been queried with (sid, j, r_j^*) for any $j \in \mathbb{I}_c$ until the first time that an honest node finishes $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$, which we proved in Proposition 1. $\square$

Proposition 1. *At the first time that an honest node P_i finishes the $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$ instance, the probability that H_1 or H_2 has been queried with (sid, j, r_j^*) for any $j \in \mathbb{I}_c$ is negligibly small.*

Proof. We prove this lemma by contradiction. Assume that there exists a PPT $\mathcal{A}$, such that in execution of $\text{WeakCoin}[\text{sid}]$, there is a non-negligible probability that H_1 or H_2 has been queried with the specified (sid, j, r_j^*) at the first time that an honest node P_i finishes the $\text{Align}[\text{sid}|\text{ali}]$ instance. Then, we can build a PPT adversary $\mathcal{B}$ which can win the raUNP security game in Fig. 7, i.e., $\text{Exp}_{\text{raUNP}}^{\mathcal{B}}(1^\lambda, n, f) = 1$, with a non-negligible probability.

We outline the strategy of $\mathcal{B}$ as follows, which runs a copy of $\mathcal{A}$ and tries to simulate the execution of our weak coin protocol until an honest node finishes the Align subroutine.

- **Setup Phase:** $\mathcal{B}$ forwards the messages it received in the raUNP security game to $\mathcal{A}$ and forwards the corruption requests and public keys of corrupted parties from $\mathcal{A}$ to the security game. In addition, the setup for Align is run by $\mathcal{B}$. Initialize two sets $\mathcal{H} \subset [n]$ and $\mathcal{C} \subset [n]$ to track the identities of honest parties and corrupted parties, respectively.
- **Protocol Simulation:** Note that in the raUNP security game, $\mathcal{B}$ receives $(c_i, \pi_i, \sigma_i)_{i \in \mathcal{C}_{\text{init}}}$. Then, $\mathcal{B}$ acts on the behalf of every so-far-honest node P_i to multicast $(\text{sid}, \text{FRESH}, c_i, \sigma_i, \pi_i)$. Next, $\mathcal{B}$ honestly follows the protocol specification to act on behalf of all honest nodes. Random oracle queries are responded to in a standard way, and $\mathcal{B}$ maintains two tables Hist_1 and Hist_2 of the query records.
- **Handle Oracle Queries:** Note that $\mathcal{A}$ can query the additional oracles in

$$\mathbb{O}_{\text{weakcoin}} = \{\mathcal{O}_{\widetilde{\text{PartD}}}, \mathcal{O}_{\widetilde{\text{Sign}}}\} \cup \mathbb{O}_{\text{Align}}.$$

When $\mathcal{A}$ queries $\mathcal{O}_{\widetilde{\text{PartD}}}$ (resp. $\mathcal{O}_{\widetilde{\text{Sign}}}$) with (sid', i) (resp. $(\text{sid}', i, \text{msg})$) such that $i \in \mathcal{H}$ and $\text{sid}' \neq \text{sid}$, $\mathcal{B}$ queries $\mathcal{O}_{\widetilde{\text{PartD}}}$ (resp. $\mathcal{O}_{\widetilde{\text{Sign}}}$) in the raUNP security game with (sid', i) (resp. $(i, (\text{sid}', \text{msg}))$) and forwards the message from $\mathcal{O}_{\widetilde{\text{PartD}}}$ to $\mathcal{A}$. Regarding queries to oracles in $\mathbb{O}_{\text{Align}}$, $\mathcal{B}$ answers them honestly by using the secret states of honest parties w.r.t. $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$.

- *Handle Corruption:* When $\mathcal{A}$ corrupts P_i , $\mathcal{B}$ checks if the number of corrupted parties has reached the limit. If not, $\mathcal{B}$ queries $\mathcal{O}_{\text{Corr}}$ with i , which returns $(dk_i, sk_i, r_i, \gamma_i, \hat{\gamma}_i, \bar{\gamma}_i, \text{State}_i)$. Then, it returns the tuple along with other secrets and randomness for SNARK and Align to $\mathcal{A}$.
- *Output:* At the first time that an honest node finishes Align, $\mathcal{B}$ obtains the binding cover $\text{CoverSet} = \{(j, (c_j^*, \pi_j^*), \cdot)\}_{j \in \mathbb{I}_c}$ as well as the set of well-covered elements WS. As ensured by Align, (c_j^*, π_j^*) shall satisfy the predicate of Align, i.e.,

$$\text{SNARK.Vrfy}(\text{crs}_{\text{agg}}, (n, f, (vk_i)_{i \in [n]}, ek, \text{tag}^*, \text{crs}_{\text{enc}}, c_j^*), \pi_j^*) = 1.$$

Then, $\mathcal{B}$ extracts all records in Hist_1 and Hist_2 with the input form (sid, j, r_j^*) for $j \in \mathbb{I}_c$, and uniformly picks one record (sid, z, r_z^*) from them. It outputs (c_z^*, π_z^*, r_z^*) in the raUNP security game.

It is easy to verify, in $\mathcal{A}$'s view, that the execution simulated by $\mathcal{B}$ is identical to a real execution of $\text{WeakCoin}[\text{sid}]$. Therefore, there is a non-negligible probability that a tuple (sid, j, r_j^*) , where $j \in \mathbb{I}_c$ and r_j^* is the decryption of c_j^* , has been issued to H_1 and H_2 . Moreover, since the number of query records must be polynomial in λ , the probability that $\mathcal{B}$ happens to pick this tuple is at least $\frac{1}{\text{poly}(\lambda)}$. There would be a non-negligible probability that $\mathcal{B}$ wins in the raUNP security game, which contradicts Lemma 5. $\square$

Proposition 2. *The event **Fair** (defined in Lemma 7) happens with a probability of at least $\varphi - \text{negl}(\lambda)$.*

Proof. For ease of discussion, we consider the following event:

- **EarlyQ:** H_1 or H_2 has been queried with (sid, j, r_j^*) for any $j \in \mathbb{I}_c$ before the first time an honest node finishes $\text{Align}[(\text{sid}, \text{ali})]$.

As shown in Lemma 1, we know $\Pr[\text{EarlyQ}] \leq \text{negl}(\lambda)$. Note that at the first time an honest node finishes $\text{Align}[(\text{sid}, \text{ali})]$, $\text{WS} = \{(j, c_j^*, \cdot)\}_{j \in \mathbb{I}_s}$ and $\text{CoverSet} = \{j, c_j^*, \cdot\}_{j \in \mathbb{I}_c}$ have been decided, so whether a tuple $(j, c_j^*, \cdot)$ is included in WS and CoverSet is independent of the values $\{\text{to}_j = H_1(\text{sid}, j, r_j^*)\}$. Therefore, for each $j \in \mathbb{I}_c$, $\Pr[\text{to}_j = \text{Max}(\{\text{to}_i\}_{i \in \mathbb{I}_c}) | \neg \text{EarlyQ}] = \frac{1}{|\mathbb{I}_c|}$, and $\Pr[\text{Max}(\{\text{to}_i\}_{i \in \mathbb{I}_c}) \in \{\text{to}_i\}_{i \in \mathbb{I}_s} | \neg \text{EarlyQ}] = \frac{n-f}{|\mathbb{I}_c|}$, where the probability is taken over the choice of the random function inside the random oracle H_1 . I.e., $\Pr[\text{Fair} | \neg \text{EarlyQ}] = \frac{n-f}{|\mathbb{I}_c|}$. Therefore,

$$\begin{aligned} \Pr[\text{Fair}] &\geq \Pr[\text{Fair} | \neg \text{EarlyQ}] \Pr[\neg \text{EarlyQ}] \\ &\geq \frac{n-f}{|\mathbb{I}_c|} \cdot (1 - \text{negl}(\lambda)) \geq \frac{n-f}{n} - \text{negl}(\lambda) \end{aligned}$$

where the probability is taken over the choice of the random function inside the random oracle H_1 , the randomness of the adversary $\mathcal{A}$, and the randomness of all honest nodes. $\square$

Proposition 3. *Under the condition that the event **Fair** (defined in Lemma 7) happens, all honest nodes output the same value.*

Proof. Let $i_{\max} \in \mathbb{I}_s$ be the index such that $\text{to}_{i_{\max}}$ is the largest value among all $\{\text{to}_j\}_{j \in \mathbb{I}_c}$. Ensured by the *well-covered set* property of Align, the tuple $(i_{\max}, (c_{i_{\max}}^*, \pi_{i_{\max}}^*), \theta_{i_{\max}})$ is in the output sets of at least $f+1$ forever-honest parties, and all of these $f+1$ honest

parties will multicast $(\text{sid}, \text{LOCALWIN}, (i_{\max}, c_{i_{\max}}^*, \pi_{i_{\max}}^*, \theta_{i_{\max}}))$. Hence, after receiving $n - f$ LOCAL-WIN messages, all honest nodes can receive a message carrying $(\text{sid}, \text{LOCALWIN}, (i_{\max}, c_{i_{\max}}^*, \pi_{i_{\max}}^*, \theta_{i_{\max}}))$. Meanwhile, due to the *binding cover* property of Align, all tuples passing the Align.Vrfy check must have been included in $\text{CoverSet} = \{(j, c_j^*, \pi_j^*)\}_{j \in \mathbb{I}_c}$. Thus, all honest nodes must decide on $\text{to}_{i_{\max}}$ is the largest value among all $\{H_1(\text{sid}, j, r_j^*)\}_{j \in \mathbb{I}_c}$, which ensures the agreement property v . $\square$

Proposition 4. *Under the condition that the event **Fair** happens, the output v of an honest node is unpredictable to $\mathcal{A}$. Namely, let $\text{State}_{\mathcal{A}}^{\text{before}}$ be the internal state of $\mathcal{A}$ before $n - f - |\mathcal{C}|$ honest nodes participating $\text{WeakCoin}[\text{sid}]$, then, for any PPT algorithm $\mathcal{D}$, and $u \leftarrow \$V$,*

$$|\Pr[\mathcal{D}(\text{State}_{\mathcal{A}}^{\text{before}}, v) = 1] - \Pr[\mathcal{D}(\text{State}_{\mathcal{A}}^{\text{before}}, u) = 1]| \leq \text{negl}(\lambda).$$

Proof. Let $\text{State}'_{\mathcal{A}}$ be the state of $\mathcal{A}$ at the first time that an honest node finishes $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$. Since when an honest node finishes $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$, $n - f - |\mathcal{C}|$ nodes must have already participated in the instance, we can assume, without loss of generality, that $\text{State}_{\mathcal{A}}^{\text{before}}$ has been encoded in $\text{State}'_{\mathcal{A}}$. Therefore, it would suffice to show $|\Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, v) = 1] - \Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, u) = 1]| \leq \text{negl}(\lambda)$.

Recall the proof of Lemma 2 where we defined an event **EarlyQ**, in which H_1 or H_2 has been queried with (sid, j, r_j^*) for any $j \in \mathbb{I}_c$ before the first time an honest node finishes $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$. Under the condition $\neg \text{EarlyQ}$, the index $i_{\max}$ is determined independent of H_2 . Therefore, $\Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, H_2(\text{sid}, i_{\max}, r_{i_{\max}}^*)) = 1 | \neg \text{EarlyQ}] = \Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, u) = 1]$. Under the condition that **Fair** happens, as we analyzed in Lemma 3, every honest node outputs $v = H_2(\text{sid}, i_{\max}, r_{i_{\max}}^*)$. Hence, $\Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, v) = 1 | \neg \text{EarlyQ}] = \Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, u) = 1]$. Note that

$$\begin{aligned} \Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, v) = 1] &= \Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, v) = 1 | \neg \text{EarlyQ}] \Pr[\neg \text{EarlyQ}] \\ &\quad + \Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, v) = 1 | \text{EarlyQ}] \Pr[\text{EarlyQ}] \end{aligned}$$

According to Lemma 1, $\Pr[\neg \text{EarlyQ}] \geq 1 - \text{negl}(\lambda)$. Therefore,

$$\Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, v) = 1] = (1 - \text{negl}(\lambda)) \Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, u) = 1] + \text{negl}(\lambda).$$

It follows that $|\Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, v) = 1] - \Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, u) = 1]| \leq \text{negl}(\lambda)$. $\square$

7 Asynchronous Strong Common Coin with Silent Setup

Following the discussion in Sect. 2, we “compile” a quadratic-communication weak common coin protocol into a quadratic-communication strong common coin protocol via two steps. First, as in Sect. 7.1, we present a leader election protocol based on our weak common coin protocol, which is sufficient for instantiating the existing quadratic-communication MVBA protocols that assume a leader election subroutine. Next, as in Sect. 7.2, a strong common coin can be easily built on the silent-setup quadratic-communication MVBA.

7.1 Leader Election with Silent Setup

Leader Election Definition. We present a definition for leader election as follows, which is adapted from [49]. In particular, compared with a weak common coin protocol with the output space of $[n]$, a leader election can ensure the *agreement* with $1 - \text{negl}(\lambda)$ probability.

Definition 3. Let Π be a protocol involving n participants in which each P_i is instructed to output an index ι in $[n]$. We say Π is an (n, f, ϕ) leader election for $\phi \in (0, 1]$, if, in any instance of Π , and for any PPT adversary $\mathcal{A}$ which corrupts up to f parties, the following properties hold.

- **Termination:** Except with a negligible probability, every honest node P_i eventually outputs a value $\iota \in [n]$.
- **Agreement:** If two honest parties outputs ι and ι' , respectively, then $\iota = \iota'$.
- **ϕ -Unpredictability:** Let $\text{State}_{\mathcal{A}}^{\text{before}}$ be the internal state of $\mathcal{A}$ before $(n - f - |\mathcal{C}|)$ honest parties are activated in this instance. Let ι be the output of an honest node. Then, for any PPT algorithm $\mathcal{D}$, it follows that

$$\Pr[\text{BadSet} \leftarrow \mathcal{D}(\text{State}_{\mathcal{A}}^{\text{before}}) : \iota \notin \text{BadSet} \subset [n]] \geq \phi \left(\frac{n - |\text{BadSet}|}{n} \right) - \text{negl}(\lambda).$$

It can capture the essence that there is always a constant probability of electing a “good” leader.

Multivalued Byzantine Agreement with Silent Setup. As an ingredient to our leader election protocol, we study a full-fledged instantiation of multivalued Byzantine Agreement (MBA) (the definition recalled in Sect. B.2).

MBA has been studied extensively in the (weak)-coin-aided model. In particular, Mostefaoui and Raynal [77] presented a communication-optimal MBA protocol in the (weak)-coin-aided model, where ℓ is the bit length of the input to MBA. We summarize their results in the following lemma.

Lemma 8 (Coin-aided MBA [77]). Assume there is a binary weak coin protocol with termination and ϕ -fairness for any constant $\phi \in (0, \frac{1}{2}]$, as per [41] (cf. Appendix B.1). Then, there is a secure MBA protocol using it as subroutines, which, except the weak coin subroutines, is deterministic and information-theoretically secure and has the communication complexity of $O(\ell n^2)$ and the round complexity of $O(1)$ in expectation, where ℓ is the bit length of an input.

In Theorem 2, we demonstrate a weak coin protocol for a general output space V with termination and $\frac{n-f}{n}$ -fairness, as per Def. 1. As we have shown in Lemma 13, our results imply a binary weak coin with termination and $\frac{n-f}{2n}$ -fairness as per Feldman and Micali [41]. Therefore, a straightforward approach to a communication-optimal MBA with a silent setup is to use multiple instances of our binary weak coin protocol under the same setup to realize their weak coin subroutine. We denote such a protocol by MBA, and its setup is **WeakCoin.Setup**. Since our weak coin protocol is not guaranteed to be universally composable, we demonstrate the security of MBA in Lemma 18 in Appendix E.

The Construction. Now, we are ready to present our leader election protocol with a silent setup. Its setup phase is exactly **WeakCoin.Setup**. After the setup, a weak coin instance with the output space of $[n]$ is invoked; After receiving ρ_i from the weak coin instance, an honest node P_i passes ρ_i to an MBA instance; If the MBA instance returns $\rho \neq \perp$ to P_i , then P_i returns ρ as the output of the leader election. Otherwise, P_i simply outputs 1. Here, the MBA instances are from Lemma 8, which use binary weak coin instances under the setup of **WeakCoin.Setup**. We describe the pseudocode in Algorithm 3, while the execution flow is outlined in a part of Fig. 5.

Theorem 3 (Leader Election). Under **WeakCoin.Setup**, the leader election protocol **Leader** in Algorithm 3 satisfies the termination and φ -unpredictability for $\varphi = \frac{n-f}{n}$. Its communication complexity and round complexity are $O(\lambda n^2)$ and $O(1)$ in expectation, respectively.

Algorithm 3 The leader election protocol **Leader** with **sid** for P_i

```
1: Invoke WeakCoin[(sid, wc,  $k$ )]
2: wait until receiving  $\rho_i$  from WeakCoin[(sid, wc,  $k$ )]
3:   Invoke MBA[(sid, ba,  $k$ )] with input  $\rho_i$ .
4: wait until receiving  $\rho'$  from MBA[(sid, ba,  $k$ )]
5:   if  $\rho' \neq \perp$ , return  $\rho'$ 
6:   else return 1
```

Proof. We argued why the components, **WeakCoin** and **MBA**, remain secure in the composition in Proposition 7 in Appendix E. Based on this fact, we prove the properties as follows.

Termination: It is naturally ensured by the termination of **WeakCoin** and **MBA**.

Agreement: It is naturally ensured by the agreement of **MBA**.

φ -unpredictability: Note that there is a fair event, denoted by Fair_k w.r.t. **WeakCoin**[(**sid**, **wc**, k)], such that $\Pr[\text{Fair}_k] \geq \varphi - \text{negl}(\lambda)$, and under which all honest nodes output the same ρ , and any PPT algorithm, given the state $\text{State}_{\mathcal{A}}^{\text{before}}$ of $\mathcal{A}$ before $n - f - |\mathcal{C}|$ honest nodes are activated in **Election**[**sid**], cannot distinguish ρ and uniformly sampled ρ' from $[n]$. Therefore,

$$\begin{aligned} & \Pr[\mathcal{D}(\text{State}_{\mathcal{A}}^{\text{before}}) \rightarrow \text{BadSet} : \rho \notin \text{BadSet}] \\ & \geq \Pr[\mathcal{D}(\text{State}_{\mathcal{A}}^{\text{before}}) \rightarrow \text{BadSet} : \rho \notin \text{BadSet} | \text{Fair}_k] \Pr[\text{Fair}_k] \\ & \geq \left(\frac{n - |\text{BadSet}|}{n} - \text{negl}(\lambda) \right) (\varphi - \text{negl}(\lambda)) = \frac{n - |\text{BadSet}|}{n} \cdot \varphi - \text{negl}(\lambda). \end{aligned}$$

MVBA with Silent Setup. As an ingredient to our strong common coin protocol, we study a full-fledged instantiation of Multivalued *Validated* Byzantine Agreement (MVBA) with silent setup (the definition recalled in Sect. B.2).

There are MVBA protocols [5, 59] that use provable broadcast (PB) and leader election as subroutines, achieving $\mathcal{O}(\lambda n^2)$ communication complexity (here the input size is $\mathcal{O}(\lambda)$) and the round complexity of $\mathcal{O}(1)$. Naturally, we can use PB in Algorithm 5 and Election in Algorithm 3 to instantiate these components, which yields a communication-optimal MVBA protocol with a silent setup. In particular, the setup is **WeakCoin.Setup**, which subsumes the setups needed for both PB and Election. In Lemma 19, we proved the composition guarantees of this MVBA scheme.

7.2 Achieving Asynchronous Strong Common Coin

Now, we are ready to present our asynchronous strong common coin. At a high level (see also Fig. 5), we use a silent-setup MVBA to help all honest nodes agree on an aggregated ciphertext c^* with a valid proof π^* so that they can decrypt this ciphertext and use (the random oracle output of) the decryption as the common coin value.

The Construction. Our strong common coin protocol requires the following building blocks: An **STHE** scheme, a digital signature scheme **DS**, a **NIZK** system **NIZK** for the relation $\mathcal{R}_{\text{enc}}$ (cf. Eq. 2), and **SNARK** for the relation $\mathcal{R}_{\text{agg}}$ (cf. Eq. 3). Basically, they are the same schemes that we need for our weak coin protocol **WeakCoin**, and so they can function under the same setup **WeakCoin.Setup**.

In addition, we need the MVBA protocol outlined in Lemma 19, whose setup is also **WeakCoin.Setup**. The external predicate **Predicate** of MVBA is parameterized by $(\text{crs}_{\text{agg}}, n, f, (vk_i)_{i \in [n]}, ek, \text{sid}, \text{crs}_{\text{enc}})$

Algorithm 4 The asynchronous strong common coin Coin protocol with identifier sid for P_i

```

1: upon activated with  $\text{sid}$  do
2:   Initialize:  $\text{Fresh}, \text{ParK} \leftarrow \emptyset$ 
3:    $r_i \leftarrow \text{Msg}_{\text{te}}, \gamma_i \leftarrow \text{Rand}_{\text{te}}, c_i \leftarrow \text{STHE.Enc}(ek, \text{sid}, r_i; \gamma_i)$ 
4:    $\sigma_i \leftarrow \text{DS.Sign}(sk_i, c_i)$   $\pi_i \leftarrow \text{NIZK.P}(\text{crs}_{\text{enc}}, (ek, \text{sid}, c_i), (r_i, \gamma_i))$ 
5:   Multicast  $(\text{sid}, \text{FRESH}, c_i, \sigma_i, \pi_i)$ 

6: upon receiving  $(\text{sid}, \text{FRESH}, c_j, \sigma_j, \pi_j)$  from node  $P_j$  for the first time do
7:   if  $\text{DS.Vrfy}(vk_j, c_j, \sigma_j) = 1 \wedge \text{NIZK.Vrfy}(\text{crs}_{\text{enc}}, (ek, \text{sid}, c_j), \pi_j) = 1$  then
8:      $\text{Fresh} \leftarrow \text{Fresh} \cup \{(j, c_j, \sigma_j, \pi_j)\}$ 
9:     if  $|\text{Fresh}| = f + 1$  and has not invoked  $\text{MVBA}[(\text{sid}, \text{mvba})]$  then
10:       $c_i^* \leftarrow \text{STHE.Eval}(ek, \text{sid}, \{c_j\}_{(j, \dots) \in \text{Fresh}})$  ▷ Aggregate then prove
11:       $\pi_i^* \leftarrow \text{SNARK.P}(\text{crs}_{\text{agg}}, (n, f + 1, (vk_z)_{z \in [n]}, ek, \text{sid}, \text{crs}_{\text{enc}}, c_i^*), \text{Fresh})$ 
12:      Invoke  $\text{MVBA}[(\text{sid}, \text{mvba})]$  with input  $(c_i^*, \pi_i^*)$ 

13: upon  $\text{MVBA}[(\text{sid}, \text{mvba})]$  returns  $(c^*, \pi^*)$  do
14:    $\zeta_i \leftarrow \text{STHE.PartDec}(ek, \text{sid}, dk_i)$ 
15:   Multicast  $(\text{sid}, \text{PartDec}, \zeta_i)$ 

16: upon receiving  $(\text{sid}, \text{PARTDEC}, \zeta_j)$  from  $P_j$  for the first time do
17:   if  $\text{STHE.PartVrfy}(ek, \text{sid}, pk_j, \zeta_j) = 1$  then
18:      $\text{ParK} \leftarrow \text{ParK} \cup \{(j, \zeta_j)\}$ 
19:   if  $|\text{ParK}| = n - f$  then
20:      $dk_{\text{sid}} \leftarrow \text{STHE.DkAgg}(ek, \text{sid}, \{\zeta_k\}_{(k, j) \in \text{ParK}})$  ▷ decryption key
21:      $r^* \leftarrow \text{STHE.Dec}(ek, \text{sid}, dk_{\text{sid}}, c^*),$  and  $v^* \leftarrow H(\text{sid}, r^*)$ .
22:   return  $v^*$ 

```

where sid is the identifier of the coin instance, and $\text{Predicate}(c^*, \pi^*) = 1$, iff

$$\text{SNARK.Vrfy}(\text{crs}_{\text{agg}}, (n, f, (vk_i)_{i \in [n]}, ek, \text{sid}, \text{crs}_{\text{enc}}, c^*), \pi^*) = 1, \quad (5)$$

which is actually the same predicate used in the **Align** inside **WeakCoin** except with different identifiers.

With these building blocks, we present our strong common coin protocol in Algorithm 4. The protocol assumes a setup of **WeakCoin.Setup**, and works as follows. First, a few steps are similar to that in our weak common protocol. After each node P_i has generated (c_i^*, π_i^*) , all nodes jointly run an **MVBA** instance to agree on certain (c^*, π^*) which satisfies **Predicate**. After that, all honest nodes can jointly decrypt c^* , and decide on $H(\text{sid}, \text{STHE.Dec}(ek, dk_{\text{sid}}, c^*))$.

Efficiency Analysis. It is easy to verify that **Coin** has $O(n^2\lambda)$ communication complexity and expected $O(1)$ round complexity. Concretely, the expected round complexity of Mostéfaoui and Raynal's **MBA** [77] is $6 + \text{Round}_{\text{ABA}}$, where $\text{Round}_{\text{ABA}}$ denotes the rounds of the underlying binary BA. The state-of-the-art binary BA from [29], based on weak coins, incurs $(7 + \text{Round}_{\text{WC}}) + (5 + \text{Round}_{\text{WC}}) \cdot (2/\phi)$ rounds in expectation. With our weak coin instantiation ($\text{Round}_{\text{WC}} = 12$, $\phi = \frac{2f+1}{3f+1} \approx 2/3$), we obtain $\text{Round}_{\text{ABA}} = 70$ and $\text{Round}_{\text{MBA}} = 76$. Our leader election thus takes $\text{Round}_{\text{LE}} = \text{Round}_{\text{WC}} + \text{Round}_{\text{MBA}} = 88$ rounds. The **MVBA** protocol from [59] requires $(7 + \text{Round}_{\text{LE}}) \cdot (3/(2\phi)) = 213.75$ rounds in expectation. Our strong common coin adds two extra rounds, bringing the total to 215.75.

In the best case (no faults and synchrony), where all honest nodes in the underlying **MBA** start with the same input and **WeakCoin** outputs the same value, our **Coin** protocol completes in just 31 rounds.

Security Analysis. We establish the security of Coin in the following theorem.

Theorem 4. *The protocol Coin in Algorithm 4 satisfies the termination, 1-fairness as per Def.1.*

Lemma 9 (Termination of coin). *The protocol Coin in Algorithm 4 satisfies the termination.*

Proof. We prove the termination by showing every honest node can execute the code of Line 19-22 in Algorithm 2. For Lines 1-5, every honest node can perform the task without issues. For 6-12, due to the completeness of the underlying NIZK and the correctness of the underlying DS, every honestly generated proof π_j and signature σ_i can be verified, so that $(j, c_j, \sigma_j, \pi_j)$ carried by the FRESH message will be included in the set Fresh. Since all honest nodes have multicasted the FRESH message, the condition in Line 9 can be satisfied, and every honest node can execute Line 9-12.

Due to the completeness of SNARK, π_i^* is a valid proof w.r.t. c_i^* and the public parameters, so (c_i^*, π_i^*) satisfies the predicate of MVBA[$\langle \text{sid}, \text{mvba} \rangle$]. Due to the termination of MVBA, every honest node can output (c^*, π^*) , so the code of Line 13-15 can be executed by all honest nodes. Due to the partial verification correctness of STHE, it follows that all honest nodes can receive enough valid partial decryptions, so the code of Lines 16-22 can be executed by all honest nodes. $\square$

Lemma 10 (Agreement of coin). *The protocol Coin in Algorithm 4 satisfies the agreement with the probability $1 - \text{negl}(\lambda)$.*

Proof. Ensured by the agreement of MVBA, all honest nodes receive the same (c^*, π^*) . Due to the external validity of MVBA, (c^*, π^*) satisfies the predicate Predicate of MVBA defined in Eq.5, i.e., π^* is a valid proof showing the well-formedness of c^* w.r.t. SNARK. The knowledge soundness of SNARK implies that c^* is an aggregation of $f + 1$ ciphertexts $\{c_j\}_{j \in S}$ for some $S \subset [n]$ and $|S| = f + 1$, and every c_j has a well-formedness proof π_j w.r.t. NIZK for $j \in S$. Due to the simulation extractability of NIZK, there are $\gamma_j \in \text{Rand}_{\text{te}}$ and $r_j \in \text{Msg}_{\text{te}}$, such that $c_j = \text{STHE.Enc}(ek, \text{sid}, r_j; \gamma_j)$. Then, the evaluation correctness of STHE ensures there exists r^* and γ^* such that $c^* = \text{STHE.Enc}(ek, \text{sid}, r^*; \gamma^*)$. Then, following the decryption correctness of STHE, all honest nodes shall decrypt c^* to r^* , and then they output the same $H(\text{sid}, r^*)$. $\square$

Lemma 11 (Unpredictability of coin). *The protocol Coin in Algorithm 4 satisfies the unpredictability and bias-resistance with the probability $1 - \text{negl}(\lambda)$.*

Proof. Let $\text{State}'_{\mathcal{A}}$ be the state of $\mathcal{A}$ at the first time that an honest P_i outputs in MVBA[$\langle \text{sid}, \text{mvba} \rangle$], where there must have been $n - f - |\mathcal{C}|$ honest nodes participating the instance of Coin[sid]. Without loss of generality, we assume $\text{State}_{\mathcal{A}}^{\text{before}}$ has been encoded in $\text{State}'_{\mathcal{A}}$. Then, it suffices to show that

$$|\Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, H(\text{sid}, r^*)) = 1] - \Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, v) = 1]| \leq \text{negl}(\lambda),$$

for any PPT algorithm $\mathcal{D}$, and $v \leftarrow \$V$.

To show this, we prove that the random oracle H has not been queried with (sid, r^*) (defined in the proof of Lemma 10) at the first time that an honest node outputs in MVBA[$\langle \text{sid}, \text{mvba} \rangle$] in Lemma 12, except with a negligible probability. With this fact, $\text{State}'_{\mathcal{A}}$ is independent of $H(\text{sid}, r^*)$. Then, it follows trivially to see $|\Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, H(\text{sid}, r^*)) = 1] - \Pr[\mathcal{D}(\text{State}'_{\mathcal{A}}, v) = 1]| \leq \text{negl}(\lambda)$. $\square$

Lemma 12. *At the first time that an honest node P_i outputs (c^*, π^*) in the MVBA[$\langle \text{sid}, \text{mvba} \rangle$] instance, the probability that H has been queried with (sid, r^*) is negligibly small, where r^* is the decryption of c^* .*

Proof. This proof is similar to the proof for Lemma 1. In particular, we prove this lemma by contradiction. Assume that there exists a PPT $\mathcal{A}$, such that in the execution of $\text{Coin}[\text{sid}]$, there is a non-negligible probability that H has been queried with the specified (sid, r^*) at the first time that an honest node P_i outputs (c^*, π^*) MVBA $[\langle \text{sid}, \text{mvba} \rangle]$ instance. Then, we can build a PPT adversary $\mathcal{B}$ which can win the raUNP security game in Fig.7, i.e., $\text{Exp}_{\text{raUNP}}^{\mathcal{B}}(1^\lambda, n, f) = 1$, with a non-negligible probability.

We outline the strategy of $\mathcal{B}$ as follows, which runs a copy of $\mathcal{A}$ and tries to simulate the execution of the coin until an honest node outputs in the MVBA subroutine.

- *Setup Phase:* Note that the setup of Coin is exactly the setup of our WeakCoin . Therefore, $\mathcal{B}$ can finish the setup in the following way: $\mathcal{B}$ forwards the messages it received in the raUNP security game to $\mathcal{A}$, and forwards the corruption requests and public keys of corrupted parties from $\mathcal{A}$ to the security game. In addition, the setup for Align is run by $\mathcal{B}$. Initialize two sets $\mathcal{H} \subset [n]$ and $\mathcal{C} \subset [n]$ to track the identities of honest parties and corrupted parties, respectively.
- *Protocol Simulation:* Note that in the raUNP security game, $\mathcal{B}$ receives $(c_i, \pi_i, \sigma_i)_{i \in \mathcal{C}_{\text{init}}}$. Then, $\mathcal{B}$ acts on the behalf of every so-far-honest node P_i to multicast $(\text{sid}, \text{FRESH}, c_i, \sigma_i, \pi_i)$. Next, $\mathcal{B}$ honestly follows the protocol specification to act on behalf of all honest nodes. Random oracle queries are responded to in a standard way, and $\mathcal{B}$ maintains two tables Hist of the query records.
- *Handle Oracle Queries:* Note that $\mathcal{A}$ can query the additional oracles in $\mathbb{O}_{\text{weakcoin}} = \{\mathcal{O}_{\text{PartD}}, \mathcal{O}_{\text{Sign}}\} \cup \mathbb{O}_{\text{Align}}$. When $\mathcal{A}$ queries $\mathcal{O}_{\text{PartD}}$ (resp. $\mathcal{O}_{\text{Sign}}$) with (sid', i) (resp. $(\text{sid}', i, \text{msg})$) such that $i \in \mathcal{H}$ and $\text{sid}' \neq \text{sid}$, $\mathcal{B}$ queries $\mathcal{O}_{\text{PartD}}$ (resp. $\mathcal{O}_{\text{Sign}}$) in the raUNP security game with (sid', i) (resp. $(i, (\text{sid}', \text{msg}))$) and forwards the message from $\mathcal{O}_{\text{PartD}}$ to $\mathcal{A}$. Regarding queries to oracles in $\mathbb{O}_{\text{Align}}$, $\mathcal{B}$ answers them honestly by using the secret states of honest parties w.r.t. $\text{Align}[\langle \text{sid}, \text{ali} \rangle]$.
- *Handle Corruption:* When $\mathcal{A}$ corrupts P_i , $\mathcal{B}$ checks if the number of corrupted parties has reached the limit. If not, $\mathcal{B}$ queries $\mathcal{O}_{\text{Corr}}$ with i , which returns $(dk_i, sk_i, r_i, \gamma_i, \hat{\gamma}_i, \bar{\gamma}_i, \text{State}_i)$. Then, it returns the tuple along with other secrets and randomness for SNARK and Align to $\mathcal{A}$.
- *Output:* At the first time that an honest node outputs (c^*, π^*) in MVBA, $\mathcal{B}$ obtains (c^*, π^*) . As ensured by the external validity,

$$\text{SNARK.Vrfy}(\text{crs}_{\text{agg}}, (n, f, (vk_i)_{i \in [n]}, ek, \text{tag}^*, \text{crs}_{\text{enc}}, c^*), \pi^*) = 1,$$

as (c^*, π^*) satisfy the predicate. Then, $\mathcal{B}$ extracts all records in Hist with the input form (sid, r') , and uniformly picks one record (sid, r^*) from them. It outputs (c^*, π^*, r^*) in the raUNP security game.

It is easy to verify, in $\mathcal{A}$'s view, that the execution simulated by $\mathcal{B}$ is identical to a real execution of $\text{Coin}[\text{sid}]$. Therefore, there is a non-negligible probability that a tuple (sid, r^*) , where r^* is the decryption of c^* , has been issued to H . Moreover, since the number of query records must be $\text{poly}(\lambda)$, the probability that $\mathcal{B}$ happens to pick this tuple is at least $\frac{1}{\text{poly}(\lambda)}$. There would be a non-negligible probability that $\mathcal{B}$ wins in the raUNP security game, which contradicts Lemma 5. $\square$

Acknowledgements

This work is supported in part by ARC Discovery Project (DP250101739), and research awards from the Ethereum Foundation, Protocol Labs, and the Stellar Development Foundation.

References

1. Abraham, I., Bacho, R., Loss, J., Stern, G.: Nearly quadratic asynchronous distributed key generation. Cryptology ePrint Archive, Paper 2025/006 (2025), <https://eprint.iacr.org/2025/006>
2. Abraham, I., Chan, T.H., Dolev, D., Nayak, K., Pass, R., Ren, L., Shi, E.: Communication complexity of byzantine agreement, revisited. Distributed Comput. **36**(1), 3–28 (2023)
3. Abraham, I., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G.: Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation. In: CRYPTO (1). Lecture Notes in Computer Science, vol. 14081, pp. 39–70. Springer (2023)
4. Abraham, I., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G., Tomescu, A.: Reaching consensus for asynchronous distributed key generation. In: PODC. pp. 363–373. ACM (2021)
5. Abraham, I., Malkhi, D., Spiegelman, A.: Asymptotically optimal validated asynchronous byzantine agreement. In: PODC. pp. 337–346. ACM (2019)
6. Attema, T., Cramer, R., Rambaud, M.: Compressed Σ -protocols for bilinear group arithmetic circuits and application to logarithmic transparent threshold signatures. In: ASIACRYPT (4). Lecture Notes in Computer Science, vol. 13093, pp. 526–556. Springer (2021)
7. Avitabile, G., Döttling, N., Magri, B., Sakas, C., Wognig, S.: Signature-based witness encryption with compact ciphertext. IACR Cryptol. ePrint Arch. p. 1477 (2024)
8. Bacho, R., Collins, D., Liu-Zhang, C., Loss, J.: Network-agnostic security comes (almost) for free in DKG and MPC. In: CRYPTO (1). Lecture Notes in Computer Science, vol. 14081, pp. 71–106. Springer (2023)
9. Bacho, R., Lenzen, C., Loss, J., Ochsenreither, S., Papachristoudis, D.: Grandline: Adaptively secure DKG and randomness beacon with (almost) quadratic communication complexity. In: CCS. ACM (2024)
10. Bacho, R., Loss, J.: On the adaptive security of the threshold BLS signature scheme. In: CCS. pp. 193–207. ACM (2022)
11. Bacho, R., Loss, J.: Adaptively secure (aggregatable) pvss and application to distributed randomness beacons. In: CCS. ACM (2023)
12. Bandarupalli, A., Bhat, A., Bagchi, S., Kate, A., Reiter, M.K.: Random beacons in monte carlo: Efficient asynchronous random beacon *without* threshold cryptography. In: CCS. pp. 2621–2635. ACM (2024)
13. Bangalore, L., Choudhury, A., Patra, A.: Almost-surely terminating asynchronous byzantine agreement revisited. In: PODC. pp. 295–304. ACM (2018)
14. Bauer, B., Fuchsbauer, G., Plouviez, A.: The one-more discrete logarithm assumption in the generic group model. In: ASIACRYPT (4). Lecture Notes in Computer Science, vol. 13093, pp. 587–617. Springer (2021)
15. Ben-Or, M., Canetti, R., Goldreich, O.: Asynchronous secure computation. In: STOC. pp. 52–61. ACM (1993)
16. Blum, E., Katz, J., Liu-Zhang, C., Loss, J.: Asynchronous byzantine agreement with subquadratic communication. In: TCC (1). Lecture Notes in Computer Science, vol. 12550, pp. 353–380. Springer (2020)
17. Blum, M.: Coin flipping by telephone. In: CRYPTO. pp. 11–15. U. C. Santa Barbara, Dept. of Elec. and Computer Eng., ECE Report No 82-04 (1981)
18. Boneh, D., Boyen, X.: Short signatures without random oracles. In: EUROCRYPT. Lecture Notes in Computer Science, vol. 3027, pp. 56–73. Springer (2004)
19. Boneh, D., Boyen, X.: Efficient selective identity-based encryption without random oracles. J. Cryptol. **24**(4), 659–693 (2011)
20. Bracha, G.: Asynchronous byzantine agreement protocols. Information and Computation **75**(2), 130–143 (1987)
21. Cachin, C., Kursawe, K., Petzold, F., Shoup, V.: Secure asynchronous byzantine consensus with optimal resilience. In: Proceedings of the 21st Annual ACM Symposium on Principles of Distributed Computing (PODC). pp. 33–42. ACM (2001)
22. Cachin, C., Kursawe, K., Shoup, V.: Random oracles in constantipole: practical asynchronous byzantine agreement using cryptography (extended abstract). In: PODC. pp. 123–132. ACM (2000)
23. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: FOCS. pp. 136–145. IEEE Computer Society (2001)
24. Canetti, R., Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Adaptive security for threshold cryptosystems. In: CRYPTO. Lecture Notes in Computer Science, vol. 1666, pp. 98–115. Springer (1999)
25. Canetti, R., Rabin, T.: Fast asynchronous byzantine agreement with optimal resilience. In: STOC. pp. 42–51. ACM (1993)
26. Choi, K., Manoj, A., Bonneau, J.: Sok: Distributed randomness beacons. In: SP. pp. 75–92. IEEE (2023)
27. Choudhury, A., Patra, A.: On the communication efficiency of statistically secure asynchronous MPC with optimal resilience. J. Cryptol. **36**(2), 13 (2023)

28. Civit, P., Dzulfikar, M.A., Gilbert, S., Guerraoui, R., Komatovic, J., Vidigueira, M.: DARE to agree: Byzantine agreement with optimal resilience and adaptive communication. In: PODC. pp. 145–156. ACM (2024)
29. Crain, T.: Two more algorithms for randomized signature-free asynchronous binary byzantine consensus with $t < n/3$ and $o(n^2)$ messages and $o(1)$ round expected termination. arXiv preprint arXiv:2002.08765 (2020)
30. Das, S., Camacho, P., Xiang, Z., Nieto, J., Bünz, B., Ren, L.: Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 356–370 (2023)
31. Das, S., Duan, S., Liu, S., Momose, A., Ren, L., Shoup, V.: Asynchronous consensus without trusted setup or public-key cryptography. In: CCS. pp. 3242–3256. ACM (2024)
32. Das, S., Ren, L.: Adaptively secure BLS threshold signatures from DDH and co-cdh. In: CRYPTO (7). Lecture Notes in Computer Science, vol. 14926, pp. 251–284. Springer (2024)
33. Das, S., Xiang, Z., Kokoris-Kogias, L., Ren, L.: Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. In: USENIX Security Symposium. pp. 5359–5376. USENIX Association (2023)
34. Das, S., Xiang, Z., Tomescu, A., Spiegelman, A., Pinkas, B., Ren, L.: Verifiable secret sharing simplified. Cryptology ePrint Archive, Paper 2023/1196 (2023), <https://eprint.iacr.org/2023/1196>
35. Das, S., Yurek, T., Xiang, Z., Miller, A., Kokoris-Kogias, L., Ren, L.: Practical asynchronous distributed key generation. In: SP. pp. 2518–2534. IEEE (2022)
36. David, B., Gazi, P., Kiayias, A., Russell, A.: Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In: EUROCRYPT (2). Lecture Notes in Computer Science, vol. 10821, pp. 66–98. Springer (2018)
37. David, B., Magri, B., Matt, C., Nielsen, J.B., Tschudi, D.: GearBox: Optimal-size shard committees by leveraging the safety-liveness dichotomy. Cryptology ePrint Archive, Paper 2021/211 (2021), <https://eprint.iacr.org/2021/211>
38. Dolev, D., Strong, H.R.: Authenticated algorithms for byzantine agreement. SIAM J. Comput. **12**(4), 656–666 (1983)
39. Feldman, P., Micali, S.: Optimal algorithms for byzantine agreement. In: STOC. pp. 148–161. ACM (1988)
40. Feldman, P.N.: Optimal algorithms for Byzantine agreement. Ph.d. thesis, Massachusetts Institute of Technology, Cambridge, MA (1988)
41. Feldman, P., Micali, S.: An optimal probabilistic protocol for synchronous byzantine agreement. SIAM J. Comput. **26**(4), 873–933 (1997)
42. Feng, H., Gao, Y., Lu, Y., Tang, Q., Xu, J.: Practical asynchronous distributed key reconfiguration and its applications. Cryptology ePrint Archive, Paper 2025/149 (2025), <https://eprint.iacr.org/2025/149>
43. Feng, H., Lu, Z., Tang, Q.: Dragon: Decentralization at the cost of representation after arbitrary grouping and its applications to sub-cubic DKG and interactive consistency. In: PODC. pp. 469–479. ACM (2024)
44. Feng, H., Mai, T., Tang, Q.: Scalable and adaptively secure any-trust distributed key generation and all-hands checkpointing. In: ACM CCS (2024)
45. Fischer, M.J., Lynch, N.A., Paterson, M.: Impossibility of distributed consensus with one faulty process. J. ACM **32**(2), 374–382 (1985)
46. Fuchsbaauer, G., Kiltz, E., Loss, J.: The algebraic group model and its applications. In: Advances in Cryptology – CRYPTO 2018. Lecture Notes in Computer Science, vol. 10992, pp. 33–62. Springer (2018)
47. Fuchsbaauer, G., Kiltz, E., Loss, J.: The algebraic group model and its applications. In: CRYPTO (2). Lecture Notes in Computer Science, vol. 10992, pp. 33–62. Springer (2018)
48. Ganesh, C., Kondi, Y., Orlandi, C., Pancholi, M., Takahashi, A., Tschudi, D.: Witness-succinct universally-composable snarks. In: EUROCRYPT (2). Lecture Notes in Computer Science, vol. 14005, pp. 315–346. Springer (2023)
49. Gao, Y., Lu, Y., Lu, Z., Tang, Q., Xu, J., Zhang, Z.: Efficient asynchronous byzantine agreement without private setups. In: ICDCS. pp. 246–257. IEEE (2022)
50. Garg, S., Hajiabadi, M., Mahmoody, M., Rahimi, A.: Registration-based encryption: Removing private-key generator from IBE. In: TCC (1). Lecture Notes in Computer Science, vol. 11239, pp. 689–718. Springer (2018)
51. Garg, S., Jain, A., Mukherjee, P., Sinha, R., Wang, M., Zhang, Y.: hints: Threshold signatures with silent setup. In: 2024 IEEE Symposium on Security and Privacy (SP). pp. 3034–3052. IEEE (2024)
52. Garg, S., Kolonelos, D., Policharla, G., Wang, M.: Threshold encryption with silent setup. In: CRYPTO (7). Lecture Notes in Computer Science, vol. 14926, pp. 352–386. Springer (2024)
53. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure distributed key generation for discrete-log based cryptosystems. J. Cryptol. **20**(1), 51–83 (2007)

54. Gilad, Y., Hemo, R., Micali, S., Vlachos, G., Zeldovich, N.: Algorand: Scaling byzantine agreements for cryptocurrencies. In: SOSP. pp. 51–68. ACM (2017)
55. Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game or A completeness theorem for protocols with honest majority. In: STOC. pp. 218–229. ACM (1987)
56. Groth, J.: On the size of pairing-based non-interactive arguments. In: EUROCRYPT (2). Lecture Notes in Computer Science, vol. 9666, pp. 305–326. Springer (2016)
57. Groth, J.: Non-interactive distributed key generation and key resharing. IACR Cryptol. ePrint Arch. p. 339 (2021)
58. Groth, J., Sahai, A.: Efficient non-interactive proof systems for bilinear groups. In: EUROCRYPT. Lecture Notes in Computer Science, vol. 4965, pp. 415–432. Springer (2008)
59. Guo, B., Lu, Y., Lu, Z., Tang, Q., Xu, J., Zhang, Z.: Speeding dumbo: Pushing asynchronous BFT closer to practice. In: NDSS. The Internet Society (2022)
60. Guo, B., Lu, Z., Tang, Q., Xu, J., Zhang, Z.: Dumbo: Faster asynchronous BFT protocols. In: CCS. pp. 803–818. ACM (2020)
61. Gurkan, K., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G., Tomescu, A.: Aggregatable distributed key generation. In: EUROCRYPT (1). Lecture Notes in Computer Science, vol. 12696, pp. 147–176. Springer (2021)
62. Huang, S., Pettie, S., Zhu, L.: Byzantine agreement with optimal resilience via statistical fraud detection. J. ACM **71**(2), 12:1–12:37 (2024)
63. Jaeger, J., Mohan, D.I.: Generic and algebraic computation models: When AGM proofs transfer to the GGM. In: CRYPTO (5). Lecture Notes in Computer Science, vol. 14924, pp. 14–45. Springer (2024)
64. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: ASIACRYPT. Lecture Notes in Computer Science, vol. 6477, pp. 177–194. Springer (2010)
65. Katz, J.: Round-optimal, fully secure distributed key generation. In: CRYPTO (7). Lecture Notes in Computer Science, vol. 14926, pp. 285–316. Springer (2024)
66. Katz, J., Koo, C.: On expected constant-round protocols for byzantine agreement. In: CRYPTO. Lecture Notes in Computer Science, vol. 4117, pp. 445–462. Springer (2006)
67. Kokoris-Kogias, E., Jovanovic, P., Gasser, L., Gailly, N., Syta, E., Ford, B.: Omniledger: A secure, scale-out, decentralized ledger via sharding. In: IEEE Symposium on Security and Privacy. pp. 583–598. IEEE Computer Society (2018)
68. Kokoris-Kogias, E., Malkhi, D., Spiegelman, A.: Asynchronous distributed key generation for computationally-secure randomness, consensus, and threshold signatures. In: CCS. pp. 1751–1767. ACM (2020)
69. Kosba, A., Zhao, Z., Miller, A., Qian, Y., Chan, H., Papamanthou, C., Pass, R., abhi shelat, Shi, E.: C0c0: A framework for building composable zero-knowledge proofs. Cryptology ePrint Archive, Paper 2015/1093 (2015), <https://eprint.iacr.org/2015/1093>
70. Libert, B., Joye, M., Yung, M.: Born and raised distributively: Fully distributed non-interactive adaptively-secure threshold signatures with short shares. In: Proceedings of the 2014 ACM symposium on Principles of distributed computing. pp. 303–312 (2014)
71. Lindell, Y., Lysyanskaya, A., Rabin, T.: On the composition of authenticated byzantine agreement. J. ACM **53**(6), 881–917 (2006)
72. Lu, Y., Lu, Z., Tang, Q., Wang, G.: Dumbo-mvba: Optimal multi-valued validated asynchronous byzantine agreement, revisited. In: PODC. pp. 129–138. ACM (2020)
73. Maurer, U.M.: Abstract models of computation in cryptography. In: IMACC. Lecture Notes in Computer Science, vol. 3796, pp. 1–12. Springer (2005)
74. Miller, A., Xia, Y., Croman, K., Shi, E., Song, D.: The honey badger of BFT protocols. In: CCS. pp. 31–42. ACM (2016)
75. Momose, A., Ren, L.: Optimal communication complexity of authenticated byzantine agreement. In: DISC. LIPIcs, vol. 209, pp. 32:1–32:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2021)
76. Mostéfaoui, A., Moumen, H., Raynal, M.: Signature-free asynchronous binary byzantine consensus with $t < n/3$, $o(n^2)$ messages, and $O(1)$ expected time. J. ACM **62**(4), 31:1–31:21 (2015)
77. Mostéfaoui, A., Raynal, M.: Signature-free asynchronous byzantine systems: from multivalued to binary consensus with $t < n/3$, $o(n^2)$ messages, and constant time. Acta Informatica **54**(5), 501–520 (2017)
78. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: CRYPTO. Lecture Notes in Computer Science, vol. 576, pp. 129–140. Springer (1991)
79. Qiu, T., Tang, Q.: Predicate aggregate signatures and applications. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 279–312. Springer (2023)
80. Rabin, M.O.: Randomized byzantine generals. In: FOCS. pp. 403–409. IEEE Computer Society (1983)

81. Shoup, V.: Lower bounds for discrete logarithms and related problems. In: EUROCRYPT. Lecture Notes in Computer Science, vol. 1233, pp. 256–266. Springer (1997)
82. de Souza, L.F., Kuznetsov, P., Tonkikh, A.: Distributed randomness from approximate agreement. In: DISC. LIPIcs, vol. 246, pp. 24:1–24:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2022)

A More Related Work

Preprocessing Coins. A few classical asynchronous coin protocols assume a preprocessing phase where the participants already share correlated secret states. The simplest solution is due to M.Rabin’s pioneering work [80], in which many random values have been secretly shared among all participants, such that in the online phase, each honest node multicasts a secret share and then can reconstruct a random value based the shares it received. The online phase incurs $O(\ell n^2)$ -bit communication for each coin of ℓ bits and only requires exactly 1 round. An obvious drawback is that the network has to share $O(N)$ random values in the preprocessing phase to supply $O(N)$ common coins in the online phase. This issue was resolved by Cachin, Kursawe, and Shoup [22] in the computational setting by leveraging cryptographic tools, including a random oracle and a non-interactive unique threshold signature scheme [32, 70]. In their scheme, the group only needs to share one random value in the preprocessing phase, which is a secret key of the threshold signature scheme. Later on, in the online phase, a pseudorandom value is recovered as a common coin at the cost of $O(\lambda n^2)$ communication and exactly one round. In addition, Cachin et al. [22] can tolerate up to $\lceil \frac{n}{3} \rceil - 1$ Byzantine nodes, and if the underlying unique threshold signature scheme is adaptively secure (for instance, using [10]), it is strongly adaptive secure [2]¹⁰. Due to its superior performance and security, Cachin et al. [22] has been widely adopted by recent practical asynchronous consensus protocols [5, 59, 60, 74]. E.Blum et al. [16] presented an asynchronous common coin protocol with subquadratic communication complexity in this model, which, however, is not strongly adaptive secure and only offers sub-optimal resilience.

Essentially, in the preprocessing-model coin protocols, the parties can obtain a common coin in the online phase by somehow “consuming” the coin that is “prepared” in the preprocessing phase. Therefore, if there is no trusted dealer to share a random value with the group in the preprocessing phase (for example, in blockchain-related applications), the group still needs to generate a coin from scratch (at least as a setup for emulating the dealer), which motivates a long line of research [4, 39, 41, 49, 68] dedicating to common coin generation without preprocessing.

Coins without preprocessing. A setup phase is considered to be “silent” if all participants do not communicate with each other, except posting their own public information to a public bulletin board only once during the setup phase. The family of silent setups covers a variety of different setups, including that for establishing the global set of participants’ identities, the standard PKI setup, and the common reference string (CRS) setup.

Information-theoretical Coins. The seminal work by Feldman and Micali [39, 41] initiated the study of the common coin without preprocessing, with a focus on the information theoretical setting, under a setup for establishing global identities and pairwise private and authenticated channels. They presented a *weak common coin* protocol, which, on rough terms, satisfies the *agreement* property of a (strong) common coin with a constant probability. While Feldman and Micali’s result assumes a *synchronous* network, Feldman [40] and Canetti and T. Rabin [25] extended it to

¹⁰ It is secure against adaptive adversaries who can retract yet-delivered messages sent by a newly corrupted node.

asynchrony. In particular, Canetti and T. Rabin’s protocol tolerates up to $\lceil \frac{n}{3} \rceil - 1$ Byzantine nodes, while the communication cost for generating a binary common coin is as large as $\Omega(n^8 \log n)$. Bangalore, Choudhury, and Patra [13] considered an even weaker form of common coin, which has the communication complexity of $O(n^6)$. These weak forms of common coins are sufficient for certain asynchronous consensus protocols such as asynchronous binary agreement [13, 25, 76]. In addition, there is a generic information-theoretical compiler, sketched by Choudhury and Patra in [27], that lifts a weak common coin protocol to a strong common coin protocol at the cost of introducing another multiplicative factor of n in the communication complexity. Recently, Huang, Pettie, and Zhu [62] presented a strong common coin protocol that does not assume private channels and tolerates up to $(\frac{n}{3+\epsilon})$ Byzantine nodes, with communication complexity of $\Omega(n^7)$ and $\text{poly}(n)$ rounds.

Computational Coins. On the other hand, cryptography has been extensively employed to improve the performance of common coin protocols. Katz and Koo [66] extend Feldman and Micali’s protocol to the computational setting with a PKI setup, which yields a computationally secure weak common coin protocol with $O(\lambda n^4)$ communication complexity and $O(1)$ rounds. In recent years, motivated by replacing the trusted setup needed by Cachin et al.’s common coin [22] with an asynchronous distributed protocol, many efforts have been devoted to improving the performance of asynchronous distributed key generation (ADKG), which has a close relation with asynchronous common coin. Namely, the consensus protocols inside ADKG will rely on certain weak coin protocols, while an ADKG protocol directly implies an asynchronous strong common coin protocol with the same complexity.

Kokoris-Kogias et al. [68] gave the first ADKG protocol, which assumes a PKI setup and has the communication complexity of $O(\lambda n^4)$ and the round complexity of $O(n)$. Later, Abraham et al. [4] and Gao et al. [49] presented ADKG protocols with $O(\lambda n^3)$ communication complexity and $O(1)$ rounds. Both works assume a setup for PKI and the CRS and are proven statically secure in the random oracle model, though with an adaptively secure component from [11], they are adaptively secure in principle in the algebraic group model (AGM) [47]. Das et al. [33, 35] focused on the practical performance and presented statically secure ADKG with $O(\lambda n^3)$ communication complexity and $O(\log n)$ round complexity. Abraham et al. [3] gave a strongly adaptively secure DKG under a CRS setup and in the AGM model, with $O(\lambda n^3)$ communication complexity and $O(1)$ round complexity. Compared with [4, 49], [3] generates the standard secret key for DLog-based cryptosystems.

Other than research on ADKG, Freitas, Kuznetsov, and Tonkikh [82] studied the *Monte Carlo* style coin, which ensures the agreement except with a small but non-negligible probability. Their construction can terminate within a pre-determined round which is related to the failure probability. The communication complexity of their protocol is still $O(n^3 \lambda)$. Bandrupalli et al. [12] adapted [82] to the hash-based setting and presented an asynchronous randomness beacon that continuously emitting common coins, which can be proved adaptively secure in the ROM. Though its amortized communication cost for each coin is subcubic, a single-shot execution incurs $O(\lambda n^3)$ communication complexity. Very recently, Das et al. [31] presented a hash-based weak coin protocol in the random oracle model, with the $O(\lambda n^3)$ communication complexity and $O(1)$ round complexity.

A.1 Concurrent Work

The concurrent and independent work [1] also achieves asynchronous distributed key generation (DKG) with subcubic communication complexity, but with fundamentally different approaches.

Technique-wise, we focus on getting an asynchronous common coin, and then compile to ADKG; while they directly construct an ADKG, which also easily implies a common coin.

For common coin: Regarding complexity: theirs incurs $O(\lambda^2 n^2 \log^c n)$ for some constant c , with $O(\log n)$ rounds; Ours achieves a communication complexity of $O(n^2 \lambda)$ with only $O(1)$ rounds.

Regarding setup, the two schemes are not directly comparable. Their approach relies on a trusted setup for verifiable random functions (VRFs), which can be interpreted as a standard PKI along with a one-time common coin after PKI initialization. In contrast, our scheme requires a structured CRS and a PKI. Without optimization, our PKI setup incurs a cubic $O(n^3 \lambda)$ communication cost, but with a key curator, this cost is reduced to quadratic $O(n^2 \lambda)$.

Regarding cryptographic assumptions, our common coin protocol is based on the GGM, while theirs relies on the AGM and the Erasure assumption.

For ADKG: Regarding complexity, theirs is the same as their common coin, $O(\lambda^2 n^2 \log^c n)$ for some constant c , with $O(\log n)$ rounds. In contrast, our ADKG incurs $O(\kappa \lambda n^2)$ -bit communication cost and $O(1)$ rounds, where κ is the size of a randomly sampled committee which contains at least one honest node except with a small failure probability. Looking closely, their communication is actually $O(s \lambda n^2 \log^c n)$, where s is the size of a randomly sampled committee which has an honest majority except with a small failure probability.

More specifically, in our construction, the committee of κ nodes is randomly sampled (from $2f + 1$ nodes where majority of them are honest), ensuring ONE of them is honest. Therefore, the failure probability is $2^{-\kappa}$. In their scheme, to get the same failure probability, the committee size s should be larger than $\kappa / \log_2 \left(\left(\frac{q}{p} \right)^q \left(\frac{1-q}{1-p} \right)^{1-q} \right)$ (according to the analysis in [37]). As $p = 1/3$ and $q = 1/2$ in their case, this implies that κ is much smaller than s , and we also do not have the $\log^c n$ factor in communication.

Regarding setup, our ADKG is the same as our coin.

Regarding cryptographic assumptions, our ADKG also needs the erasure assumption.

Regarding functionality, our ADKG can generate a secret key in a scalar field for conventional DLog systems, while their ADKG generates a secret key in a pairing-friendly ECC group.

B Preliminaries

B.1 Binary Weak Common Coin

In the definition of [41], the termination is the same as ours, and the ρ -fairness is defined as follows:

- ϕ -fairness: let v_i be the output of P_i . Then for every $b \in \{0, 1\}$, it holds that

$$\Pr[\forall i \in \mathcal{H} : v_i = b] \geq \phi - \text{negl}(\lambda).$$

We have the following lemma regarding the relation between our definition and their definition.

Lemma 13. *A protocol with $V = \{0, 1\}$ and satisfying φ -fairness in our definition directly satisfies $\varphi/2$ -fairness in the definition of [41].*

Proof. According to our definition, there is an event **Fair** such that $\Pr[\mathbf{Fair}] \geq \varphi - \text{negl}(\lambda)$, and $\Pr[\exists b, v_i = b, \forall i \in \mathcal{H} | \mathbf{Fair}] = 1$. So $\Pr[\exists b, v_i = b, \forall i \in \mathcal{H}] \geq \varphi - \text{negl}(\lambda)$. Then, if there exists $b_0 \in \{0, 1\}$ s.t. $\Pr[v_i = b_0, \forall i \in \mathcal{H}] < \varphi/2 - \epsilon$, for some non-negligible function ϵ . It follows that $\Pr[v_i = 1 - b_0, \forall i \in \mathcal{H}] > \varphi/2 + \epsilon - \text{negl}(\lambda)$. So, the adversary can trivially distinguish the output b from a uniform bit. $\square$

B.2 Other Cryptographic and Consensus Tools

Non-interactive (Zero-Knowledge) Argument An argument system Φ for an NP relation $\mathcal{R}$ can be described by the following three algorithms.

- **Setup**(1^λ) generates a common reference string **crs**.
- **Prove**(**crs**, x , w) on inputs a statement x and its witness w such that $(x, w) \in R$, produces a proof π .
- **Vrfy**(**crs**, x , π) validates the proof π for x .

We consider the following properties of an argument system Φ .

- *Completeness.* Φ is complete, if for any $(x, w) \in \mathcal{R}$, **crs** $\leftarrow$ **Setup**(1^λ), it holds that

$$\Pr[\text{Vrfy}(\mathbf{crs}, x, \text{Prove}(\mathbf{crs}, x, w)) = 1] = 1$$

- *Knowledge Soundness.* Φ is knowledge-sound, if for any PPT adversary $\mathcal{A}$, there is an PPT algorithm $E_{\mathcal{A}}$, such that

$$\Pr \left[\begin{array}{l} \mathbf{crs} \leftarrow \text{Setup}(1^\lambda), (x, \pi) \leftarrow \mathcal{A}(\mathbf{crs}), w \leftarrow E_{\mathcal{A}}(x, \pi) : \\ \text{Vrfy}(\mathbf{crs}, x, \pi) = 1 \wedge (x, w) \notin R \end{array} \right] \leq \text{negl}(\lambda).$$

- *Zero Knowledge.* Φ is zero-knowledge, if there is a pair of simulator algorithms $\{\text{SimSetup}, \text{SimProve}\}$, and for any PPT adversary $\mathcal{A}$, it holds that

$$\left| \Pr \left[\begin{array}{l} \mathbf{crs} \leftarrow \text{Setup}(1^\lambda) : \\ 1 \leftarrow \mathcal{A}^{\mathcal{O}_0(\cdot)}(\mathbf{crs}) \end{array} \right] - \Pr \left[\begin{array}{l} (\mathbf{crs}, \mathbf{tk}) \leftarrow \text{SimSetup}(1^\lambda) : \\ 1 \leftarrow \mathcal{A}^{\mathcal{O}_1(\cdot)}(\mathbf{crs}) \end{array} \right] \right| \leq \text{negl}(\lambda),$$

where $\mathcal{O}_0$ on input $(x, w) \in \mathcal{R}$ returns $\pi \leftarrow \text{Prove}(\mathbf{crs}, x, w)$, and $\mathcal{O}_1$ returns $\pi \leftarrow \text{SimProve}(\mathbf{crs}, \mathbf{tk}, x)$.

- *Simulation Extractability.* Φ is simulation extractable if, in addition to the zero-knowledge simulation algorithm, $\{\text{SimSetup}, \text{SimProve}\}$, there is a PPT extractor algorithm **SimExt**, such that, for any PPT adversary $\mathcal{A}$, the following equation holds, where $\mathcal{O}$ returns $\text{SimProve}(\mathbf{crs}, \mathbf{tk}, x)$, and $\text{Hist}_{\mathcal{O}}$ records all x queried to $\mathcal{O}$.

$$\Pr \left[\begin{array}{l} (\mathbf{crs}, \mathbf{tk}) \leftarrow \text{SimSetup}(1^\lambda), (x, \pi) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot)}(\mathbf{crs}) \text{ s.t. } x \notin \text{Hist}_{\mathcal{O}_1} \\ w \leftarrow \text{SimExt}(\mathbf{crs}, \mathbf{tk}, x, \pi) : \text{Vrfy}(\mathbf{crs}, x, \pi) = 1 \wedge (x, w) \notin R \end{array} \right] \leq \text{negl}(\lambda).$$

In addition, Φ is *succinct* if the proof size, i.e., $|\pi|$, is at most logarithmic in $|w|$ (the witness size).

In this paper, we consider the following two types of non-interactive argument systems with different combinations of properties.

- **NIZK:** Satisfy the completeness, the zero-knowledge property, and the simulation extractability. Due to technical reasons, which will become clear when demonstrating our coin protocol, we require NIZK in the standard model, which admits various standard instantiations, such as Groth-Sahai [58] (plus generic compiler for simulation extractability [69]), that work for a wide range of relations and even all NP relations.
- **SNARK:** Satisfy the completeness, knowledge soundness, and the succinctness. In particular, there are SNARK schemes, such as Groth-16 [56] and [48], that enjoy *constant* proof size and work for all NP relations.

Multi-valued Byzantine Agreement (MBA). In an (n, t, ℓ) -MBA protocol, there are n parties $P_1, \dots, P_n$, each P_i having an ℓ -bit initial input v_i . Against any adversary $\mathcal{A}$ that corrupts up to t parties, a secure MBA ensures the following properties:

- **Validity.** If all honest parties share the same input v , they all eventually output v .
- **Agreement.** If an honest party P_i outputs v , then all honest party eventually outputs v .
- **Termination.** All honest parties eventually produce an output message.

Multi-valued Validated Byzantine Agreement (MVBA). In an (n, t, ℓ) -MVBA protocol with a predicate $\text{Predicate} : \{0, 1\}^\ell \rightarrow \{0, 1\}$, there are n parties $P_1, \dots, P_n$, each P_i having an ℓ -bit initial input v_i . Against any adversary $\mathcal{A}$ that corrupts up to t parties, a secure MVBA ensures the following properties:

- **External Validity.** If an honest party outputs v , then it must be that $\text{Predicate}(v) = 1$.
- **Agreement.** If an honest party P_i outputs v , then all honest party eventually outputs v .
- **Termination.** If at least $n - f$ honest nodes are activated in the instance, and the input v_i of each P_i is valid, *i.e.*, $\text{Predicate}(v_i) = 1$, then all honest nodes eventually output a value.

Provable Broadcast. In an instance of a provable broadcast (PB) protocol, there is a designated party, the sender, who has an input message v to be delivered to the network. When v is “successfully” delivered, the sender can obtain a proof ρ , which can be verified by an algorithm $\text{PB}[\text{sid}].\text{Vrfy}$, where sid is the session ID of the instance. In addition, an external predicate Predicate on the message may be considered, such that only messages satisfying this predicate shall be delivered. Following Abraham et al. [5], an (n, f) -PB protocol shall satisfy the following properties except with a negligible probability, for any session ID sid , and any PPT adversary controlling up to f nodes.

- **Integrity.** An honest node delivers at most one message.
- **External Validity.** If an honest node delivers v' , then $\text{Predicate}(v') = 1$.
- **Abandon-ability.** An honest node can call a function $\text{Abandon}(\text{sid})$ of the instance, such that this node will not deliver any message for this instance.
- **Termination.** If the sender is honest, and the input v satisfies $\text{Predicate}(v) = 1$, then any honest party eventually delivers v for this instance if it does not call $\text{Abandon}(\text{sid})$. In addition, if no honest party called $\text{Abandon}(\text{sid})$, then the sender can obtain a proof ρ , such that $\text{PB}[\text{sid}].\text{Vrfy}(v, \rho) = 1$.
- **Provability.** For any two pairs (v, ρ) and (v', ρ') , if $\text{PB}[\text{sid}].\text{Vrfy}(v', \rho') = \text{PB}[\text{sid}].\text{Vrfy}(v, \rho) = 1$ it follows that $v' = v$. In addition, if a (potentially corrupted) sender can output a valid proof ρ for v , then at least $f + 1$ forever-honest nodes delivered v in this instance.

Algorithm 5 The provable broadcast protocol PB with identifier sid , the sender P_s , and the predicate Predicate

```

// Protocol for the sender  $P_s$ 
Initialize local variable  $S = \emptyset$ 
1: upon receiving input  $v_s$  satisfying  $\text{Predicate}(v) = 1$  do
2:   Multicast ( $\text{sid}$ ,  $\text{INPUT}$ ,  $v_s$ )
3:   wait until  $|S| = n - f$ 
4:    $\varsigma \leftarrow \text{STS.Combine}(vk, S, (\text{sid}, v_s))$ 
5:   return  $\text{b-proof} = \varsigma$ .
6: upon receiving ( $\text{sid}$ ,  $\text{ECHOINPUT}$ ,  $\varsigma_{s,j}$ ) from  $P_j$  for the first time do
7:   if  $\text{TSig.PartVrfy}(pvk, pvk_j, \varsigma_{s,j}, (\text{sid}, v_s)) = 1$  then
8:      $S \leftarrow S \cup \{(j, \varsigma_{s,j})\}$ 
// Protocol for other  $P_i$ 
Initialize local variable  $v[\text{sid}] = \perp$  and  $\text{stop} = 0$ 
9: upon receiving ( $\text{sid}$ ,  $\text{INPUT}$ ,  $v_s$ ) from  $P_s$  for the first time do
10:  if  $\text{Predicate}(v_s) = 1$  and  $\text{stop} = 0$  then
11:     $v[\text{sid}] \leftarrow v_s$ 
12:     $\varsigma_{s,i} \leftarrow \text{TSig.PartS}(tvk_i, tsk_i, (\text{sid}, v_s))$ 
13:    Send ( $\text{sid}$ ,  $\text{ECHOINPUT}$ ,  $\varsigma_{s,i}$ ) to  $P_s$ 
14:    Return  $v[\text{sid}]$ 
15: procedure  $\text{Abandon}(\text{sid})$ 
16:    $\text{stop} \leftarrow 1$ 
17:    $\text{Vrfy}(\text{sid}, \text{b-proof}, v)$ 
18:   return  $\text{TSig.Vrfy}(pvk, (\text{sid}, v), \text{bproof})$ 

```

Instantiation. There is a simple construction based on threshold signature [22] to realize PB with $O(n(\ell + \lambda))$ communication complexity and 2 rounds. In particular, the sender first multicasts the message v to all recipients. Upon receiving a message v from the sender, a recipient will deliver the message and return a partial signature on v to the sender if the message satisfies the predicate. After receiving $2f + 1$ partial signatures, the sender can aggregate them into a constant-size threshold signature, which is the delivery proof ρ for v . We detail this construction in Algorithm 5 in Appendix B.2. Note that the threshold signature can be realized using a silent setup threshold signature (STS) scheme, for example, [30].

For completeness, we include an instantiation available in Algorithm 5, which follows the classical construction of [5, 22].

B.3 Silent-Setup Threshold Signature

A threshold signature scheme allows a group of nodes to jointly sign a message w.r.t. a group public key, while an adversary controlling below a threshold number of nodes cannot forge a signature. It is a silent threshold signature scheme if it has a silent setup phase¹¹.

Formally, an (n, t) silent threshold signature (STS) scheme can be described using the following algorithms.

- $\text{SilSetup}(1^\lambda, n, t) \rightarrow (\text{crs}, vk, (vk_i), (sk_i))$. It is the silent setup phase of STS. We denote the public verification key and the secret signing key by vk and sk , respectively.

¹¹ We remark that existing silent threshold signature schemes lack uniqueness and therefore do not imply a common coin under Cachin et al.'s framework [22].

- $\text{PartSign}(vk, vk_i, sk_i, \text{msg}) \rightarrow \sigma_i$. It signs a message msg using the signing secret key sk_i of the user i and produces a partial signature σ_i .
- $\text{PartVrfy}(vk, vk_i, \sigma_i, \text{msg}) \rightarrow b \in \{0, 1\}$. It validates a partial signature σ_i on the message msg under the public key vk_i of the user i .
- $\text{Combine}(vk, \{(vk_i, \sigma_i)\}_{i \in S}, \text{msg}) \rightarrow \sigma$. It combines $t + 1$ valid signatures for the same message msg under distinct public keys into one signature σ .
- $\text{Vrfy}(vk, \sigma, \text{msg}) \rightarrow b \in \{0, 1\}$. It validates a combined signature σ for a message msg under the group public key vk .

An STS scheme satisfies the correctness and the unforgeability.

Correctness. An (n, t) -STS scheme is *correct*, if for every $\lambda \in \mathbb{N}$ and any PPT adversary $\mathcal{A}$, under an adversary-involved setup $\text{SilSetup}^{\mathcal{A}}(1^\lambda, n, t) \rightarrow (\text{crs}, vk, (vk_i)_{i \in [n]}; \text{state}_{\mathcal{A}}; (sk_i)_{i \in [n] \setminus \mathcal{C}_{\text{init}}})$, we have the following properties.

- *Partial Verification Correctness.* For any m and any $i \notin \mathcal{C}_{\text{init}}$, it holds that

$$\Pr[\text{PartVrfy}(vk, vk_i, \text{PartSign}(vk, vk_i, sk_i, \text{msg}), \text{msg}) = 1] = 1.$$

- *Combination Correctness.* For any subset $S \subseteq [n]$ with $|S| = t + 1$, and $\{(vk_i, \sigma_i)\}_{i \in S}$ such that $\text{PartVrfy}(vk, vk_i, \sigma_i, \text{msg})$ for the same message msg , it holds that

$$\Pr[\text{Vrfy}(vk, \text{Combine}(vk, (vk_i, \sigma_i)_{i \in S}, \text{msg}) = 1] = 1.$$

Unforgeability. At a high level, an (n, t) -STS scheme satisfies the unforgeability if any PPT adversary $\mathcal{A}$ controlling up to t nodes cannot forge a valid signature on a message that has not been signed by any honest nodes. Furthermore, we allow the adversary to perform the chosen message attacks (modeled by a signing oracle $\mathcal{O}_{\text{PartS}}$ in Fig.). In addition, an adaptive adversary may corrupt participants at any point; the capability of adaptive corruption is modeled by $\mathcal{O}_{\text{Corr}}$.

Formally, an (n, t) -STS scheme is unforgeable, if for any PPT adversary $\mathcal{A}$ and $\lambda \in \mathbb{N}$, it holds that

Formally, we say an (n, t) -STS scheme satisfies the unforgeability if for any PPT adversary $\mathcal{A}$, it holds that, for every $\lambda \in \mathbb{N}$,

$$\Pr[\text{UNF}^{\mathcal{A}}(1^\lambda, n, t) = 1] \leq \text{negl}(\lambda). \quad (6)$$

An (n, f) -STHE scheme satisfies the adaptive unforgeability, if

$$\Pr[\text{Ada-UNF}^{\mathcal{A}}(1^\lambda, n, t) = 1] \leq \text{negl}(\lambda). \quad (7)$$

Instantiation. Silent threshold signatures have various instantiations [6, 30, 51, 79]. In this work, we consider the scheme from [30] as the instantiation, as it offers the constant signature size and the constant verification cost while ensuring the adaptive unforgeability under the q -SDH assumption in the random oracle model. We remark that while the corruption oracle in the security of [30] does not leak the randomness w.r.t. partial signing, the partial signing algorithm in [30] (and most other existing schemes) is deterministic, so there is no randomness to leak.

Ada -UNF ^A (1 ^λ , n, t)	
$\mathcal{C}, \text{State}_1, \dots, \text{State}_n \leftarrow \emptyset, \text{Msgs} \leftarrow \perp$	
$(\text{crs}, vk, (vk_i)_{i \in [n]}; \text{state}_{\mathcal{A}}; (sk_i)_{i \in [n] \setminus \mathcal{C}_{\text{init}}}) \leftarrow \text{SilSetup}^{\mathcal{A}}(1^\lambda, n, t), \mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}_{\text{init}}$	
$(\sigma^*, \text{msg}^*) \leftarrow \mathcal{A}^{\boxed{\mathcal{O}_{\text{Corr}}(\cdot)}, \mathcal{O}_{\text{PartS}}(\cdot)}(\text{state}_{\mathcal{A}})$	
if $\text{msg}^* \notin \text{Msgs} \wedge \text{Vrfy}(vk, \sigma^*, \text{msg}^*) = 1$, then return 1; else return 0	
$\mathcal{O}_{\text{Corr}}(i)$	$\mathcal{O}_{\text{PartS}}(\text{msg}, i)$
if $i \in [n] \setminus \mathcal{C} \wedge \mathcal{C} < t$	if $i \notin \mathcal{C}$
$\mathcal{C} \leftarrow \mathcal{C} \cup \{i\}$	$\text{Msgs} \leftarrow \text{Msgs} \cup \{\text{msg}\}$
return (sk_i, State_i)	$\gamma \leftarrow \text{Rand}_{\text{PartS}}, \sigma_i \leftarrow \text{PartSign}(vk, vk_i, sk_i, \text{msg}; \gamma)$
	$\text{State}_i \leftarrow \text{State}_i \cup \{(\text{msg}, i, \gamma, \sigma_i)\}$ return σ_i

Fig. 8. Security Game of STS. $\boxed{\mathcal{O}_{\text{Corr}}}$ is only included in Ada-UNF

C Supplementary Material to §4.2

C.1 Composition Insecurity of An AVSS Scheme

We show a concrete example of a protocol that satisfies all security properties in the stand-alone setting becoming insecure when two instances of the same protocol run together. Our example is an artificial modification of the asynchronous verifiable secret sharing (AVSS) protocol by Das et al. [34]. Das et al.'s AVSS works as follows:

- The dealer samples a random polynomial f whose $f(0)$ is the secret, sends every receiver P_i a share $f(i)$ and a polynomial commitment [64] com to f , along with some proof showing $f(i)$ is the correct evaluation w.r.t. the committed polynomial.
- Each receiver P_i , after verifying $f(i)$ is an evaluation at i of the committed polynomial, signs com , and returns the signature to the dealer.
- Then, the dealer, after collecting $n - f$ valid signatures, will *broadcast* (via a RBC [20]) the commitment com , all $n - f$ signatures, and the remaining $f(j)$ for all P_j which has not returned the signature.

To show what might go wrong, let us focus on the following important property of AVSS.

- *Completeness of AVSS*: If an honest node terminates in the AVSS instance, then every honest node can deliver a correct share.

Note that the AVSS scheme described above satisfies the completeness because the valid shares are either available via the broadcast or the corresponding receivers have confirmed the delivery by their signature.

Now, let us assume the underlying signature scheme is a one-time signature, which ensures that an adversary cannot forge a signature of vk_i when only seeing one signature of P_i , while the signature can be easily forged after seeing two signatures under vk_i . It is easy to see that the protocol can still guarantee single-instance security. However, when two instances using the same

PKI setup run together, the adversary can forge the signature of honest nodes without sending corresponding shares, which violates completeness.

Note that AVSS is fundamental to almost all asynchronous common coin protocols, and more importantly, each common coin instance will invoke several AVSS instances. Therefore, a single-instance-secure AVSS is not secure in the composition of coin protocols, which indicates the composition security for asynchronous coins and consensus deserves a more serious treatment.

D The Details of Our STHE Scheme

Our STHE scheme represents a minimal modification from the STE scheme in [52]. To put it simply, the encryption algorithm in the STE scheme involves uniformly sampling a group element Γ , which will also be included in the ciphertext as a public element. To enable the desired tag-homomorphism, we need to replace the random element Γ with $H(\text{tag})$, where H is a hash function modeled as a random oracle.

The STE scheme [52] can be considered as signature-based witness encryption [7] w.r.t. the STS scheme [51], where the plaintext is encrypted w.r.t. a “message” and a public key, and it can be decrypted by a valid signature for the “message” under the public key. The group element Γ in the ciphertext is the “message” from this perspective. As a message $\mathbf{m}$ in the STS scheme is encoded as $H(\mathbf{m})$, a “natural” version of STE, which is discussed in the introduction of [52], indeed uses $H(\text{tag})$ in place of a random Γ . The final scheme of STE in [52] uses a random element Γ for the purpose of avoiding the additional random oracle assumption.

For completeness, we include the description of the STHE scheme in this section, which is basically the “natural” version of STE scheme [52], while we add the homomorphic evaluation algorithm `Eval` to make it an STHE.

Notations and Bilinear Groups. We introduce the notations for polynomial and group operations. The following content is directly from [52] and [51].

We use the following notations for polynomials over finite fields. Let $\mathbb{H} \subset \mathbb{F}$ be a multiplicative subgroup of a finite field $\mathbb{F}$. Let ω be the generator of $\mathbb{H} = \{\omega, \omega^2, \dots, \omega^{|\mathbb{H}|} = 1\}$.

Let $L_1(x), L_2(x), \dots, L_{|\mathbb{H}|}(x)$ be the Lagrange basis polynomial. That is, L_i is the unique degree $|\mathbb{H}| - 1$ polynomial defined by:

$$L_i(\omega^j) = \begin{cases} 1 & \text{when } i = j, \\ 0 & \text{when } i \neq j. \end{cases}$$

Let $Z(x) = \prod_{i=1}^{|\mathbb{H}|} (x - \omega^i)$ be the vanishing polynomial on $\mathbb{H}$. Since $\mathbb{H}$ is a multiplicative subgroup, $Z(x) = x^{|\mathbb{H}|} - 1$ and

$$L_i(x) = \frac{\omega^i}{|\mathbb{H}|} \cdot \frac{x^{|\mathbb{H}|} - 1}{x - \omega^i}.$$

Note that $L_i(0) = |\mathbb{H}|^{-1}$. In the construction, We use $|\mathbb{H}| = n + 1$, where n is the number of parties.

A Bilinear Group $\mathcal{BG}$, generated as $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e) \leftarrow \mathcal{BG}(1^\lambda)$, is specified by three groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ (the first two we call ‘source groups’ and the third ‘target group’) of prime order $p = 2^{\Theta(\lambda)}$, a bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ that we call ‘pairing’ and one random generator g_1, g_2 for each group. We use the *implicit notation*, i.e., $[x]_s := x \cdot g_s$ and more generally $[A]_s$ represents a matrix of the corresponding group elements, for $s \in \{1, 2, t\}$.

Also, we denote the group operation additively, $[x]_s + [y]_s = [x + y]_s$, for $s \in \{1, 2, t\}$. By $[x]_1 \circ [y]_2 = [y]_2 \circ [x]_1 = [x \cdot y]_T$, we denote the pairing $e([x]_1, [y]_2)$. We note that the way it is defined ‘ $\circ$ ’ is commutative, for instance $([a]_1, [b]_2)^\top \circ ([c]_2, [d]_1)$ is well-defined and gives the outcome $[ac + bd]_T$.

All the algorithms of our constructions implicitly take as input a Bilinear Group generated by $\mathcal{BG}(\lambda)$, even if it is not explicitly stated.

The construction. Most construction descriptions are from [52]. We simplified the interfaces by merging a few algorithms and variables. For example, the user key generation algorithm and the “hint” generation algorithm are included in a single key generation algorithm; the group encryption key and aggregation public key are treated as a single group public key. We also decouple the decryption algorithm into a decryption key aggregation algorithm and a decryption algorithm using this key, which enables the use of a single key to decrypt all ciphertexts under the same tag. We highlighted the changes for tag-homomorphism in green.

- $\text{SilSetup}(1^\lambda, n, t)$: It can be described by a CRS generation algorithm CRS , a key generation algorithm KeyGen , and a deterministic group key aggregation algorithm GroupPkGen .
 - $\text{CRS}(1^\lambda, n)$: Sample $\tau \leftarrow \mathbb{Z}_p$ and output:

$$\text{crs} = ([\tau^1]_1, \dots, [\tau^{n+1}]_1, [\tau^1]_2, \dots, [\tau^{n+1}]_2).$$

- $\text{KeyGen}(\text{CRS}, i, n)$: Sample $x_i \leftarrow \mathbb{Z}_p^*$, and set $\text{dk}_i = x_i$, $\text{enck}_i = [x_i]_1$. Then, compute

$$\text{hint}_i = \left(\begin{array}{c} [\text{dk}_i L_i(\tau)]_1, [\text{dk}_i (L_i(\tau) - L_i(0))]_1, \left[\text{dk}_i \frac{L_i^2(\tau) - L_i(\tau)}{Z(\tau)} \right]_1, \\ \left[\text{dk}_i \frac{L_i(\tau) - L_i(0)}{\tau} \right]_1, \left\{ \left[\text{dk}_i \frac{L_i(\tau) L_j(\tau)}{Z(\tau)} \right]_1 \right\}_{j \in [0, n], j \neq i} \end{array} \right).$$

Output $ek_i = (\text{enck}_i, \text{hint}_i)$, and $\text{dk}_i = x_i$.

- $\text{GroupPKGen}(\text{CRS}, \{\text{hint}_i, ek_i\}_{i \in [n]})$: Verify the validity of each ek_i : for each $i \in [n]$, run $\text{isValid}(\text{CRS}, ek_i)$ (isValid is defined below) and let $V \subseteq [n]$ be the set of the indices with valid public keys. Set $\text{dk}_0 = 1$ and set $\text{dk}_i = 0$ for each $i \notin V$. Compute

$$\text{ak} = \left(\begin{array}{c} V, \{ek_i\}_{i \in V \cup \{0\}}, \{[\text{dk}_i (L_i(\tau) - L_i(0))]_1\}_{i \in V \cup \{0\}}, \\ \left\{ \left[\text{dk}_i \frac{L_i^2(\tau) - L_i(\tau)}{Z(\tau)} \right]_1 \right\}_{i \in V \cup \{0\}}, \left\{ \left[\text{dk}_i \frac{L_i(\tau) - L_i(0)}{\tau} \right]_1 \right\}_{i \in V \cup \{0\}}, \\ \left\{ \left[\sum_{j \in S, j \neq i} \text{sk}_j \frac{L_j(\tau) L_i(\tau)}{Z(\tau)} \right]_1 \right\}_{i \in V \cup \{0\}} \end{array} \right).$$

- enck :

$$\text{enck} = \left(\left[\sum_{i \in V} \text{dk}_i L_i(\tau) \right]_1, [Z(\tau)]_2 \right) := (C, Z).$$

Output $ek = (\text{ak}, \text{enck})$.

- $\text{Enc}(ek, \text{tag}, \text{msg})$: Set $[\gamma]_2 = H(\text{tag}) \in \mathbb{G}_2$ (rather than sample $[\gamma]_2 \leftarrow \$ \mathbb{G}_2$ in [52]), parse $ek := (ak, C, Z)$, set $Z_0 = [\tau - \omega^0]_2$ and set

$$A_{\text{tag}} = \begin{pmatrix} C & [1]_2 & Z & [\tau]_2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & [\tau]_2 & [1]_2 & 0 & 0 & 0 \\ 0 & [\gamma]_2 & 0 & 0 & 0 & [1]_1 & 0 & 0 \\ [\tau^t]_1 & 0 & 0 & 0 & 0 & 0 & [1]_2 & 0 \\ [1]_1 & 0 & 0 & 0 & 0 & 0 & 0 & Z_0 \end{pmatrix}.$$

$$\mathbf{b} = ([0]_T, [0]_T, [0]_T, [0]_T, [1]_T)^\top.$$

Notice that each column of A contains elements from the same source group (looking ahead, this is so that the pairing $A \circ \omega$ can be properly performed.)

Sample a vector $\mathbf{s} \leftarrow (\mathbb{Z}_p^*)^5$ and output:

$$c = \left(\mathbf{s}^\top \cdot A_{\text{tag}}, \mathbf{s}^\top \cdot \mathbf{b} + \text{msg} \right).$$

- $\text{Eval}(ek, \text{tag}, \{c_i\}_{i \in S})$: Parse $c_i = (\mathbf{ct}_{i,1}, \mathbf{ct}_{i,2})$. Output $c^* = (\sum_{i \in S} \mathbf{ct}_{i,1}, \sum_{i \in S} \mathbf{ct}_{i,2})$.
- $\text{PartDec}(ek, \text{tag}, \mathbf{dk}_i)$: Output $\zeta_i = \mathbf{dk}_i \cdot H(\text{tag})$.
- $\text{PartVrfy}(ek, \text{tag}, ek_i, \zeta_i)$: Parse $ek_i := (ak_i, \text{enck}_i, \text{hint}_i)$, and output 1 if and only if $\text{enck}_i \circ H(\text{tag}) = [1]_1 \circ \zeta_i$.
- $\text{DecAgg}(ek, \text{tag}, \{\zeta_i\}_{i \in S})$: Parse $ek := (ak, \text{enck} = (C, Z))$, and compute the subset of indices with valid ek_i 's, $S_v = S \cap V$. Then proceed as follows:
 1. Compute a polynomial $B(X)$ by interpolating 0 on all ω_i with $i \notin S_v$ and 1 on ω^0 , i.e., interpolate B as $\{(\omega^0, 1), (\omega^i, 0)_{i \notin S_v}\}$ and set

$$B = \left[\sum_{i=0}^n B(\omega^i) L_i(\tau) \right]_2.$$

2. Set

$$\mathbf{aPK} = \frac{1}{n+1} \left(\sum_{i \in S_v} B(\omega^i) \text{enck}_i + [1]_1 \right).$$

3. Compute polynomials $Q_x(X)$ and $Q_z(X)$ such that

$$\text{SK}(X)B(X) = \frac{\mathbf{aSK}}{n+1} + Q_z(X)Z(X) + Q_x(X)X$$

where

$$\mathbf{aSK} = \sum_{i=0}^M \text{SK}(\omega^i) B(\omega^i) := \sum_{i \in S_v} \mathbf{dk}_i B(\omega^i) + 1.$$

By definition, B evaluates to 0 outside S_v and to 1 on ω^0 .

According to Lemma 1, they can be computed as:

$$Q_z(X) = \sum_{i=0}^{n+1} B(\omega^i) \left(\mathbf{dk}_i \frac{L_i^2(X) - L_i(X)}{Z(X)} \right) + \left(\sum_{i=0}^{n+1} B(\omega^i) \sum_{\substack{j=0 \\ j \neq i}}^{n+1} \mathbf{dk}_j \frac{L_i(X) L_j(X)}{Z(X)} \right),$$

$$Q_x(X) = \sum_{i=0}^{n+1} B(\omega^i) \left(\text{dk}_i \frac{L_i(X) - L_i(0)}{X} \right).$$

Then using ak , compute $Q_z = [Q_z(\tau)]_1$.

4. Compute $Q_x = [Q_x(\tau)]_1$.
5. Compute $\hat{Q}_x = [Q_x(\tau) \cdot \tau]_1$.
6. Set $\zeta^* = \frac{1}{n+1} (\sum_{i \in S_v} B(\omega^i) \zeta_i + H(\text{tag}))$.
7. Compute $\hat{B} = [\tau B(\tau)]_1$.
8. Compute a KZG evaluation at ω^0 , i.e., compute $Q_0(X)$ such that $B(X) - 1 = Q_0(X)(X - \omega^0)$ and compute $Q_0 = [Q_0(\tau)]_1$.
9. Output:

$$dk_{\text{tag}} = \left(B, -\text{aPK}, -Q_z, -Q_x, \hat{Q}_x, \zeta^*, -\hat{B}, -Q_0 \right)^\top,$$

- $\text{Dec}(ek, \text{tag}, c, dk_{\text{tag}})$: Parse $c = (\text{ct}_1, \text{ct}_2)$ and output:

$$\text{msg}^* = \text{ct}_2 - \text{ct}_1 \circ dk_{\text{tag}}.$$

In order to confirm that hint_i is well-formed in the Preprocess algorithm, we use an `isValid` algorithm which we define as follows:

$$\begin{aligned} \text{isValid}(\text{CRS}, ek_i) &\rightarrow \{0, 1\} : \text{ parses } ek_i := (\text{enck}_i, \text{hint}_i), \\ &\text{ and } \text{hint}_i := (h_1, h_2, h_3, h_4, \{h_{5,j}\}_{j \in [0, M], j \neq i}) \end{aligned}$$

And outputs 1 iff it holds that:

1. $h_1 \circ [1]_2 = \text{enck}_i \circ [L_i(\tau)]_2$,
2. $h_2 \circ [1]_2 = \text{enck}_i \circ [(L_i(\tau) - L_i(0))]_2$,
3. $h_3 \circ [1]_2 = \text{enck}_i \circ \left[\frac{L_i^2(\tau) - L_i(\tau)}{Z(\tau)} \right]_2$,
4. $h_4 \circ [1]_2 = \text{enck}_i \circ \left[\frac{L_i(\tau) - L_i(0)}{\tau} \right]_2$,
5. $h_5 \circ [1]_2 = \text{enck}_i \circ \left[\frac{L_i(\tau) L_j(\tau)}{Z(\tau)} \right]_2$, for each $j \in [0, M], j \neq i$.

Efficiency. It is almost identical to the STE scheme in [52], except that the ciphertext size is slightly reduced since there is no random element $\Gamma \in \mathbb{G}_2$ included in the ciphertext. In particular, the CRS contains $n + 1$ elements in $\mathbb{G}_1$ and $n + 1$ elements in $\mathbb{G}_2$, whose size is $O(\lambda n)$. The public key ek_i of each node P_i contains $n + 4$ elements in $\mathbb{G}_1$, whose size is also $O(\lambda n)$. The group public key ek contains $5n + 6$ elements in $\mathbb{G}_1$ and one element in $\mathbb{G}_2$, whose size is also $O(\lambda n)$. The size of dk_i is $O(\lambda)$. Notice that each node obtains this information in the setup phase.

A ciphertext contains 2 elements in $\mathbb{G}_1$, 6 elements in $\mathbb{G}_2$, and 1 element in $\mathbb{G}_T$, so its size is $O(\lambda)$. A partial decryption key consists of 1 element in $\mathbb{G}_2$, which is $O(\lambda)$ bits.

Correctness. The *partial verification correctness* and the *decryption correctness* trivially follow the correctness of the STE scheme. The evaluation correctness is easy to verify. In particular, for every $c_i = \text{Enc}(ek, \text{tag}, \text{msg}_i; \gamma')$, it has the following form:

$$c_i = (\text{ct}_{i,1}, \text{ct}_{i,2}) = \left(\mathbf{s}_i^\top \cdot A_{\text{tag}}, \mathbf{s}_i^\top \cdot b + \text{msg}_i \right),$$

where $\mathbf{s}_i$ is the internal randomness (γ' in the notation) for each ciphertext, while A_{tag} is uniquely determined by tag , so every c_i has the same A_{tag} and b . Thus, $c^* = \text{Eval}(ek, tag, \{c_i\}_{i \in S})$ has the following form:

$$(\sum_{i \in S} \text{ct}_{i,1}, \sum_{i \in S} \text{ct}_{i,2}) := \left(\left(\sum_{i \in S} \mathbf{s}_i^\top \right) \cdot A_{tag}, \left(\sum_{i \in S} \mathbf{s}_i^\top \right) \cdot b + \sum_{i \in S} \text{msg}_i \right),$$

which is a valid encryption for $\sum_{i \in S} \text{msg}_i$ w.r.t. the randomness $\mathbf{s}^* = \sum_{i \in S} \mathbf{s}_i^\top$.

D.1 STHE Security

While the STHE scheme described above represents a minor modification from the STE scheme in [52], the security we defined is significantly stronger than the security definition of STE [52]. Specifically, in our definition, (1) the adversary is allowed to query the partial decryption oracle $\mathcal{O}_{\text{PartD}}$ to obtain the partial decryption key w.r.t. a tag $tag \neq tag^*$ from any uncorrupted party, and (2) it can adaptively corrupt parties. In contrast, in [52], the corruption is static, and the adversary cannot obtain the partial decryptions of any uncorrupted parties.

In this section, we argue that the STHE scheme indeed satisfies the stronger security definition. Our arguments are based on the original security proof for STE in [52]. At a high level, for handling the queries to $\mathcal{O}_{\text{PartD}}$, we use a standard trick for random oracles so that a partial decryption key $\zeta_i = \text{dk}_i \cdot H(tag)$ can be simulated as $\hat{r}_{tag} \cdot \text{enck}_i = \text{dk}_i \cdot \hat{r}_{tag} \cdot [1]_2$ when $H(tag)$ is programmed as $\hat{r}_{tag} \cdot [1]_2$. In this way, the simulator can respond to queries to $\mathcal{O}_{\text{PartD}}$ without using any information about dk_i . In addition, for handling the *adaptive* corruption, we can apply the simulation techniques from [14] in a black box manner, which ensures that in the GGM, even under adaptive corruption, the adversary still does not know any non-trivial information about the secret keys of uncorrupted parties.

In the following, we first recall the basic background of the generic group model and some useful results from [14], and then we discuss why STHE satisfies the stronger security definition.

Generic Group Model. In GGM, the adversary cannot see the concrete presentations of group elements and can only use generic group operations. In particular, the adversary makes oracle queries of each group operation it wishes to perform and receives a handle for the resulting group elements. Generally, the adversary is initially provided by the handle of the element $[1]_1, [1]_2$ and $[1]_T$, and it can query the oracle GC_{MP} for group operations, which takes as input two strings ξ and ξ' and a bit b , which indicates whether to compute the addition or the subtraction of the group elements. In groups supporting pairing operations, there is an oracle GC_{Pair} , which takes as input two strings ξ and ξ' and returns $\hat{\xi}$, handling for an element in $\mathbb{G}_T$ as the result of this pairing. Note that the “scalar multiplication” can be done trivially with the oracle GC_{MP} .

Useful Results from [14]. Now we recall some useful results from [14], which demonstrates the one-more discrete logarithm problem is unconditionally hard in GGM. In particular, the adversary is given (the handlings of) n random group elements $(g_1, \dots, g_n)$, and can query the oracle DLog , which returns the scalar (i.e., discrete logarithm) of the queried (handling of) group element, for at most $n - 1$ times. The adversary’s goal is to produce the scalars of all the n elements. In the security proof in [14], the authors show a simulator (in [14, Fig.6]) that can simulate the security game without ever computing the scalars of g_i , such that the adversaries cannot output those scalars

except with a negligible probability. We summarize some useful results from [14] in the following lemma.

Lemma 14 ([14]). *Consider an adversary $\mathcal{A}$ which is given the handleings of $(g_1, \dots, g_n)$ and only allowed to query the oracle DLog with those handleings. There is a simulator $\mathcal{S}_{\text{GGM}}$, which can simulate the handleings $(\xi_1, \dots, \xi_n)$ of $(g_1, \dots, g_n)$ and responses for DLog and GCMP , without assigning a scalar k_i to a handling ξ_i (such that ξ_i is the handling of $k_1 \cdot [1]_1$) until DLog is queried with ξ_i . In particular, $\mathcal{S}_{\text{GGM}}$ uses random variables $K_1, \dots, K_n$ to represent the scalars $(k_1, \dots, k_n)$, and ensures the invariance in simulating group operation oracles by recording each query as a polynomial in $(K_1, \dots, K_n)$. Except with a negligible probability that $\mathcal{S}_{\text{GGM}}$ may abort, the simulation, at the point of $\mathcal{A}$'s view, is indistinguishable from a “real” security game where $(k_1, \dots, k_n)$ are created for $(g_1, \dots, g_n)$ and all the oracle queries are responded honestly.*

Security analysis of STHE. The security proof for STHE can be done by going through the following hybrid security games.

Hybrid 0. This is the standard security game of STHE. Since the scheme is in the generic group model and the random oracle model, the adversary $\mathcal{A}$ also has access to the random oracle H and oracles w.r.t. GGM

Hybrid 1. This hybrid falls back to the “tag-free” setting, where the encryption algorithm STHE.Enc does not take tag as an input, and the random oracle H and the partial decryption oracle $\mathcal{O}_{\text{PartD}}$ are totally removed from the security game. Instead, the encryption algorithm uses a freshly sampled random group element $[\gamma]_2$ in place of $H(\text{tag})$, just like the original STE scheme in [52], while the random element $[\gamma^*]_2$ used in the challenge oracle $\mathcal{O}_b$ is sampled in advance and available to $\mathcal{A}$. For clarity, we describe this hybrid in Fig.9.

Hybrid1 $^{\mathcal{A}}_b(1^\lambda, n, f)$	$\mathcal{O}_b(m_0, m_1)$
$\mathcal{C} \leftarrow \emptyset, [\gamma^*]_2 \leftarrow \mathbb{G}_2$	return $c_b \leftarrow \text{Enc}(ek, m_b; [\gamma^*]_2)$
$(\text{crs}, ek, (ek_i)_{i \in [n]}; \text{state}_{\mathcal{A}}; (dk_i)_{i \in [n] \setminus \mathcal{C}_{\text{init}}})$	
$\leftarrow \text{SilSetup}^{\mathcal{A}([\gamma^*]_2)}(1^\lambda, n, t), \mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}_{\text{init}}$	$\mathcal{O}_{\text{Corr}}(i)$
$1 \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Corr}}(\cdot) \mathcal{O}_b(\cdot)}(\text{state}_{\mathcal{A}})$	if $i \in [n] \setminus \mathcal{C} \wedge \mathcal{C} < t$
	$\mathcal{C} \leftarrow \mathcal{C} \cup \{i\}, \text{return } (dk_i)$

Fig. 9. Hybrid 1

For the two hybrids, we have the following results.

Proposition 5. *If there is a PPT adversary $\mathcal{A}_0$ that can win the security game in Hybrid 0 with a non-negligible probability, then there exists a PPT adversary $\mathcal{A}_1$ that can win the security game in Hybrid 1 with a non-negligible probability.*

Proof. $\mathcal{A}_1$ can invoke $\mathcal{A}_0$ as a subroutine by simulating Hybrid 0 based on the information from Hybrid 1. In particular, $\mathcal{A}_1$ can program the random oracle H such that $H(\text{tag}^*) := [\gamma^*]_2$. For other $\text{tag} \neq \text{tag}^*$, it samples $\hat{r}_{\text{tag}} \leftarrow \mathbb{Z}_p$, and returns $\hat{r}_{\text{tag}} \cdot [1]_2$ as the output of $H(\text{tag})$. Then, $\mathcal{A}_1$ can respond a query (tag, i) to $\mathcal{O}_{\text{PartD}}$ as $\hat{r}_{\text{tag}} \cdot \text{enck}_i$. When $\mathcal{A}_0$ queries $\mathcal{O}_b$ or $\mathcal{O}_{\text{Corr}}$, $\mathcal{A}_1$ just forwards

the query to the oracles in Hybrid 1. Note that the simulation provided by $\mathcal{A}_1$ is perfect at the point of $\mathcal{A}_0$'s view. So if $\mathcal{A}_0$ can distinguish $\mathcal{O}_0$ and $\mathcal{O}_1$ in Hybrid 1, it can also distinguish them in the simulation; So can $\mathcal{A}_1$. $\square$

Hybrid 2. In this hybrid, the public key of each participant P_i is generated as $(\text{enck}_i, ([\text{dk}_i\tau]_1, \dots, [\text{dk}_i\tau^n]_1))$. Note that the standard public key with hint_i can be deterministically computed from this simplified one. This hybrid is only for simplifying notations, as used in [52].

Hybrid 3. In this hybrid, the adversary $\mathcal{A}$ is interacting with a simulator $\mathcal{S}_3$, which simulates the setup phase with $\mathcal{A}$ and simulates the oracles $\mathcal{O}_{\text{Corr}}$ and $\mathcal{O}_b$. In particular, $\mathcal{S}_3$ runs a copy of $\mathcal{S}_{\text{GGM}}$ (c.f. Lemma. 14), and interacts with $\mathcal{A}$ in the following manner.

- **Setup:** $\mathcal{S}_{\text{GGM}}$ provides the handlings of $[1]_1, [1]_2, [1]_T$ as well as $\xi_1, \dots, \xi_n$ to $\mathcal{S}_3$. $\mathcal{S}_3$ samples $\tau \in \mathbb{Z}_p$, and queries the group operation oracle to create the CRS and the public keys for all uncorrupted parties. In particular, the public key for an uncorrupted party P_i is generated as follows:
 - $\text{enck}_i \leftarrow \xi_i$, while the handlings for each $[\text{dk}_i \cdot \tau^j]_1$ is obtained by querying the oracle GC_{MP} to realize the scalar multiplication between ξ_i and τ^j .
All queries for generic group operations are realized by querying the corresponding oracle provided by $\mathcal{S}_{\text{GGM}}$.
- **Queries:** $\mathcal{O}_b$ is responded by following the Fig.9. When $\mathcal{A}$ queries $\mathcal{O}_{\text{Corr}}$ with i , $\mathcal{S}_3$ queries DLog with ξ_i provided by $\mathcal{S}_{\text{GGM}}$. When DLog returns $k_i \in \mathbb{Z}_p$, $\mathcal{S}_3$ returns $\text{dk}_i := k_i$ to $\mathcal{A}$.

Note that under the condition that $\mathcal{S}_{\text{GGM}}$ does not abort, Hybrid 3 is identical to Hybrid 2 at the point of $\mathcal{A}$'s view. Following Lemma 14, the probability that $\mathcal{S}_{\text{GGM}}$ aborts is negligible.

Hybrid 4. This hybrid is almost identical to Hybrid 3, except that in the challenge oracle $\mathcal{O}_b$, the encryption algorithm $\text{Enc}(ek, m_b; \Gamma^*)$ is realized as follows:

$$c = \left(\mathbf{s}^\top \cdot A, R^* + m_b \right),$$

where R^* is a uniformly sampled element in $\mathbb{G}_T$, and A is the matrix A_{tag} while $[\gamma]_2 = H(\text{tag})$ is now replaced with Γ^* .

It is easy to see that in Hybrid 4, the adversary has zero advantage to win this security game. So, it remains to show that any PPT adversary cannot distinguish Hybrid 4 and Hybrid 3.

At this point, the remaining part has been exactly done in [52]. The original proof in [52] focuses on a set of secret variables that are unknown to the adversary, which includes X (the random variable of the CRS trapdoor τ), $\{K_i\}_{i \in \mathcal{H}}$ (the random variables for the decryption keys of honest participants), $(S_1, \dots, S_5)$ (the random variables for $\mathbf{s} \in \mathbb{Z}_p^5$ used in encryption algorithm), and Γ^* (the random variable for the scalar γ^* of $[\gamma^*]_2$). It is proved that, assuming all secret random variables are uniformly distributed, $[S_5]_T$ (which is $\mathbf{s}^\top \cdot \mathbf{b}$) is *indistinguishable* with R^* , even when the adversary is given the handlings of $[f_1(Y)]_1$ for all $f_1(Y) \in L_1(Y)$, $[f_2(Y)]_2$ for all $f_2(Y) \in L_2(Y)$, and $[1]_T$, where

$$L_1(Y) := \left(1, X, \dots, X^{n+1}, \{K_i, K_i X, \dots, K_i X^n\}_{i \in \mathcal{H}^*}, \underbrace{S_3}_{S^\top a_6}, \sum_{i \in \mathcal{H}} K_i L_i(X) + \sum_{i \in \mathcal{C}} \text{dk}_i L_i(X) + S_4 X^t + S_5 \right),$$

$$L_2(Y) := \left(1, X, \dots, X^{n+1}, \Gamma^*, \underbrace{S_1 + \Gamma S_3}_{S^\top_{a_2}}, \underbrace{S_1 X^{n+1} - S_1}_{S^\top_{a_3}}, \underbrace{S_1 X + S_2 X}_{S^\top_{a_4}}, \right. \\ \left. \underbrace{S_2}_{S^\top_{a_5}}, \underbrace{S_4}_{S^\top_{a_7}}, \underbrace{S_5 X - S_5}_{S^\top_{a_8}} \right).$$

In Hybrid 3 and Hybrid 4, note that the secret random variables are exactly $(X, \{K_i\}_{i \in \mathcal{H}}, \Gamma^*, (S_1, \dots, S_5))$, and all information available to the adversary that is related to those secret random variables can be expressed as $[f_1(Y)]_1$ or $[f_2(Y)]_2$ for some $f_1(Y) \in L_1(Y)$ or $f_2(Y) \in L_2(Y)$. Note that due to Lemma 14, the secret keys of all uncorrupted parties are random variables that have not been assigned concrete scalars in $\mathbb{Z}_p$. Therefore, their distributions are uniform, which cannot be biased by the adversary due to adaptive corruption. The other secret information, which includes $\tau \in \mathbb{Z}_p$, γ^* (the scalar of Γ^*), and the scalars of $(S_1, \dots, S_5)$, are all uniformly sampled, and the adversary never has access to them. Therefore, the remaining proofs can be done by following [52].

E Analysis of Composition Security

Composition of Align We demonstrate a few facts that are useful for proving security when composing our Align protocol with other protocol instances.

Recall that the PB protocol uses a silent-setup threshold signature scheme STS as a building block. The security game of STS in Fig.8 enables the adversary to query partial signatures on any messages except the challenge message via an oracle $\mathcal{O}_{\text{PartS}}$. We show that exposing this partial signing oracle to the adversary $\mathcal{A}$ against Align will not undermine the security of Align.

Lemma 15. *Align[sid] is secure even when the adversary can access $\mathcal{O}_{\text{Align}}$, which consists of the following oracles:*

- $\mathcal{O}_{\widehat{\text{PartS}}}$ which on input $(\text{sid}, i, \text{msg})$ returns $\mathcal{O}_{\text{PartS}}((\text{sid}', \text{msg}), i)$.
- $\mathcal{O}_{\widehat{\text{Corr}}}$, which on input i returns the secrets and randomness used to respond to the previous queries with the form of $(\cdot, i, \cdot)$.

Note that the adversary attacking Align[sid] can only query $\mathcal{O}_{\widehat{\text{PartS}}}$ with $(\text{sid}', \cdot, \cdot)$ s.t. $\text{sid}' \neq \text{sid}$.

Proof. Note that the security of Align solely relies on the security of each PB instance, which, in turn, relies on the unforgeability of the underlying STS scheme. Specifically, PB[sid] is secure when the adversary cannot forge an STS signature on a message in the form of $(\text{sid}, \cdot)$. On the other hand, the security game of STS enables the adversary to query $\mathcal{O}_{\text{PartS}}$ on any messages except the challenge message, which, in this case, must be in the form of $(\text{sid}, \cdot)$. Therefore, allowing the adversary to query $\mathcal{O}_{\widehat{\text{PartS}}}$ with messages not in the form of $(\text{sid}, \cdot)$ as well as $\mathcal{O}_{\widehat{\text{Corr}}}$ will not undermine the security of the instance PB[sid]. $\square$

Since the protocol Align is deterministic after the setup phase, it is easy to see that, once the inputs are known, all information an adversary can access in the instance Align[sid] can be generated by itself with the help of $\mathcal{O}_{\widehat{\text{PartS}}}$ and $\mathcal{O}_{\widehat{\text{Corr}}}$. We summarize this observation in the following lemma.

Lemma 16. *Let Align.Setup[sid] be an instance of the setup phase of Align. Let Align[(sid, sid₀)] be an instance under the setup. Then Align[(sid, sid₀)] is $\mathcal{O}_{\text{Align}}$ under Align.Setup[sid].*

Next, we show `Align` remains secure when it is used as a subroutine in `WeakCoin`.

Proposition 6. *`Align`[(`sid`, `ali`)] remains secure in `WeakCoin`[`sid`].*

Proof. It naturally follows Lemma 1. In particular, `WeakCoin`[`sid`] has no inputs, and all messages and states due to the code outside `Align`[(`sid`, `ali`)] are generated without using any information related to `Align`[(`sid`, `ali`)], except the outputs of `Align`[(`sid`, `ali`)], which, however, are public to the adversary $\mathcal{A}$ against `Align`[(`sid`, `ali`)]. Therefore, there is a PPT algorithm $\mathcal{S}$, which on input of the view $\text{VIEW}_{\mathcal{A}_{\text{Align}}}$ of $\mathcal{A}_{\text{Align}}$ can output VIEW' , such that $(\text{VIEW}_{\mathcal{A}_{\text{Align}}}, \text{VIEW}')$ distributes identically to the entire view of $\mathcal{A}$ in the composition. $\square$

Composition of the weak coin. We now demonstrate a few facts that are useful for proving security when composing our `WeakCoin` protocol with other protocol instances. We consider the following oracles:

- $\mathcal{O}_{\widehat{\text{PartD}}}$: On input (sid', i) , if $\text{sid}' \neq \text{sid}$, returns $\text{SHE.PartD}(ek, \text{sid}', dk_i)$
- $\mathcal{O}_{\widehat{\text{Sign}}}$: On input $(\text{sid}', i, \text{msg})$, if $\text{sid}' \neq \text{sid}$, returns $\text{DS.Sign}(sk_i, (\text{sid}', \text{msg}))$.
- Oracles in $\mathbb{O}_{\text{Align}}$, defined in Lemma 15 for the underlying `Align`.

We denote the set of the above oracles as well as the corruption oracle by $\mathbb{O}_{\text{WeakCoin}}$.

We have proved that our `WeakCoin` protocol is secure even when the adversary has access to oracles in $\mathbb{O}_{\text{WeakCoin}}$ in Theorem 2. Now we show an instance of `WeakCoin` is $\mathbb{O}_{\text{WeakCoin}}$ -emulatable under its setup.

Lemma 17. *Let `WeakCoin.Setup`[`sid`] be an instance of the setup. Let `WeakCoin`[(`sid`, `sid`₀)] be an instance under the setup of `WeakCoin.Setup`[`sid`]. Then, `WeakCoin`[(`sid`, `sid`₀)] is $\mathbb{O}_{\text{WeakCoin}}$ -emulatable under `WeakCoin.Setup`[`sid`].*

Proof. Given the public information from the `WeakCoin.Setup`[`sid`], in `WeakCoin`[(`sid`, `sid`₀)], the `FRESH` message sent by an honest node P_i can be generated by querying $\mathcal{O}_{\widehat{\text{PartD}}}$ and $\mathcal{O}_{\widehat{\text{Sign}}}$ with messages in the form of $(\langle \text{sid}, \text{sid}_0 \rangle, i, \cdot)$. Then, the input of P_i to `Align`[(`sid`, `sid`₀, `ali`)] is based on the `FRESH` messages it received, which in turn can be generated by querying $\mathcal{O}_{\widehat{\text{PartD}}}$ and $\mathcal{O}_{\widehat{\text{Sign}}}$ with messages in the form of $(\langle \text{sid}, \text{sid}_0 \rangle, i, \cdot)$. Then, as summarized in Lemma 16, the messages due to `Align`[(`sid`, `sid`₀, `ali`)] can be generated with the help of oracles in $\mathbb{O}_{\text{Align}}$. Similarly, all subsequent messages sent by P_i do not need secret keys of P_i , so they can be generated by a PPT algorithm on the input of previous messages. The internal states leaked to $\mathcal{A}$ due to corruption can also be generated with the help of the corruption oracle in $\mathbb{O}_{\text{WeakCoin}}$. Therefore, `WeakCoin`[(`sid`, `sid`₀)] is $\mathbb{O}_{\text{WeakCoin}}$ -emulatable under `WeakCoin.Setup`[`sid`].

Composition of MBA. Applying Theorem 2 and Lemma 17, we can prove that `WeakCoin` remains secure when it is used as a subroutine in `MBA`. In particular, we have the following lemma for the security of `MBA`.

Lemma 18. *Let `MBA`[(`sid`, `ba`)] be an instance under the setup `WeakCoin.Setup`[`sid`], while the $\{v_i\}_{i \in \mathcal{H}}$ is the input set of the initial honest parties. Then, if any instance of `WeakCoin` under the setup of `WeakCoin.Setup`[`sid`] is secure even when the adversary has access to $\{v_i\}_{i \in \mathcal{H}}$, then `MBA`[(`sid`, `ba`)] is secure.*

In addition, the security of `MBA`[(`sid`, `ba`)] preserved when the adversary has access to oracles in $\mathbb{O}_{\text{WeakCoin}}$ under the restriction that queries must not start with $\langle \text{sid}, \text{ba} \rangle$, and `MBA`[(`sid`, `ba`)] is $\mathbb{O}_{\text{WeakCoin}}$ -emulatable under `WeakCoin.Setup`[`sid`].

Proof. Let $\text{WeakCoin}[\langle \text{sid}, \text{sid}_0, \text{sid}' \rangle]$ be an instance inside $\text{MBA}[\langle \text{sid}, \text{sid}_0 \rangle]$. We can view $\text{MBA}[\langle \text{sid}, \text{sid}_0 \rangle]$ and its setup as a composition of

$$(\text{WeakCoin.Setup}[\text{sid}], \text{WeakCoin}[\langle \text{sid}, \text{sid}_0, \text{sid}' \rangle])$$

and the “remaining part” that includes the other instances of WeakCoin inside $\text{MBA}[\langle \text{sid}, \text{sid}_0 \rangle]$ and the coin-aided part of $\text{MBA}[\langle \text{sid}, \text{sid}_0 \rangle]$. The other instances of WeakCoin are $\mathbb{O}_{\text{WeakCoin}}$ -emulatable under $\text{WeakCoin.Setup}[\text{sid}]$, according to Lemma 17. Since the coin-aided part of $\text{MBA}[\langle \text{sid}, \text{sid}_0 \rangle]$ is information-theoretical and deterministic, all messages and states w.r.t. this part can be efficiently generated based on the inputs $\{v_i\}_{i \in \mathcal{H}}$ and the output of coin instances. Then, it is easy to see that the “remaining part” is $\mathbb{O}_{\text{WeakCoin}}$ -emulatable under $\text{WeakCoin.Setup}[\text{sid}]$. On the other hand, under the assumption that $\text{WeakCoin}[\langle \text{sid}, \text{sid}_0, \text{sid}' \rangle]$ is secure in the stand-alone setting even if the adversary has access to $\{v_i\}_{i \in \mathcal{H}}$ and $\mathbb{O}_{\text{WeakCoin}}$, following Lemma 1, $\text{MBA}[\langle \text{sid}, \text{sid}_0 \rangle]$ is secure. $\square$

Composition of leader election. Now, we show that all components in our leader election protocol are secure.

Proposition 7. *All instances of WeakCoin and MBA inside $\text{Election}[\text{sid}]$ remain secure.*

Proof. It follows the fact that all $\text{WeakCoin}[\langle \text{sid}, \text{wc}, k \rangle]$ and $\text{MBA}[\langle \text{sid}, \text{ba}, k \rangle]$ instances for $k = 1, \dots$ are $\mathbb{O}_{\text{WeakCoin}}$ -emulatable, and all honest inputs to each $\text{MBA}[\langle \text{sid}, \text{ba}, k \rangle]$ are outputs of $\text{WeakCoin}[\langle \text{sid}, \text{wc}, k \rangle]$ that are also $\mathbb{O}_{\text{WeakCoin}}$ -emulatable. Then, as ensured by Theorem 2 and Lemma 18, they all remain secure in the composition. $\square$

The following proposition states useful facts when using Election as a subroutine.

Proposition 8. *The security of $\text{Election}[\text{sid}]$ preserves even when the adversary has access to oracles in $\mathbb{O}_{\text{WeakCoin}}$ under the restriction that queries must not start with sid , and $\text{Election}[\text{sid}]$ is $\mathbb{O}_{\text{WeakCoin}}$ -emulatable under its setup.*

Proof. It trivially follows the security of the underlying components.

Composition of MVBA.

Lemma 19 (MVBA with Silent Setup). *Let MVBA be the protocol obtained by instantiating the PB and leader election subroutines in the MVBA protocol of [59] with instances of PB (Algorithm 5) and Election (Algorithm 3), respectively. Let $\text{WeakCoin.Setup}[\text{sid}]$ be an instance of the setup, and let $\text{MVBA}[\langle \text{sid}, \text{mvba} \rangle]$ be an instance under the setup with $\{v_i\}_{i \in \mathcal{H}}$ as the inputs of honest nodes. Then, if any instance of PB and any instance of Election under the setup of $\text{WeakCoin.Setup}[\text{sid}]$ are secure even when the adversary has access to $\{v_i\}_{i \in \mathcal{H}}$ and oracles in $\mathbb{O}_{\text{WeakCoin}}$, $\text{MVBA}[\langle \text{sid}, \text{mvba} \rangle]$ is secure.*

In addition, the security of $\text{MVBA}[\langle \text{sid}, \text{mvba} \rangle]$ is preserved when the adversary has access to oracles in $\mathbb{O}_{\text{WeakCoin}}$ under the restriction that queries must not start with $\langle \text{sid}, \text{mvba} \rangle$, and $\text{MVBA}[\langle \text{sid}, \text{mvba} \rangle]$ is $\mathbb{O}_{\text{WeakCoin}}$ -emulatable under $\text{WeakCoin.Setup}[\text{sid}]$.

Proof. For any $\text{PB}[\langle \text{sid}, \text{mvba}, \text{pb}_0 \rangle]$ instance inside $\text{MVBA}[\langle \text{sid}, \text{mvba} \rangle]$, we can view $\text{MVBA}[\langle \text{sid}, \text{mvba} \rangle]$ as a composition of

$$(\text{WeakCoin.Setup}[\text{sid}], \text{PB}[\langle \text{sid}, \text{mvba}, \text{pb}_0 \rangle])$$

and the remaining part, including other instances of PB and Election and others, that are $\mathbb{O}_{\text{WeakCoin}}$ -emulatable. So, $\text{PB}[\langle \text{sid}, \text{mvba}, \text{pb}_0 \rangle]$ remains secure in the composition, if it is secure when the adversary has access to $\mathbb{O}_{\text{WeakCoin}}$ and $\{v_i\}_{i \in \mathcal{H}}$. Similar arguments apply to any instance of Election inside $\text{MVBA}[\langle \text{sid}, \text{mvba} \rangle]$. Hence, $\text{MVBA}[\langle \text{sid}, \text{mvba} \rangle]$ is secure under the assumption. It follows trivially that $\text{MVBA}[\langle \text{sid}, \text{mvba} \rangle]$ is secure even when the adversary has access to $\mathbb{O}_{\text{WeakCoin}}$ and that it is $\mathbb{O}_{\text{WeakCoin}}$ -emulatable. $\square$

F Sketch of ADKG Protocol in [42]

The execution flow of the coin-aided ADKG protocol is illustrated in Fig.10. The figure is directly from [42].

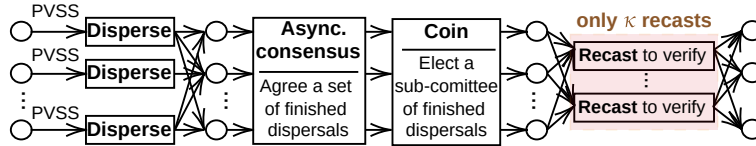


Fig. 10. High-level execution flow of the coin-aided ADKG in [42].

Ingredients. This protocol relies on the following key components.

First, a publicly verifiable secret sharing (PVSS) scheme [11, 34] allows a dealer to generate a publicly verifiable sharing transcript. Provided that all honest nodes observe the same valid transcript, they can obtain correct secret shares of the dealer’s secret. Second, an asynchronous provable dispersal (APD) protocol [72] enables a sender to disperse a long message of $\ell > n$ bits across the network with a communication cost of $O(\ell + n\lambda)$. Once dispersal is completed, the sender receives a proof of completion. As long as this proof exists, all honest nodes can recover the sender’s input during the recast phase with a total communication cost of $O(n\ell + n^2\lambda)$. This can be implemented with a silent setup threshold signature (see Appendix B.3). Third, the protocol uses a multi-valued validated Byzantine agreement (MVBA) protocol. In this work, we adopt the Dumbo-MVBA protocol [72], which incurs a cost of $O(n\ell + n^2\lambda)$ to agree on an ℓ -bit value among n nodes. Dumbo-MVBA is implemented using a strong common coin and a silent setup threshold signature. Lastly, the protocol requires a strong common coin mechanism.

With our common coin protocol, all components can be instantiated under a silent setup.

Protocol Overview. The protocol progresses in several phases:

Initially, each node generates a PVSS transcript of size $O(n\lambda)$. Next, all nodes invoke separate APD instances to disperse their PVSS transcripts. This phase incurs a total communication cost of $O(n^2\lambda)$. Upon obtaining the dispersal proof, each sender multicasts this proof to the network. The network then uses the MVBA protocol to agree on $n - f$ APD instances that have valid dispersal proofs, incurring an additional communication cost of $O(n^2\lambda)$.

After this, a strong common coin is employed to select κ instances from the agreed $n - f$ APD instances. The network then recasts these selected κ instances to ensure that all honest nodes obtain the same set of valid PVSS transcripts, with a communication complexity of $O(\kappa n^2 \lambda)$. Finally, each node derives its secret key share from the aggregated secret values in the recast PVSS transcripts. The security of the DKG protocol holds since, with probability $1 - 2^{-\kappa}$, at least one of the κ recast transcripts is generated by a forever-honest node, ensuring that the corresponding secret remains unknown to the adversary.

Adaptive Security Consideration. To achieve adaptive security, the protocol can use either the PVSS scheme from [8], which supports group-element secrets in a pairing-friendly ECC group, or the interactive PVSS scheme from [34], which handles secrets in a conventional scalar field. A detailed construction and adaptive security proof is provided in [42, Appendix A].